ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ

Τμήμα Πληροφορικής



ΠΤΥΧΙΑΚΗ ΕΡΓΑΣΙΑ

Εφαρμογή Διαχείρισης Κρατήσεων και Προγραμμάτων Γυμναστηρίου

Ρίμπας Χρήστος του Αριστοτέλη, Π18133

Επιβλέπων: Καθηγητής Ε. Σακκόπουλος

Πειραιάς, Σεπτέμβριος 2026



Περίληψη

Πολλά μικρά και μεσαία γυμναστήρια διαχειρίζονται ακόμη τις κρατήσεις των μαθημάτων τους χειροκίνητα, με αποτέλεσμα ασάφεια για τις διαθέσιμες θέσεις, υπερβάσεις της χωρητικότητας και επιβάρυνση του προσωπικού. Η παρούσα πτυχιακή εργασία παρουσιάζει τον σχεδιασμό, την υλοποίηση και την αξιολόγηση μιας διαδικτυακής εφαρμογής για τη διαχείριση κρατήσεων και προγραμμάτων γυμναστηρίου, η οποία εξυπηρετεί πολλά γυμναστήρια από την ίδια εγκατάσταση (multi-tenant).

Η εφαρμογή αποτελείται από ένα REST API σε ASP.NET Core με βάση δεδομένων PostgreSQL και από δύο εφαρμογές Angular: μία για τα μέλη, σχεδιασμένη κυρίως για κινητά, και μία για το προσωπικό. Τα μέλη βλέπουν το εβδομαδιαίο πρόγραμμα, κάνουν και ακυρώνουν κρατήσεις και μπαίνουν σε λίστα αναμονής, ενώ οι εκπαιδευτές και οι διαχειριστές οργανώνουν τα μαθήματα, τους χρήστες και τις συνδρομές. Τα δεδομένα όλων των γυμναστηρίων βρίσκονται σε κοινή βάση και διαχωρίζονται αυτόματα με βάση το αναγνωριστικό που περιέχει το token σύνδεσης, ενώ η κράτηση εκτελείται μέσα σε transaction με κλείδωμα της εγγραφής του μαθήματος. Ολόκληρο το σύστημα εκτελείται τοπικά με μία εντολή μέσω Docker.

Το ερευνητικό ερώτημα ήταν αν μια τέτοια εφαρμογή μπορεί να αποκλείει την υπέρβαση της χωρητικότητας υπό ταυτόχρονες κρατήσεις, διατηρώντας αποδεκτό χρόνο απόκρισης και πλήρη απομόνωση των δεδομένων κάθε γυμναστηρίου. Για την απάντηση πραγματοποιήθηκαν ελεγχόμενα πειράματα σε πραγματική PostgreSQL, σε σύγκριση με μια απλή υλοποίηση χωρίς κλείδωμα, μετρήσεις του χρόνου απόκρισης υπό αυξανόμενο φορτίο, δοκιμές απομόνωσης των δεδομένων και έλεγχος προσβασιμότητας.

Το συμπέρασμα είναι ότι η προσέγγιση ικανοποιεί τις απαιτήσεις για τα μεγέθη ενός γυμναστηρίου, με κόστος χρόνου απόκρισης που αυξάνεται γραμμικά μόνο όταν πολλοί χρήστες στοχεύουν ταυτόχρονα το ίδιο μάθημα.

Λέξεις-κλειδιά: διαχείριση κρατήσεων, multi-tenancy, concurrency control, web εφαρμογή, ASP.NET Core, Angular



Πίνακας Περιεχομένων

Περίληψη 2
1. Εισαγωγή 8
1.1 Αντικείμενο και σκοπός της εργασίας 8
1.2 Περιγραφή του προβλήματος και κίνητρο 8
1.3 Στόχοι και συνεισφορά 8
1.4 Ερευνητικό ερώτημα 9
1.5 Βασικοί ορισμοί 9
1.6 Εύρος και περιορισμοί 11
1.7 Παραδοτέα της εργασίας 12
1.8 Δομή της εργασίας 12
2. Υπόβαθρο και σχετική εργασία 13
2.1 Επιστημονικό υπόβαθρο 13
2.1.1 Software as a Service (SaaS) και multi-tenancy 13
2.1.2 Transactions και concurrency control 13
2.1.3 Αρχιτεκτονική και ασφάλεια web εφαρμογών 14
2.2 Υπάρχουσες λύσεις 14
2.2.1 Green Fitness Club 14
2.2.2 Planet Fitness & More 15
2.2.3 Wellness Hub Fitness Studio 16
2.3 Κριτική και σύγκριση 17
3. Ανάλυση απαιτήσεων και σχεδίαση 19
3.1 Ρόλοι χρηστών (πελάτης / προσωπικό / διαχειριστής) 19
3.2 Λειτουργικές απαιτήσεις 19
3.3 Μη λειτουργικές απαιτήσεις 20
3.4 Διάγραμμα περιπτώσεων χρήσης (Use Case) 21
3.5 Περιγραφή περιπτώσεων χρήσης 22
3.6 Σχεδίαση της διεπαφής χρήστη (wireframes) 27
3.6.1 Καταστάσεις της διεπαφής 31
3.6.2 Από τα wireframes στην τελική εφαρμογή 31
3.7 Σχεδίαση βάσης δεδομένων (ERD) 32
4. Αρχιτεκτονική και υλοποίηση 33
4.1 Γενική αρχιτεκτονική (frontend / backend / βάση) 33
4.1.1 Πλαίσιο συστήματος (system context) 33
4.1.2 Containers και επικοινωνία 33

4.2 Multi-tenancy 34
4.3 Ταυτοποίηση και εξουσιοδότηση (JWT, ρόλοι) 35
4.4 Τεχνολογίες που χρησιμοποιήθηκαν 35
4.4.1 Frontend (Angular, Nx, Angular Material, Tailwind CSS) 36
4.4.2 Backend (.NET, ASP.NET Core, EF Core, ASP.NET Core Identity) 36
4.4.3 Βάση δεδομένων (PostgreSQL) 36
4.4.4 Υποδομή και εργαλεία (Docker, Git/GitHub) 36
4.5 Ροές εκτέλεσης βασικών λειτουργιών 37
4.5.1 Εγγραφή και σύνδεση χρήστη (invitation / registration) 37
4.5.2 Κράτηση θέσης σε μάθημα (atomic booking) 38
4.5.3 Λίστα αναμονής (waitlist) 38
4.5.4 Ακύρωση κράτησης και πολιτική ακύρωσης 39
4.6 Ιδιαίτερα απαιτητικά χαρακτηριστικά 40
4.6.1 Ατομικές κρατήσεις χωρίς overbooking 40
4.6.2 Λίστα αναμονής και αυτόματη προαγωγή 41
4.6.3 Πολιτική ακυρώσεων 42
4.7 Ασφάλεια 42
4.7.1 Προστασία της διαδικασίας σύνδεσης (account lockout, rate limiting) 42
4.7.2 Αποθήκευση κωδικών και tokens προσκλήσεων (hashing) 42
4.7.3 Απομόνωση δεδομένων ανά tenant 42
4.7.4 CORS και ασφάλεια επικοινωνίας client–API 42
4.7.5 Ιδιωτικότητα και προσωπικά δεδομένα (GDPR) 43
4.7.6 Όρια εμπιστοσύνης (trust boundaries) 43
4.8 Εκτέλεση με Docker (deployment) 43
4.9 Αρχιτεκτονικές αποφάσεις 44
4.10 Γνωστά ζητήματα 45
5. Αξιολόγηση 46
5.1 Λειτουργικός έλεγχος (unit tests, QA) 46
5.2 Πρωτόκολλο αξιολόγησης 46
5.3 Αποτελέσματα 47
5.3.1 ΕΕ1: υπερκρατήσεις με και χωρίς κλείδωμα 47
5.3.2 ΕΕ2: χρόνος απόκρισης υπό ταυτοχρονισμό 48
5.3.3 ΕΕ3: απομόνωση των δεδομένων 49
5.3.4 Προσβασιμότητα 49
5.3.5 Σύνοψη 50
5.4 Απειλές εγκυρότητας 50

6. Οδηγός εγκατάστασης και χρήσης 51
6.1 Προαπαιτούμενα 51
6.2 Εκτέλεση με Docker 51
6.3 Διευθύνσεις και έτοιμοι λογαριασμοί 51
6.4 Δημιουργία και διαχείριση νέων χρηστών 52
6.5 Σταμάτημα και καθαρή εκκίνηση 52
6.6 Εκτέλεση για ανάπτυξη (χωρίς Docker) 52
6.7 Μεταβλητές περιβάλλοντος 53
6.8 Συχνά προβλήματα 53
6.9 Χρήση της εφαρμογής 54
6.9.1 Λειτουργίες πελάτη (customer app) 54
6.9.2 Λειτουργίες προσωπικού και διαχειριστή (staff app) 56
7. Συμπεράσματα και μελλοντική εργασία 59
7.1 Συμπεράσματα 59
7.2 Περιορισμοί 59
7.3 Δυσκολίες που αντιμετωπίστηκαν 60
7.4 Μελλοντικές επεκτάσεις 60
Βιβλιογραφία 61
Παράρτημα Α — Επισκόπηση των API endpoints 64
Α.1 Ταυτοποίηση (authentication) 64
Α.2 Πρόγραμμα και μαθήματα 64
Α.3 Κρατήσεις και λίστα αναμονής 64
Α.4 Συνδρομές και πλάνα 65
Α.5 Διαχείριση χρηστών, προσκλήσεων και ρυθμίσεων 65
Παράρτημα Β — Συντομογραφίες 66
Παράρτημα Γ — Δήλωση χρήσης εργαλείων τεχνητής νοημοσύνης 67

Κατάλογος Εικόνων

Εικόνα 1 — *Green Fitness Club: πρόγραμμα και κράτηση μαθήματος.* 15
Εικόνα 2 — *Planet Fitness (TrainYourPulse): κράτηση με επιλογή μαθήματος, προπονητή και πόντους επιβράβευσης.* 16
Εικόνα 3 — *Wellness Hub: εβδομαδιαίο πρόγραμμα και συνδρομή.* 17
Εικόνα 4 — *Wireframe: οθόνη σύνδεσης (login).* 28
Εικόνα 5 — *Wireframe: αρχική οθόνη πελάτη.* 29
Εικόνα 6 — *Wireframe: εβδομαδιαίο πρόγραμμα και κράτηση.* 29
Εικόνα 7 — *Wireframe: πίνακας προσωπικού (dashboard).* 30
Εικόνα 8 — *Wireframe: διαχείριση ωρών μαθημάτων.* 30
Εικόνα 9 — *Wireframe: διαχείριση χρηστών.* 31
Εικόνα 10 — *Αρχική οθόνη πελάτη: συνδρομή και επερχόμενες κρατήσεις.* 54
Εικόνα 11 — *Εβδομαδιαίο πρόγραμμα και κράτηση μαθήματος.* 55
Εικόνα 12 — *Ιστορικό κρατήσεων πελάτη.* 56
Εικόνα 13 — *Πίνακας προσωπικού (dashboard διαχειριστή).* 57
Εικόνα 14 — *Διαχείριση ωρών μαθημάτων και πρόσβαση στο παρουσιολόγιο.* 58
Εικόνα 15 — *Διαχείριση χρηστών και ρόλων.* 58

Κατάλογος Διαγραμμάτων

Διάγραμμα 1 — *Περιπτώσεις χρήσης (Use Case).* 22
Διάγραμμα 2 — *Χάρτης ροής οθονών των δύο εφαρμογών.* 28
Διάγραμμα 3 — *Οντοτήτων–συσχετίσεων (ERD).* 32
Διάγραμμα 4 — *Πλαίσιο συστήματος (C4 – system context).* 33
Διάγραμμα 5 — *Αρχιτεκτονική του συστήματος.* 34
Διάγραμμα 6 — *Ροή εγγραφής μέσω πρόσκλησης και σύνδεσης.* 37
Διάγραμμα 7 — *Ροή ατομικής κράτησης θέσης (atomic booking).* 38
Διάγραμμα 8 — *Ροή εισόδου στη λίστα αναμονής.* 39
Διάγραμμα 9 — *Ροή ακύρωσης κράτησης με πολιτική και αυτόματη προαγωγή.* 40
Διάγραμμα 10 — *Deployment: τοπική εκτέλεση ολόκληρου του συστήματος με Docker Compose.* 44
Διάγραμμα 11 — *EE1: επιβεβαιωμένες κρατήσεις με και χωρίς κλείδωμα (C = 5, μέσος όρος 10 επαναλήψεων).* 48
Διάγραμμα 12 — *EE2: p95 του χρόνου απόκρισης ανά σενάριο και αριθμό ταυτόχρονων χρηστών.* 49

Κατάλογος Πινάκων

Πίνακας 1 — *Συγκριτική αξιολόγηση χαρακτηριστικών με τις υπάρχουσες εφαρμογές.* 17
Πίνακας 2 — *Λειτουργικές απαιτήσεις (FR).* 19
Πίνακας 3 — *Μη λειτουργικές απαιτήσεις (NFR) και κριτήρια επαλήθευσης.* 20
Πίνακας 4 — *Περίπτωση χρήσης UC-01: Εγγραφή μέσω πρόσκλησης και σύνδεση.* 22
Πίνακας 5 — *Περίπτωση χρήσης UC-02: Κράτηση θέσης σε μάθημα.* 23
Πίνακας 6 — *Περίπτωση χρήσης UC-03: Ακύρωση κράτησης.* 24
Πίνακας 7 — *Περίπτωση χρήσης UC-04: Είσοδος σε λίστα αναμονής.* 25
Πίνακας 8 — *Περίπτωση χρήσης UC-05: Δημιουργία και διαχείριση μαθήματος.* 26
Πίνακας 9 — *Περίπτωση χρήσης UC-06: Ανάθεση συνδρομής σε χρήστη.* 27

Πίνακας 10 — *Τεχνολογίες, εκδόσεις και αιτιολόγηση της επιλογής τους.* 35
Πίνακας 11 — *Διαφορές ανάμεσα στην εκτέλεση για ανάπτυξη και την εκτέλεση με Docker.* 44
Πίνακας 12 — *Αρχιτεκτονική απόφαση AA-1: Μοντέλο multi-tenancy.* 44
Πίνακας 13 — *Αρχιτεκτονική απόφαση AA-2: Έλεγχος ταυτοχρονισμού στις κρατήσεις.* 45
Πίνακας 14 — *Αρχιτεκτονική απόφαση AA-3: Μηχανισμός ταυτοποίησης.* 45
Πίνακας 15 — *Αρχιτεκτονική απόφαση AA-4: Οργάνωση του frontend.* 45
Πίνακας 16 — *Αυτοματοποιημένες δοκιμές ανά περιοχή λειτουργικότητας.* 46
Πίνακας 17 — *EE1: αποτελέσματα ταυτόχρονων κρατήσεων σε μάθημα χωρητικότητας C = 5 (10 επαναλήψεις ανά γραμμή).* 47
Πίνακας 18 — *EE2: p95 του χρόνου απόκρισης σε ms (σε παρένθεση η τυπική απόκλιση ανάμεσα στις 5 επαναλήψεις).* 48
Πίνακας 19 — *Βαθμολογίες Lighthouse Accessibility ανά οθόνη.* 49
Πίνακας 20 — *Σύνοψη επαλήθευσης των μη λειτουργικών απαιτήσεων.* 50
Πίνακας 21 — *Έτοιμοι λογαριασμοί της τοπικής εκτέλεσης (Docker).* 51
Πίνακας 22 — *Διευθύνσεις των υπηρεσιών στην τοπική εκτέλεση.* 51
Πίνακας 23 — *Μεταβλητές περιβάλλοντος του API.* 53
Πίνακας 24 — *Endpoints ταυτοποίησης.* 64
Πίνακας 25 — *Endpoints προγράμματος και μαθημάτων.* 64
Πίνακας 26 — *Endpoints κρατήσεων και λίστας αναμονής.* 64
Πίνακας 27 — *Endpoints συνδρομών και συνδρομητικών πλάνων.* 65
Πίνακας 28 — *Endpoints διαχείρισης χρηστών, προσκλήσεων και ρυθμίσεων.* 65
Πίνακας 29 — *Συντομογραφίες.* 66

Κατάλογος Κωδίκων

Κώδικας 1 — *Επιβολή της απομόνωσης ανά γυμναστήριο (C#, backend/src/GymBooking.Api/Services/TenantMiddleware.cs και backend/src/GymBooking.Data/AppDbContext.cs, γρ. 34–41).* 35
Κώδικας 2 — Ατομική κράτηση με κλείδωμα γραμμής (C#, backend/src/GymBooking.Api/Services/BookingService.cs, γρ. 23–88, συντομευμένο).41 41
Κώδικας 3 — *Κλείδωμα λογαριασμού και περιορισμός ρυθμού στη σύνδεση (C#, backend/src/GymBooking.Api/Program.cs, γρ. 56–74 και 108–114, συντομευμένο).* 42

1. Εισαγωγή

Η ευρεία χρήση των smartphones και των διαδικτυακών υπηρεσιών έχει αλλάξει τον τρόπο με τον οποίο οι άνθρωποι οργανώνουν τις καθημερινές τους δραστηριότητες, συμπεριλαμβανομένης της γυμναστικής. Οι χρήστες θεωρούν πλέον δεδομένο ότι μπορούν να ενημερώνονται και να εξυπηρετούνται ηλεκτρονικά, χωρίς να εξαρτώνται από τηλεφωνική επικοινωνία ή φυσική παρουσία. Στο πλαίσιο αυτό, η παρούσα εργασία ασχολείται με την ανάπτυξη μιας εφαρμογής που κάνει πιο εύκολη, μέσω smartphone ή Η/Υ, τη διαδικασία των κρατήσεων σε ένα γυμναστήριο. Στο παρόν κεφάλαιο περιγράφονται το αντικείμενο και ο σκοπός της εργασίας, το πρόβλημα που αντιμετωπίζεται, οι στόχοι και το ερευνητικό ερώτημα, οι βασικοί ορισμοί, το εύρος και οι περιορισμοί της εργασίας, καθώς και τα παραδοτέα της.

1.1 Αντικείμενο και σκοπός της εργασίας

Αντικείμενο της παρούσας πτυχιακής εργασίας είναι ο σχεδιασμός και η ανάπτυξη μιας ολοκληρωμένης διαδικτυακής εφαρμογής για τη διαχείριση κρατήσεων και προγραμμάτων γυμναστηρίου. Η εφαρμογή απευθύνεται σε δύο κατηγορίες χρηστών: τα μέλη του γυμναστηρίου, που βλέπουν το πρόγραμμα και κάνουν κρατήσεις σε μαθήματα, και το προσωπικό (εκπαιδευτές και διαχειριστές), που οργανώνει τα μαθήματα και διαχειρίζεται τους χρήστες και τις συνδρομές.

Η εφαρμογή σχεδιάστηκε ώστε να μπορεί να εξυπηρετεί περισσότερα από ένα γυμναστήρια από την ίδια εγκατάσταση, με πλήρη διαχωρισμό των δεδομένων του καθενός (multi-tenant). Έτσι, η εργασία δεν αποτελεί μόνο μια πτυχιακή εργασία, αλλά και μια βάση για ένα πραγματικό προϊόν λογισμικού.

Σκοπός της εργασίας είναι να καλυφθεί ολόκληρος ο κύκλος ανάπτυξης μιας σύγχρονης εφαρμογής: από την ανάλυση των απαιτήσεων και τον σχεδιασμό της αρχιτεκτονικής, μέχρι την υλοποίηση, τον έλεγχο, την αξιολόγηση και την εκτέλεσή της με Docker. Παράλληλα, έγινε προσπάθεια να βρεθεί λύση σε ζητήματα που εμφανίζονται σε πραγματικές εφαρμογές, όπως η ταυτόχρονη πρόσβαση πολλών χρηστών, η ασφάλεια και η σωστή διαχείριση των κρατήσεων, ώστε να μην ξεπερνιέται ποτέ η χωρητικότητα ενός μαθήματος.

1.2 Περιγραφή του προβλήματος και κίνητρο

Πολλά μικρά και μεσαία γυμναστήρια εξακολουθούν να διαχειρίζονται τις κρατήσεις τους τηλεφωνικά, με μηνύματα ή σε χαρτί. Ωστόσο με αυτούς τους τρόπους δεν υπάρχει σαφής εικόνα για το πόσες θέσεις απομένουν σε κάθε μάθημα, εύκολα προκύπτουν υπερβάσεις της χωρητικότητας, ενώ για την ακύρωση ή η αλλαγή μιας κράτησης πρέπει ο πελάτης να επικοινωνήσει με το γυμναστήριο.

Επίσης, τα μέλη σήμερα περιμένουν να μπορούν να δουν το πρόγραμμα και να κλείσουν τη θέση τους, από το κινητό τους, οποιαδήποτε ώρα. Από την πλευρά του γυμναστηρίου, χρειάζεται ένα απλό εργαλείο για να οργανώνει τα μαθήματα, να παρακολουθεί τις συμμετοχές και να διαχειρίζεται τις συνδρομές χωρίς απαραίτητα την ύπαρξη γραμματείας.

Κίνητρο της εργασίας ήταν να καλυφθεί αυτή η ανάγκη με μια σύγχρονη εφαρμογή, η οποία αυτοματοποιεί τις κρατήσεις, κρατά αξιόπιστα τη διαθεσιμότητα των θέσεων και υποστηρίζει λειτουργίες όπως η λίστα αναμονής και η πολιτική ακυρώσεων. Η αρχιτεκτονική πολλαπλών γυμναστηρίων (multi-tenant) προστέθηκε ώστε η ίδια λύση να μπορεί να αξιοποιηθεί από περισσότερες επιχειρήσεις, κάτι που είναι κοντά στον τρόπο λειτουργίας ενός πραγματικού προϊόντος λογισμικού ως υπηρεσία (SaaS) [1].

1.3 Στόχοι και συνεισφορά

Οι βασικοί στόχοι που τέθηκαν για την εργασία είναι οι εξής:

- Ο σχεδιασμός και η υλοποίηση μιας πλήρους εφαρμογής κρατήσεων, από τη βάση δεδομένων και το backend μέχρι τις εφαρμογές(frontend) που χρησιμοποιεί ο τελικός χρήστης.
- Η υποστήριξη διαφορετικών ρόλων (μέλος, εκπαιδευτής, διαχειριστής), με ξεχωριστά δικαιώματα για τον καθένα.
- Η διαχείριση των κρατήσεων ακόμη και όταν πολλοί χρήστες κάνουν κρατήσεις ταυτόχρονα, ώστε να μην ξεπερνιέται ποτέ η χωρητικότητα ενός μαθήματος.
- Η κάλυψη αναγκών ενός γυμναστηρίου, όπως οι συνδρομές, η λίστα αναμονής και η πολιτική ακυρώσεων.
- Η προστασία των δεδομένων και της πρόσβασης, με ασφαλή σύνδεση, ρόλους και διαχωρισμό των δεδομένων κάθε γυμναστηρίου.
- Η δυνατότητα εκτέλεσης ολόκληρου του συστήματος τοπικά, με μία εντολή, μέσω Docker.

Η εργασία είναι ένα ολοκληρωμένο παράδειγμα διαδικτυακής εφαρμογής, που αντιμετωπίζει ζητήματα που συναντώνται σε πραγματικά συστήματα. Έμφαση δόθηκε στη σωστή διαχείριση της ταυτόχρονης πρόσβασης, στην οργάνωση του κώδικα σε επίπεδα και στην ασφάλεια.

1.4 Ερευνητικό ερώτημα

Το κεντρικό ζήτημα που εξετάζει η εργασία είναι αν μια διαδικτυακή εφαρμογή κρατήσεων, η οποία εξυπηρετεί πολλά γυμναστήρια από την ίδια εγκατάσταση, μπορεί να λειτουργεί αξιόπιστα όταν πολλοί χρήστες ενεργούν ταυτόχρονα. Το ερευνητικό ερώτημα διατυπώνεται ως εξής:

Μπορεί μια multi-tenant εφαρμογή κρατήσεων, με κοινή βάση δεδομένων και διαχωρισμό των δεδομένων ανά γυμναστήριο, να αποκλείει την υπέρβαση της χωρητικότητας των μαθημάτων σε ταυτόχρονες κρατήσεις, έχοντας παράλληλα μικρό χρόνο απόκρισης και απομόνωση των δεδομένων κάθε γυμναστηρίου;

Το ερώτημα αναλύεται σε τρία ερωτήματα:

- **ΕΕ1:** Πόσο καλά προστατεύει από το να γεμίσει ένα μάθημα πάνω από τις θέσεις του το να γίνεται η κράτηση μέσα σε transaction που κλειδώνει την εγγραφή του μαθήματος, σε σύγκριση με μια απλή υλοποίηση χωρίς κλείδωμα;
- **ΕΕ2:** Πόσο πιο αργή γίνεται η κράτηση εξαιτίας του κλειδώματος, όσο αυξάνονται τα αιτήματα που φτάνουν την ίδια στιγμή;
- **ΕΕ3:** Μπορούμε να είμαστε σίγουροι ότι κανένας χρήστης δεν μπορεί να δει ή να αλλάξει δεδομένα άλλου γυμναστηρίου, όταν τα δεδομένα χωρίζονται με βάση τον κωδικό του γυμναστηρίου που παίρνουμε από το token σύνδεσης;

Οι απαντήσεις στα ερωτήματα αυτά δίνονται στο κεφάλαιο 5, με μετρήσεις και αυτόματα τεστ. Για το ΕΕ1 γίνεται ένα πείραμα όπου πολλοί χρήστες κλείνουν την ίδια στιγμή θέση στο ίδιο μάθημα, μία φορά με κλείδωμα και μία χωρίς. Για το ΕΕ2 μετριέται ο χρόνος απάντησης όσο αυξάνονται τα αιτήματα. Για το ΕΕ3 εκτελούνται τεστ που προσπαθούν να δουν ή να αλλάξουν δεδομένα άλλου γυμναστηρίου.

1.5 Βασικοί ορισμοί

Για να γίνει πιο εύκολα κατανοητή η εργασία, σε αυτή την ενότητα εξηγούνται οι βασικές έννοιες και οι τεχνικοί όροι που χρησιμοποιούνται στη συνέχεια.

- **Διαδικτυακή εφαρμογή (web application):** εφαρμογή που εκτελείται μέσω ενός φυλλομετρητή (browser), χωρίς εγκατάσταση στον υπολογιστή του χρήστη.



- **Αρχιτεκτονική πελάτη–εξυπηρετητή (client–server):** μοντέλο όπου ο πελάτης (η εφαρμογή του χρήστη) στέλνει αιτήματα και ο εξυπηρετητής (ο διακομιστής) τα επεξεργάζεται και απαντά.
- **Frontend και backend:** το frontend είναι το μέρος με το οποίο αλληλεπιδρά ο χρήστης, το backend είναι η πλευρά του διακομιστή που εκτελεί τη λογική και διαχειρίζεται τα δεδομένα.
- **REST API:** τρόπος επικοινωνίας μεταξύ εφαρμογών μέσω του πρωτοκόλλου HTTP, όπου κάθε πόρος (π.χ. μάθημα, κράτηση) προσπελαύνεται με τυποποιημένες μεθόδους (GET, POST κ.λπ.) [2].
- **Endpoint (σημείο πρόσβασης):** μια συγκεκριμένη διεύθυνση του API που εκτελεί μια ενέργεια, για παράδειγμα τη δημιουργία μιας κράτησης.
- **JSON:** ελαφριά μορφή αναπαράστασης δεδομένων σε κείμενο, που χρησιμοποιείται για την ανταλλαγή δεδομένων μεταξύ πελάτη και εξυπηρετητή.
- **Λογισμικό ως υπηρεσία (SaaS):** μοντέλο διάθεσης λογισμικού όπου η εφαρμογή προσφέρεται κεντρικά μέσω διαδικτύου και εξυπηρετεί πολλούς πελάτες.
- **Μισθωτής (tenant):** ένας ανεξάρτητος πελάτης της πλατφόρμας — εδώ, ένα γυμναστήριο — του οποίου τα δεδομένα είναι απομονωμένα από εκείνα των υπολοίπων.
- **Πολυμισθωτικότητα (multi-tenancy):** σχεδίαση όπου μία εγκατάσταση εξυπηρετεί πολλούς μισθωτές (γυμναστήρια), με πλήρη διαχωρισμό των δεδομένων τους [1].
- **Ρόλος (role):** κατηγορία χρήστη που καθορίζει τα δικαιώματά του, στην εφαρμογή υπάρχουν τρεις ρόλοι: μέλος, εκπαιδευτής και διαχειριστής.
- **Ταυτοποίηση (authentication) και εξουσιοδότηση (authorization):** η ταυτοποίηση επιβεβαιώνει ποιος είναι ο χρήστης (σύνδεση), ενώ η εξουσιοδότηση καθορίζει τι του επιτρέπεται να κάνει, με βάση τον ρόλο του.
- **JWT (JSON Web Token):** υπογεγραμμένο ψηφιακό διακριτικό που εκδίδεται κατά τη σύνδεση και συνοδεύει κάθε αίτημα, πιστοποιώντας την ταυτότητα και τους ρόλους του χρήστη [3].
- **Σχεσιακή βάση δεδομένων:** βάση που οργανώνει τα δεδομένα σε πίνακες με σαφείς σχέσεις μεταξύ τους (εδώ, η PostgreSQL) [4].
- **ORM (Object-Relational Mapping):** εργαλείο που αντιστοιχίζει τα αντικείμενα του κώδικα σε εγγραφές της βάσης, ώστε ο προγραμματιστής να μη γράφει απευθείας SQL (εδώ, το Entity Framework Core) [5].
- **Transaction:** σύνολο ενεργειών στη βάση που εκτελούνται ως ενιαία, αδιαίρετη μονάδα, είτε ολοκληρώνονται όλες, είτε καμία [6].
- **Κλείδωμα εγγραφής (locking):** προσωρινή δέσμευση μιας εγγραφής κατά τη διάρκεια ενός transaction, ώστε να μην την τροποποιούν ταυτόχρονα άλλοι χρήστες [7].
- **Μετάπτωση σχήματος (migration):** ελεγχόμενη αλλαγή στη δομή της βάσης δεδομένων, ώστε αυτή να συμβαδίζει με τον κώδικα.
- **Τύπος μαθήματος (class type):** το είδος του μαθήματος, για παράδειγμα Yoga ή CrossFit, με προεπιλεγμένη διάρκεια και χωρητικότητα.
- **Μάθημα / συνεδρία (class session):** ένα συγκεκριμένο μάθημα σε ορισμένη ημερομηνία και ώρα, με εκπαιδευτή, χωρητικότητα και αριθμό κρατήσεων.
- **Κράτηση (booking):** η δέσμευση θέσης ενός μέλους σε μια συνεδρία.

- **Λίστα αναμονής (waitlist):** μηχανισμός όπου, όταν ένα μάθημα είναι γεμάτο, οι χρήστες μπαίνουν σε ουρά και προάγονται αυτόματα όταν ελευθερωθεί θέση.
- **Συνδρομή (subscription):** η ενεργή συνδρομή ενός μέλους, βάσει ενός πλάνου, η οποία καταναλώνεται στις κρατήσεις.
- **Συνδρομητικό πλάνο (membership plan):** ο ορισμός μιας συνδρομής, για παράδειγμα απεριόριστο μηνιαίο ή πακέτο ορισμένων συνεδριών.
- **Παρουσιολόγιο (roster):** η λίστα των εγγεγραμμένων μελών μιας συνεδρίας, ορατή στο προσωπικό.
- **Πρόσκληση (invitation):** σύνδεσμος με μοναδικό διακριτικό για την εγγραφή νέου χρήστη, τον οποίο εκδίδει ο διαχειριστής.
- **Πολιτική ακυρώσεων (cancellation policy):** ο κανόνας που ορίζει το ελάχιστο χρονικό περιθώριο πριν από ένα μάθημα, μέσα στο οποίο δεν επιτρέπεται η ακύρωση από το μέλος.
- **CORS:** μηχανισμός ασφαλείας που ελέγχει ποιες εξωτερικές διευθύνσεις (origins) επιτρέπεται να καλούν το API.
- **Docker / container:** τεχνολογία που «πακετάρει» μια εφαρμογή μαζί με τις εξαρτήσεις της, ώστε να εκτελείται με τον ίδιο τρόπο σε οποιοδήποτε περιβάλλον [8].

1.6 Εύρος και περιορισμοί

Η εργασία καλύπτει την ανάπτυξη μιας διαδικτυακής εφαρμογής, που αποτελείται από ένα API στην πλευρά του διακομιστή(server) και δύο εφαρμογές για τον χρήστη: μία για τα μέλη και μία για το προσωπικό του γυμναστηρίου. Σε αυτήν περιλαμβάνονται:

- η σύνδεση και η εγγραφή μέσω πρόσκλησης, με ρόλους χρηστών
- το εβδομαδιαίο πρόγραμμα και οι κρατήσεις
- η λίστα αναμονής και η πολιτική ακυρώσεων
- οι συνδρομές και η κατανάλωσή τους
- η διαχείριση μαθημάτων, χρηστών και ρυθμίσεων
- η υποστήριξη πολλών γυμναστηρίων
- η τοπική εκτέλεση ολόκληρου του συστήματος με Docker

Ορισμένες λειτουργίες έμειναν εκτός εύρους, ώστε η εργασία να επικεντρωθεί στις κρατήσεις:

- οι ηλεκτρονικές πληρωμές (οι συνδρομές ανατίθενται από τον διαχειριστή)
- οι εφαρμογές για κινητά (και οι 2 εφαρμογές είναι responsive σε desktop και smartphones)
- οι ειδοποιήσεις με email ή push για υπενθυμίσεις και για όταν κάποιος παίρνει θέση από τη λίστα αναμονής
- η αυτόματη δημιουργία νέου γυμναστηρίου από τον πελάτη (τα γυμναστήρια δημιουργούνται μέσω ρυθμίσεων).

Υπάρχουν επίσης ορισμένοι περιορισμοί:

- Η εφαρμογή δεν έχει εγκατασταθεί σε περιβάλλον παραγωγής, εκτελείται τοπικά με Docker, όπου τα email των προσκλήσεων παραλαμβάνονται από τον τοπικό εξυπηρετητή Papercut.
- Η αξιολόγηση βασίζεται σε δοκιμαστικά δεδομένα και σε ελεγχόμενες μετρήσεις, όχι σε πραγματική χρήση από γυμναστήριο.
- Το περιβάλλον χρήστη είναι μόνο στα ελληνικά, αν και ο κώδικας είναι οργανωμένος ώστε να υποστηρίξει στο μέλλον και άλλες γλώσσες.



1.7 Παραδοτέα της εργασίας

Τα παραδοτέα της εργασίας είναι τα εξής:

1. Το κείμενο της πτυχιακής εργασίας, σε μορφή PDF και DOCX.
2. Ο πλήρης πηγαίος κώδικας της εφαρμογής, σε repository στο GitHub στο οποίο δίνεται πρόσβαση. Περιλαμβάνει το backend σε .NET, το frontend σε Angular με τις δύο εφαρμογές και τις κοινές βιβλιοθήκες, καθώς και τα tests.
3. Τα αρχεία Docker Compose για την τοπική εκτέλεση ολόκληρου του συστήματος και με δοκιμαστικά δεδομένα, μαζί με τις οδηγίες (κεφάλαιο 6 και README).
4. Η τεκμηρίωση του API, μέσω Scalar/OpenAPI και συνοπτικά στο Παράρτημα Α.
5. Τα scripts και τα αποτελέσματα των μετρήσεων της αξιολόγησης (κεφάλαιο 5).
6. Το βίντεο επίδειξης των βασικών λειτουργιών της εφαρμογής.

1.8 Δομή της εργασίας

Η υπόλοιπη εργασία είναι οργανωμένη ως εξής. Στο κεφάλαιο 2 παρουσιάζεται το υπόβαθρο και η σχετική εργασία: η επιστημονική βιβλιογραφία που σχετίζεται με το αντικείμενο, υπάρχουσες εφαρμογές με παρόμοιο αντικείμενο και η κριτική σύγκρισή τους με την δικιά μας εφαρμογή. Το κεφάλαιο 3 αφορά την ανάλυση των απαιτήσεων και τη σχεδίαση: τους ρόλους των χρηστών, τις λειτουργικές και μη λειτουργικές απαιτήσεις, τις περιπτώσεις χρήσης, τα πρόχειρα σχέδια της διεπαφής και το μοντέλο δεδομένων. Στο κεφάλαιο 4 περιγράφονται η αρχιτεκτονική και η υλοποίηση του συστήματος: η γενική δομή, οι τεχνολογίες, οι ροές εκτέλεσης των βασικών λειτουργιών, τα πιο απαιτητικά χαρακτηριστικά, η ασφάλεια, η εκτέλεση με Docker και οι βασικές αρχιτεκτονικές αποφάσεις. Το κεφάλαιο 5 παρουσιάζει την αξιολόγηση της εφαρμογής, ενώ το κεφάλαιο 6 αποτελεί οδηγό εγκατάστασης και χρήσης. Το κεφάλαιο 7 περιλαμβάνει τα συμπεράσματα, τους περιορισμούς και τις προτάσεις για μελλοντική εργασία. Ακολουθούν η βιβλιογραφία και τα παραρτήματα, με την επισκόπηση των endpoints του API, τον κατάλογο συντομογραφιών και τη δήλωση χρήσης εργαλείων τεχνητής νοημοσύνης.

2. Υπόβαθρο και σχετική εργασία

Στο κεφάλαιο αυτό παρουσιάζεται το υπόβαθρο της εργασίας: αρχικά οι βασικές έννοιες από τη σχετική βιβλιογραφία και στη συνέχεια υπάρχουσες λύσεις με παρόμοιο αντικείμενο, οι οποίες συγκρίνονται κριτικά με την εφαρμογή που αναπτύχθηκε.

2.1 Επιστημονικό υπόβαθρο

2.1.1 Software as a Service (SaaS) και multi-tenancy

Στο μοντέλο Software as a Service (SaaS), μία εγκατάσταση της εφαρμογής εξυπηρετεί πολλούς πελάτες. Οι Krebs κ.ά. ορίζουν το multi-tenancy ως την αρχιτεκτονική όπου πολλοί πελάτες (tenants) χρησιμοποιούν την ίδια εφαρμογή, αλλά ο καθένας τη βλέπει σαν να είναι μόνο δική του. Ως βασικά θέματα αναφέρουν την απομόνωση των δεδομένων (isolation), την απόδοση (performance), τη δυνατότητα προσαρμογής για κάθε πελάτη (customization) και την κλιμάκωση (scalability) [1].

Για την αποθήκευση των δεδομένων, οι Chong και Carraro περιγράφουν τρεις τρόπους, από τον πιο απομονωμένο προς τον πιο κοινόχρηστο: ξεχωριστή βάση για κάθε tenant, κοινή βάση με ξεχωριστό schema για κάθε tenant, και κοινό schema όπου κάθε εγγραφή έχει το αναγνωριστικό (TenantId) του tenant στον οποίο ανήκει. Όσο περισσότερα πράγματα μοιράζονται οι tenants, τόσο μικραίνει το κόστος για τον καθένα, αλλά η απομόνωση δεν εξασφαλίζεται πια από την υποδομή και πρέπει να την εξασφαλίσει ο κώδικας της εφαρμογής [9], [10]. Οι Aulbach κ.ά. δείχνουν ότι οι κοινοί πίνακες κλιμακώνονται καλύτερα όταν οι tenants είναι πολλοί [11]. Οι Bezemer και Zaidman σημειώνουν ότι το multi-tenancy κάνει πιο εύκολη τη συντήρηση, αρκεί οι βασικές αρχιτεκτονικές επιλογές, και κυρίως ο διαχωρισμός των δεδομένων, να γίνουν σωστά από την αρχή [12].

Η εργασία αυτή χρησιμοποιεί την προσέγγιση του κοινού schema (AA-1, ενότητα 4.9). Γι' αυτό η απομόνωση των δεδομένων γίνεται και ελέγχεται μέσα στον κώδικα (EE3).

2.1.2 Transactions και concurrency control

Ένα transaction εξασφαλίζει ότι μια ομάδα ενεργειών στη βάση εκτελείται είτε ολόκληρη είτε καθόλου, ότι δεν επηρεάζεται από άλλα transactions που τρέχουν την ίδια στιγμή και ότι το αποτέλεσμα αποθηκεύεται μόνιμα. Οι ιδιότητες αυτές είναι γνωστές ως ACID [6]. Ο πιο συνηθισμένος τρόπος για να εκτελούνται σωστά πολλά transactions ταυτόχρονα είναι το κλείδωμα (locking): ένα transaction κλειδώνει τα δεδομένα που επεξεργάζεται και τα υπόλοιπα περιμένουν μέχρι να τελειώσει [7].

Οι Berenson κ.ά. δείχνουν ότι τα πιο χαλαρά επίπεδα απομόνωσης (isolation levels) επιτρέπουν να εμφανιστούν λάθη στα δεδομένα, όπως το lost update: δύο transactions διαβάζουν την ίδια τιμή και η αλλαγή του δεύτερου «σβήνει» την αλλαγή του πρώτου [13]. Το επίπεδο Read Committed, που χρησιμοποιεί η PostgreSQL από προεπιλογή, δεν προστατεύει από αυτό το πρόβλημα όταν η εφαρμογή πρώτα διαβάζει, μετά αποφασίζει και μετά γράφει. Γι' αυτό η PostgreSQL δίνει τη δυνατότητα να κλειδωθεί ρητά μια γραμμή με την εντολή SELECT … FOR UPDATE [14].

Υπάρχουν και δύο άλλες λύσεις. Η πρώτη είναι το optimistic concurrency control των Kung και Robinson: δεν κλειδώνει τίποτα, αλλά στο τέλος του transaction ελέγχει αν κάποιος άλλος άλλαξε στο μεταξύ τα ίδια δεδομένα και, αν ναι, το transaction ξεκινά από την αρχή. Λειτουργεί καλά όταν τέτοιες συγκρούσεις (conflicts) είναι σπάνιες, όχι όμως όταν πολλοί χρήστες θέλουν το ίδιο δεδομένο την ίδια στιγμή [15]. Η δεύτερη είναι το επίπεδο Serializable της PostgreSQL (Serializable Snapshot Isolation), που εγγυάται ότι το αποτέλεσμα θα είναι το ίδιο σαν τα transactions να είχαν εκτελεστεί ένα-ένα. Όταν όμως δύο transactions συγκρούονται, ακυρώνει το ένα και η εφαρμογή πρέπει να το ξανατρέξει [16].

Σε ένα δημοφιλές μάθημα όλες οι κρατήσεις αφορούν την ίδια ακριβώς εγγραφή στη βάση, οπότε οι συγκρούσεις είναι συχνές. Γι' αυτό επιλέχθηκε το ρητό κλείδωμα (AA-2). Το πείραμα EE1 δείχνει στην πράξη ότι στην απλή υλοποίηση, χωρίς κλείδωμα, εμφανίζεται το lost update.

2.1.3 Αρχιτεκτονική και ασφάλεια web εφαρμογών

Το REST είναι ένας τρόπος σχεδίασης της επικοινωνίας ανάμεσα σε client και server, όπου η επικοινωνία είναι stateless: κάθε αίτημα περιέχει όλες τις πληροφορίες που χρειάζεται ο server για να το εξυπηρετήσει, χωρίς να θυμάται τίποτα από τα προηγούμενα αιτήματα [2]. Αυτό ταιριάζει με την ταυτοποίηση (authentication) μέσω υπογεγραμμένων tokens (JWT) [3]. Για τα JWT, οι καλές πρακτικές προτείνουν να ελέγχονται ο αλγόριθμος υπογραφής, ποιος το εξέδωσε (issuer), για ποιον προορίζεται (audience) και πότε λήγει [17]. Η οργάνωση του backend σε επίπεδα (layers), δηλαδή παρουσίαση, business logic και δεδομένα, χωρίζει τις δουλειές και κάνει πιο εύκολη τη συντήρηση [18], [19].

Όσον αφορά την ασφάλεια, η λίστα OWASP Top 10 βάζει στην πρώτη θέση το Broken Access Control, δηλαδή τις περιπτώσεις όπου ένας χρήστης μπορεί να δει ή να κάνει πράγματα που δεν θα έπρεπε [20]. Σε μια multi-tenant εφαρμογή, η απομόνωση των δεδομένων κάθε tenant είναι στην ουσία κι αυτή μια μορφή ελέγχου πρόσβασης (access control). Για τις επιθέσεις brute force, όπου κάποιος δοκιμάζει πολλούς κωδικούς μέχρι να βρει τον σωστό, οι προτεινόμενες λύσεις είναι το προσωρινό κλείδωμα του λογαριασμού (account lockout) και ο περιορισμός του πλήθους των αιτημάτων (rate limiting) [21]. Και οι δύο αναλύονται στην ενότητα 4.7.1.

2.2 Υπάρχουσες λύσεις

Πριν από τον σχεδιασμό της εφαρμογής, μελετήθηκαν τρεις υπάρχουσες εφαρμογές διαχείρισης κρατήσεων που χρησιμοποιούνται από πραγματικά γυμναστήρια και στούντιο. Στόχος ήταν να εντοπιστούν οι λειτουργίες και οι πρακτικές αντίστοιχων εφαρμογών, αλλά και τα σημεία που θα μπορούσε να καλύψει διαφορετικά η παρούσα εργασία. Για κάθε εφαρμογή παρουσιάζονται οι βασικές δυνατότητες μαζί με ένα στιγμιότυπο, ενώ στην επόμενη ενότητα υπάρχει συγκριτικός πίνακας χαρακτηριστικών.

2.2.1 Green Fitness Club

Το Green Fitness Club είναι εφαρμογή για κινητά ενός στούντιο Pilates. Η κράτηση γίνεται μέσα από ημερολόγιο και εβδομαδιαίο πρόγραμμα, όπου κάθε μάθημα εμφανίζει τη χωρητικότητά του (για παράδειγμα 5/8 θέσεις) και το χρονικό όριο μέσα στο οποίο επιτρέπεται η κράτηση (για παράδειγμα «έως και 1 ώρα πριν»), όταν το περιθώριο περάσει, το μάθημα σημειώνεται ως εκπρόθεσμο.

Όταν ένα μάθημα είναι γεμάτο, ο χρήστης μπορεί να μπει σε λίστα αναμονής. Το ιστορικό κρατήσεων δείχνει την κατάσταση κάθε κράτησης (π.χ. «Επερχόμενη») και επιτρέπει την ακύρωση. Στην ενότητα των συνδρομών εμφανίζονται η περίοδος ισχύος, ο αριθμός των κρατήσεων και των πραγματοποιημένων μαθημάτων, ενώ ο κατάλογος «Πακέτα Συνδρομών» παρουσιάζει τα διαθέσιμα πλάνα με την τιμή και τη διάρκειά τους.

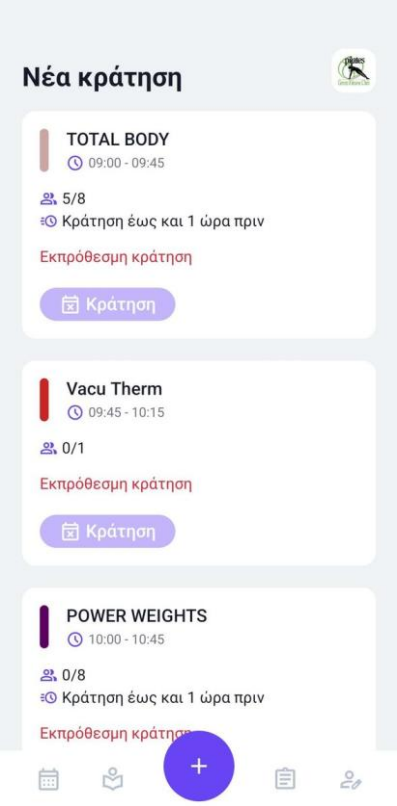
Επιπλέον, η εφαρμογή διαθέτει κωδικό QR για την είσοδο στο μάθημα και ρυθμίσεις όπου ο χρήστης επιλέγει γλώσσα, φωτεινό ή σκοτεινό θέμα και ενημερώνει τα στοιχεία του. Συνολικά, είναι μια εφαρμογή πολύ κοντά στη λογική της παρούσας εργασίας, εστιασμένη ωστόσο στην εξυπηρέτηση ενός μόνο στούντιο.

Κύρια ροή:

1. σύνδεση στην εφαρμογή
2. επιλογή ημέρας από το ημερολόγιο ή το εβδομαδιαίο πρόγραμμα

3. επιλογή μαθήματος, όπου φαίνονται η πληρότητα και το χρονικό όριο κράτησης
4. κράτηση ή, αν το μάθημα είναι γεμάτο, είσοδος στη λίστα αναμονής
5. παρακολούθηση ή ακύρωση της κράτησης από το ιστορικό

Κριτική: Η πληρότητα και το χρονικό όριο εμφανίζονται στο ίδιο σημείο με το κουμπί της κράτησης, οπότε ο χρήστης γνωρίζει αμέσως αν μπορεί να κλείσει θέση, η λογική αυτή υπάρχει και στην κάρτα μαθήματος της παρούσας εφαρμογής, όπως και η λίστα αναμονής. Λείπουν όμως φίλτρα ανά είδος μαθήματος ή προπονητή, και η εφαρμογή λειτουργεί σε ένα μόνο στούντιο.



Εικόνα 1 *— Green Fitness Club: πρόγραμμα και κράτηση μαθήματος.*

2.2.2 Planet Fitness & More

Η εφαρμογή Planet Fitness & More (με την ονομασία TrainYourPulse) αποτελεί μια πλούσια, εμπορική λύση. Το πρόγραμμα εμφανίζεται ανά ημέρα, με φωτογραφία για κάθε μάθημα και ένδειξη των διαθέσιμων θέσεων, ενώ ο κατάλογος μαθημάτων είναι εκτενής και συνοδεύεται από φίλτρα ανά μάθημα, προπονητή και επίπεδο δυσκολίας. Η κράτηση γίνεται επιλέγοντας μάθημα, προπονητή, ημερομηνία και ώρα.

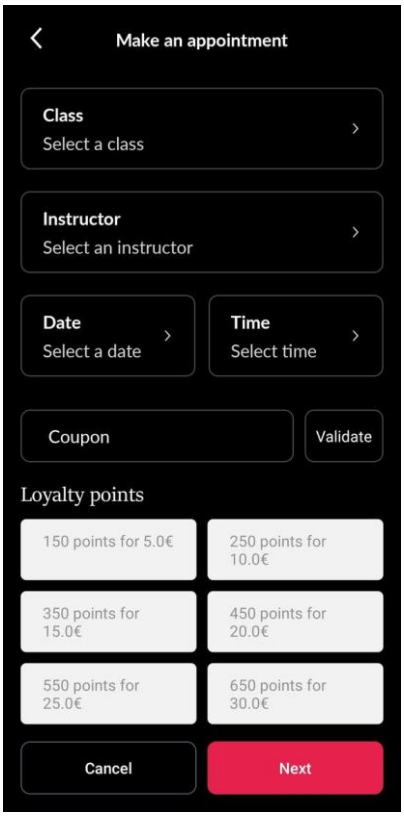
Πέρα από τον πυρήνα των κρατήσεων, η εφαρμογή προσφέρει αρκετές πρόσθετες δυνατότητες: ειδοποιήσεις, κωδικό QR για είσοδο, πόντους επιβράβευσης και εκπτωτικά κουπόνια, ηλεκτρονικό κατάστημα, προσωπικό πρόγραμμα προπόνησης, online μαθήματα, καθώς και χάρτη με τις τοποθεσίες. Πρόκειται για ένα ολοκληρωμένο εμπορικό σύστημα γύρω από το γυμναστήριο, με αντίστοιχα μεγαλύτερη δυσκολία στη χρήση.

Κύρια ροή:

1. σύνδεση στην εφαρμογή
2. προβολή του προγράμματος ανά ημέρα
3. φιλτράρισμα ανά μάθημα, προπονητή ή επίπεδο

4. επιλογή μαθήματος, προπονητή, ημερομηνίας και ώρας
5. κράτηση

Κριτική: Τα φίλτρα κάνουν εύκολη την εύρεση μαθήματος σε ένα μεγάλο πρόγραμμα και υιοθετήθηκαν (είδος μαθήματος και εκπαιδευτής). Ο πυρήνας της κράτησης «χάνεται» όμως ανάμεσα σε πολλές πρόσθετες λειτουργίες (κατάστημα, πόντοι, online μαθήματα), κάτι που αυξάνει την δυσκολία χρήσης, ενώ δεν εντοπίστηκε λίστα αναμονής.



Εικόνα 2 — Planet Fitness (TrainYourPulse): κράτηση με επιλογή μαθήματος, προπονητή και πόντους επιβράβευσης.

2.2.3 Wellness Hub Fitness Studio

Το Wellness Hub Fitness Studio είναι μια responsive διαδικτυακή εφαρμογή ενός στούντιο. Το εβδομαδιαίο πρόγραμμα υποστηρίζει πλοήγηση ανά ημέρα και επιλογή μέσω ημερολογίου, δείχνει την πληρότητα κάθε μαθήματος (για παράδειγμα 1/4 ή «5/5 γεμάτο») και προσφέρει φίλτρα ανά μάθημα και προπονητή. Ο χρήστης βλέπει το ιστορικό των κρατήσεων και τις συνδρομές του με τις ενεργές και τις διαθέσιμες κρατήσεις και την ημερομηνία λήξης.

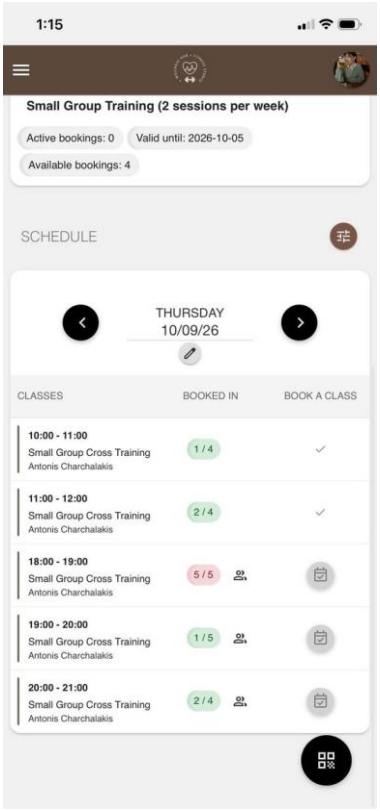
Η εφαρμογή δίνει ιδιαίτερη έμφαση στην οικονομική διαχείριση, με αναλυτικό ιστορικό συναλλαγών (τιμή, έκπτωση, τρόπος πληρωμής), αποθήκευση πιστωτικής κάρτας και στοιχεία τιμολόγησης. Ξεχωρίζει επίσης για τις ενσωματώσεις της, καθώς συγχρονίζει αυτόματα τις κρατήσεις με το Google Calendar, ενώ διαθέτει προσωπικό ημερολόγιο προπονήσεων, κωδικό QR και ρυθμίσεις προφίλ και ειδοποιήσεων.

Κύρια ροή:

1. σύνδεση στην εφαρμογή
2. εβδομαδιαίο πρόγραμμα με πλοήγηση ανά ημέρα
3. φιλτράρισμα ανά μάθημα και προπονητή
4. επιλογή μαθήματος με ένδειξη πληρότητας

5. κράτηση, με αυτόματο συγχρονισμό στο Google Calendar
6. προβολή της συνδρομής (ενεργές και διαθέσιμες κρατήσεις, ημερομηνία λήξης)

Κριτική: Η συνδρομή, με το υπόλοιπο και τη λήξη της, είναι πάντα ορατή, κάτι πολύ χρήσιμο για τον χρήστη, γι' αυτό η αρχική οθόνη της παρούσας εφαρμογής δείχνει τη συνδρομή δίπλα στις επερχόμενες κρατήσεις. Η εφαρμογή δίνει μεγάλη έμφαση στις πληρωμές, που είναι εκτός του εύρους της παρούσας εργασίας (ενότητα 1.6), ενώ δεν εντοπίστηκε λίστα αναμονής.



Εικόνα 3 — *Wellness Hub: εβδομαδιαίο πρόγραμμα και συνδρομή.*

2.3 Κριτική και σύγκριση

Στον Πίνακα 1 συγκρίνονται τα βασικά χαρακτηριστικά των τριών εφαρμογών με την παρούσα εργασία. Η σύγκριση δεν είναι εξαντλητική, αλλά εστιάζει στις λειτουργίες που έχουν σημασία για τη διαχείριση κρατήσεων και συνδρομών ενός γυμναστηρίου.

Πίνακας 1 — *Συγκριτική αξιολόγηση χαρακτηριστικών με τις υπάρχουσες εφαρμογές.*

Χαρακτηριστικό	**Green Fitness**	**Planet Fitness**	**Wellness Hub**	**Η παρούσα εφαρμογή**	**Τεκμήριο / σχόλιο**
Κράτηση σε μάθημα	✓	✓	✓	✓	Κύρια λειτουργία όλων (Εικόνες 1–3)
Πρόγραμμα ανά ημέρα / εβδομάδα	✓	✓	✓	✓	Ημερήσια ή εβδομαδιαία προβολή
Φίλτρα (μάθημα / προπονητής)	—	✓	✓	✓	Planet Fitness και ανά επίπεδο δυσκολίας
Ιστορικό κρατήσεων	✓	✓	✓	✓	Με την κατάσταση κάθε κράτησης
Ακύρωση με πολιτική / όριο χρόνου	✓	✓	✓	✓	Χρονικό όριο ανά γυμναστήριο
Συνδρομές / πακέτα με κατανάλωση	✓	✓	✓	✓	Υπόλοιπο και λήξη της συνδρομής

Χαρακτηριστικό	**Green Fitness**	**Planet Fitness**	**Wellness Hub**	**Η παρούσα εφαρμογή**	**Τεκμήριο / σχόλιο**
Λίστα αναμονής (waitlist)	✓	—	—	✓	Εντοπίστηκε μόνο στο Green Fitness
Προσωπικό ημερολόγιο προπονήσεων	—	✓	✓	✓	Καταγραφή προπονήσεων από το μέλος
QR check-in	✓	✓	✓	—	Είσοδος στο μάθημα με κωδικό QR
Πόντοι / κουπόνια επιβράβευσης	—	✓	—	—	Μόνο στο Planet Fitness
Online μαθήματα	—	✓	—	—	Μόνο στο Planet Fitness
Συγχρονισμός με Google Calendar	—	—	✓	—	Μόνο στο Wellness Hub
Φωτεινό/σκοτεινό θέμα & πολυγλωσσικό περιβάλλον	✓	—	✓	—	Η παρούσα: μόνο σκοτεινό θέμα και ελληνικά
Πλατφόρμα πολλαπλών γυμναστηρίων (multi-tenant)	—	—	—	✓	Οι τρεις εξυπηρετούν ένα στούντιο
Ανοιχτή / self-hosted εγκατάσταση (Docker)	—	—	—	✓	Τοπική εγκατάσταση με Docker Compose
Ξεχωριστή εφαρμογή διαχείρισης προσωπικού	—	—	—	✓	Εξετάστηκαν μόνο οι εφαρμογές των μελών

Όπως φαίνεται, και οι τρεις εφαρμογές, όπως και η παρούσα, καλύπτουν πλήρως τον πυρήνα της λειτουργίας: κράτηση σε μάθημα, εμφάνιση προγράμματος, διαχείριση συνδρομών και ακύρωση κρατήσεων.

Σε ορισμένα σημεία η παρούσα εφαρμογή υστερεί σε σχέση με τις πιο ώριμες εμπορικές λύσεις: δεν διαθέτει, τουλάχιστον στην τρέχουσα έκδοση, κωδικό QR για είσοδο, πόντους και κουπόνια επιβράβευσης, online μαθήματα ή συγχρονισμό με εξωτερικό ημερολόγιο.

Ταυτόχρονα, η παρούσα εργασία προσφέρει τρία στοιχεία που δεν εντοπίστηκαν σε καμία από τις τρεις εφαρμογές. Πρώτον, είναι σχεδιασμένη ως πλατφόρμα πολλαπλών γυμναστηρίων (multi-tenant), όπου κάθε γυμναστήριο λειτουργεί ανεξάρτητα με πλήρη απομόνωση των δεδομένων του, ενώ οι τρεις εφαρμογές εξυπηρετούν ένα μόνο στούντιο. Δεύτερον, διατίθεται ως λύση που μπορεί να εγκατασταθεί τοπικά μέσω Docker. Τρίτον, περιλαμβάνει μια ξεχωριστή, αποκλειστική εφαρμογή διαχείρισης για το προσωπικό, πέρα από την εφαρμογή των πελατών. Επιπλέον, υποστηρίζει λίστα αναμονής, την οποία από τις τρεις εφαρμογές διαθέτει μόνο το Green Fitness Club.

3. Ανάλυση απαιτήσεων και σχεδίαση

Στο κεφάλαιο αυτό καταγράφονται οι απαιτήσεις της εφαρμογής και η σχεδιασή της: πρώτα οι ρόλοι των χρηστών και οι ενέργειες που μπορεί να κάνει ο καθένας, στη συνέχεια οι λειτουργικές και οι μη λειτουργικές απαιτήσεις και οι περιπτώσεις χρήσης, και τέλος η σχεδίαση της διεπαφής χρήστη και της βάσης δεδομένων.

3.1 Ρόλοι χρηστών (πελάτης / προσωπικό / διαχειριστής)

Οι διαθέσιμοι ρόλοι του συστήματος είναι:

- **τα μέλη** του γυμναστηρίου, που θέλουν να βρίσκουν και να κλείνουν εύκολα θέση σε μαθήματα
- **οι εκπαιδευτές**, που οργανώνουν τα μαθήματά τους και παρακολουθούν τις συμμετοχές
- **οι διαχειριστές** (ιδιοκτήτες ή γραμματεία), που διαχειρίζονται χρήστες, συνδρομές και ρυθμίσεις
- **ο πάροχος της πλατφόρμας**, που φιλοξενεί πολλά γυμναστήρια στην ίδια εγκατάσταση και ενδιαφέρεται για την απομόνωση των δεδομένων τους.

Στη συνέχεια, οι ανάγκες αυτές αποτυπώνονται σε ρόλους χρηστών μέσα στο σύστημα.

Το σύστημα αναγνωρίζει τέσσερις τύπους χρηστών. Οι ρόλοι είναι ιεραρχικοί: κάθε ανώτερος ρόλος περιλαμβάνει τις δυνατότητες των κατώτερων.

- Επισκέπτης (μη συνδεδεμένος): μπορεί μόνο να συνδεθεί ή να εγγραφεί μέσω πρόσκλησης που έχει λάβει.
- Μέλος (πελάτης): βλέπει το εβδομαδιαίο πρόγραμμα, κάνει και ακυρώνει κρατήσεις, μπαίνει και βγαίνει από τη λίστα αναμονής, και βλέπει τις κρατήσεις και τη συνδρομή του.
- Εκπαιδευτής: έχει όλες τις δυνατότητες του μέλους και επιπλέον δημιουργεί και ακυρώνει μαθήματα, διαχειρίζεται τους τύπους μαθημάτων, βλέπει τους πελάτες που έχουν δηλώσει συμμετοχή σε κάθε μάθημα και μπορεί να κάνει ή να ακυρώσει μια κράτηση.
- Διαχειριστής: έχει όλες τις δυνατότητες του εκπαιδευτή και επιπλέον διαχειρίζεται τους χρήστες και τους ρόλους τους, τα συνδρομητικά πλάνα και τις συνδρομές, στέλνει προσκλήσεις εγγραφής και ρυθμίζει τις παραμέτρους του γυμναστηρίου.

3.2 Λειτουργικές απαιτήσεις

Οι λειτουργικές απαιτήσεις της εφαρμογής συνοψίζονται στον Πίνακα 2. Για κάθε απαίτηση δίνονται ο κωδικός της, η προτεραιότητα, η πηγή από την οποία προέκυψε (Μ: μελέτη υπαρχουσών εφαρμογών, ενότητα 2.2, Σ: στόχος της εργασίας, ενότητα 1.3), οι σχετικές περιπτώσεις χρήσης (ενότητα 3.5) και οι αυτοματοποιημένες δοκιμές που την επαληθεύουν. Όλες οι απαιτήσεις ελέγχθηκαν επιπλέον χειροκίνητα μέσα από τη διεπαφή.

Πίνακας 2 — Λειτουργικές απαιτήσεις (FR).

Κωδ.	Απαίτηση	Προτερ.	Πηγή	UC	Επαλήθευση (tests)
FR-01	Σύνδεση με email/κωδικό και έκδοση JWT	Υψηλή	Σ	UC-01	AuthEndpointsTests, TokenServiceTests
FR-02	Εγγραφή μόνο μέσω έγκυρης πρόσκλησης (μη ληγμένης, μη χρησιμοποιημένης, λήξη 7 ημερών)	Υψηλή	Σ	UC-01	InvitationsTests
FR-03	Δημιουργία πρόσκλησης από διαχειριστή (ρόλος μέλους ή εκπαιδευτή) με σύνδεσμο εγγραφής	Υψηλή	Σ	UC-01	InvitationsTests

Κωδ.	Απαίτηση	Προτερ.	Πηγή	UC	Επαλήθευση (tests)
FR-04	Εβδομαδιαίο πρόγραμμα με πλοήγηση ανά εβδομάδα και φίλτρα είδους και εκπαιδευτή	Υψηλή	Μ	UC-02	ScheduleTests, InstructorsTests
FR-05	Κράτηση θέσης χωρίς υπέρβαση χωρητικότητας, διπλή κράτηση ή χρονική επικάλυψη	Πολύ υψηλή	Μ, Σ	UC-02	BookingTests, BookingEndpointsTests
FR-06	Κράτηση μόνο με ενεργή συνδρομή, κατανάλωση στην κράτηση, επιστροφή στην ακύρωση	Υψηλή	Μ	UC-02, UC-03	BookingSubscriptionTests
FR-07	Ακύρωση από το μέλος σύμφωνα με το περιθώριο ακύρωσης του γυμναστηρίου	Υψηλή	Μ	UC-03	CancellationPolicyTests, BookingTests
FR-08	Είσοδος/έξοδος σε λίστα αναμονής γεμάτου μαθήματος, με σειρά FIFO	Μέτρια	Μ	UC-04	WaitlistServiceTests, WaitlistEndpointsTests
FR-09	Αυτόματη προαγωγή του επόμενου έγκυρου μέλους όταν ελευθερώνεται θέση	Μέτρια	Σ	UC-03, UC-04	WaitlistPromoteTests
FR-10	Δημιουργία/ενημέρωση/απενεργοποίηση τύπων μαθημάτων από το προσωπικό	Υψηλή	Μ	UC-05	ClassTypesTests
FR-11	Δημιουργία και ακύρωση προγραμματισμένων μαθημάτων από το προσωπικό	Υψηλή	Μ	UC-05	ClassSessionsTests
FR-12	Παρουσιολόγιο, και κράτηση/ακύρωση εκ μέρους πελάτη από το προσωπικό	Μέτρια	Σ	UC-03	StaffBookingTests
FR-13	Διαχείριση χρηστών (αλλαγή ρόλου, ενεργοποίηση/απενεργοποίηση)	Μέτρια	Σ	—	UserServiceTests
FR-14	Συνδρομητικά πλάνα και ανάθεση συνδρομής σε μέλος	Μέτρια	Μ	UC-06	MembershipPlansTests, SubscriptionsTests
FR-15	Ρυθμίσεις γυμναστηρίου (όνομα, περιθώριο ακύρωσης)	Χαμηλή	Σ	—	TenantSettingsTests

3.3 Μη λειτουργικές απαιτήσεις

Πέρα από το τι κάνει η εφαρμογή, τέθηκαν και απαιτήσεις για τον τρόπο με τον οποίο λειτουργεί. Κάθε μη λειτουργική απαίτηση συνοδεύεται από μετρήσιμο κριτήριο και τρόπο επαλήθευσης (Πίνακας 3). Τα όρια των απαιτήσεων που επαληθεύονται με μετρήσεις ορίστηκαν πριν από τις μετρήσεις, οι οποίες παρουσιάζονται στο κεφάλαιο 5.

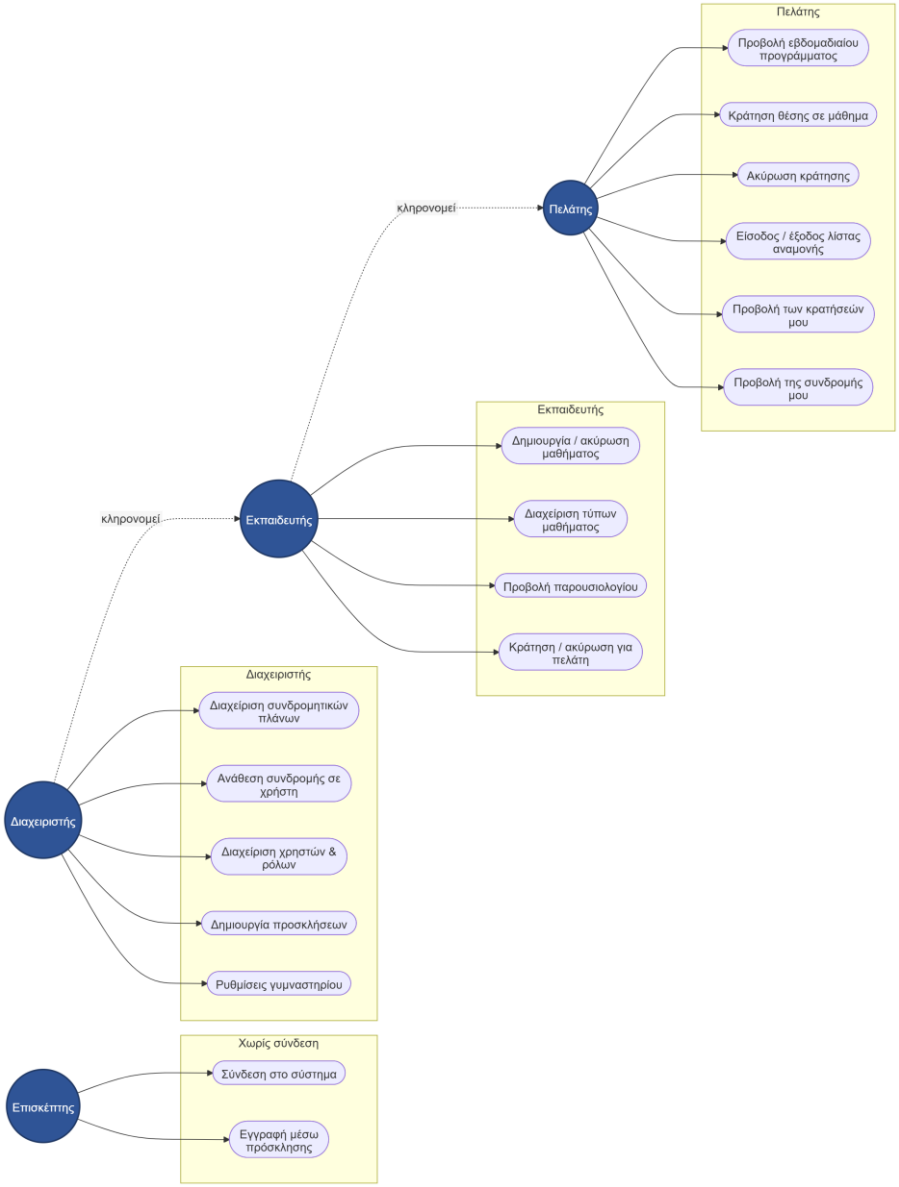
Πίνακας 3 *— Μη λειτουργικές απαιτήσεις (NFR) και κριτήρια επαλήθευσης.*

Κωδ.	Κατηγορία	Μετρήσιμο κριτήριο	Επαλήθευση
NFR-01	Ακεραιότητα	Σε N ταυτόχρονα αιτήματα κράτησης (N > C) σε μάθημα χωρητικότητας C: ακριβώς C επιτυχίες και 0 υπερκρατήσεις	Concurrent_bookings_never_exceed_capacity, πείραμα κεφαλαίου 5
NFR-02	Απόδοση	χρόνος απόκρισης κράτησης και προβολής προγράμματος, με 50 ταυτόχρονους χρήστες στο τοπικό περιβάλλον (Docker)	Μετρήσεις κεφαλαίου 5
NFR-03	Απομόνωση tenant	0 εγγραφές άλλου γυμναστηρίου σε κάθε ερώτημα (χρήστες, μαθήματα, κρατήσεις, συνδρομές, λίστα αναμονής, προσκλήσεις)	TenantIsolationTests, TenantIsolationHttpTests (9 tests)
NFR-04	Ασφάλεια σύνδεσης	Κλείδωμα λογαριασμού μετά από 5 αποτυχημένες προσπάθειες για 15 λεπτά, έως 10 αιτήματα σύνδεσης ανά λεπτό ανά IP.	Login_locks_account_after_five_failed_attempts, χειροκίνητος έλεγχος στην τοπική εκτέλεση
NFR-05	Εξουσιοδότηση	Κάθε ενέργεια διαχείρισης απαντά 403 σε χρήστη χωρίς τον απαιτούμενο ρόλο, το token λήγει σε 2 ώρες	CreateInvitation_as_non_admin_returns_403, CreatePlan_as_non_admin_returns_403, CreateClassType_as_plain_user_returns_403, TokenServiceTests

Κωδ.	Κατηγορία	Μετρήσιμο κριτήριο	Επαλήθευση
NFR-06	Προστασία στοιχείων	Κωδικοί αποθηκεύονται μόνο ως hash και tokens προσκλήσεων μόνο ως SHA-256, κανένα σε καθαρή μορφή	Επιθεώρηση κώδικα και βάσης
NFR-07	Ιδιωτικότητα	Αποθηκεύονται μόνο όνομα, email και στοιχεία κρατήσεων/συνδρομών, λογική διαγραφή (soft-delete)	Επιθεώρηση μοντέλου δεδομένων (ERD)
NFR-08	Χρηστικότητα / συμβατότητα	Η εφαρμογή μελών χρησιμοποιείται σε πλάτος 375 px χωρίς οριζόντια κύλιση, η εφαρμογή προσωπικού σε ≥ 1280 px, με πτυσσόμενο μενού	Χειροκίνητος έλεγχος με προσομοίωση κινητού
NFR-09	Προσβασιμότητα	Βαθμολογία Lighthouse Accessibility ≥ 90 στις κύριες οθόνες	Μέτρηση κεφαλαίου 5
NFR-10	Φορητότητα	Όλο το σύστημα ξεκινά με μία εντολή σε μηχάνημα με μόνο Docker Desktop, με αυτόματα migrations και demo δεδομένα	Εκτέλεση σε καθαρό περιβάλλον, SeedingTests, DemoDataSeederTests
NFR-11	Συντηρησιμότητα	Κώδικας σε επίπεδα (Api/Core/Data) και κοινές βιβλιοθήκες, όλα τα αυτοματοποιημένα tests (81 backend, 26 frontend) περνούν	dotnet test (backend), nx test (frontend)
NFR-12	Χρόνος / γλώσσα	Ημερομηνίες αποθηκεύονται σε UTC και εμφανίζονται σε ώρα Ελλάδας, περιβάλλον στα ελληνικά	Επιθεώρηση κώδικα, χειροκίνητα

3.4 Διάγραμμα περιπτώσεων χρήσης (Use Case)

Το διάγραμμα περιπτώσεων χρήσης (Διάγραμμα 1) συνοψίζει τους ρόλους των χρηστών και τις ενέργειες που υποστηρίζει το σύστημα. Οι τέσσερις ρόλοι, επισκέπτης, μέλος, εκπαιδευτής και διαχειριστής, συνδέονται με τις αντίστοιχες περιπτώσεις χρήσης, ενώ οι διακεκομμένες γραμμές δηλώνουν ότι κάθε ανώτερος ρόλος κληρονομεί τις δυνατότητες του προηγούμενου. Τα διαγράμματα της εργασίας σχεδιάστηκαν με το εργαλείο Mermaid [22].



Διάγραμμα 1 *— Περιπτώσεις χρήσης (Use Case).*

3.5 Περιγραφή περιπτώσεων χρήσης

Στη συνέχεια δίνονται δομημένες περιγραφές των βασικών περιπτώσεων χρήσης του συστήματος (Πίνακες 4–9), στη μορφή use case description των Kupersmith [23]: τίτλος με κωδικό (Use case ID, Title), Actors, Description, Preconditions, Postconditions, Primary path, Alternate path, Exception path, καθώς και Priority και Frequency of Use. Σύμφωνα με τον οδηγό εκπόνησης της πτυχιακής, κάθε περιγραφή συμπληρώνεται με τα πεδία Goal / Value, Trigger, Acceptance criteria, σε μορφή «Δεδομένου ότι – όταν – τότε», και Traceability προς τις απαιτήσεις, τις οθόνες και τις αυτοματοποιημένες δοκιμές.

Πίνακας 4 *— Περίπτωση χρήσης UC-01: Εγγραφή μέσω πρόσκλησης και σύνδεση.*

UC-01 — Εγγραφή μέσω πρόσκλησης και σύνδεση	
Actors	Επισκέπτης (κύριος), Διαχειριστής (δευτερεύων, εκδίδει την πρόσκληση).

UC-01 — Εγγραφή μέσω πρόσκλησης και σύνδεση	
Description	Ο νέος χρήστης εγγράφεται μέσω συνδέσμου πρόσκλησης και στη συνέχεια συνδέεται στην εφαρμογή.
Goal / Value	Ο νέος χρήστης αποκτά λογαριασμό στο σωστό γυμναστήριο και με τον σωστό ρόλο, και συνδέεται με ασφάλεια.
Trigger	Ο χρήστης ανοίγει τον σύνδεσμο πρόσκλησης που έλαβε (εγγραφή) ή την οθόνη σύνδεσης (σύνδεση).
Preconditions	Ο διαχειριστής έχει εκδώσει πρόσκληση για το email του χρήστη, ο σύνδεσμος δεν έχει λήξει ούτε χρησιμοποιηθεί.
Postconditions	Δημιουργείται λογαριασμός με τον αντίστοιχο ρόλο και ο χρήστης αποκτά πρόσβαση μέσω token (JWT).
Primary path	1. Ο χρήστης ανοίγει τον σύνδεσμο της πρόσκλησης. 2. Το σύστημα ελέγχει ότι το token είναι έγκυρο, δεν έχει λήξει και δεν έχει χρησιμοποιηθεί. 3. Ο χρήστης συμπληρώνει ονοματεπώνυμο και κωδικό. 4. Το σύστημα δημιουργεί τον λογαριασμό, αναθέτει τον ρόλο και σημειώνει την πρόσκληση ως χρησιμοποιημένη. 5. Ο χρήστης συνδέεται με email και κωδικό, το σύστημα εκδίδει token πρόσβασης.
Alternate path	Αν ο χρήστης είναι ήδη εγγεγραμμένος, προχωρά απευθείας στη σύνδεση.
Exception path	Άκυρο/ληγμένο/χρησιμοποιημένο token ή λανθασμένα στοιχεία σύνδεσης → εμφάνιση μηνύματος και διακοπή.
Priority / Frequency of Use	Υψηλή / Χαμηλή (εγγραφή), Υψηλή (σύνδεση).
Acceptance criteria	**Δεδομένου ότι** υπάρχει έγκυρη πρόσκληση, **όταν** ο χρήστης συμπληρώσει τα στοιχεία του, **τότε** δημιουργείται ενεργός λογαριασμός με τον ρόλο και το γυμναστήριο της πρόσκλησης και η πρόσκληση δεν μπορεί να ξαναχρησιμοποιηθεί. **Δεδομένου ότι** η πρόσκληση έχει λήξει ή έχει ήδη χρησιμοποιηθεί, **όταν** ανοιχτεί ο σύνδεσμος, **τότε** η εγγραφή απορρίπτεται. **Δεδομένου ότι** ο χρήστης έχει δώσει λάθος κωδικό 5 φορές, **όταν** ξαναπροσπαθήσει, **τότε** ο λογαριασμός κλειδώνεται για 15 λεπτά.
Traceability	FR-01, FR-02, FR-03, NFR-04, NFR-05, NFR-06, οθόνες: Σύνδεση (Εικόνα 4), Εγγραφή, Προσκλήσεις, tests: AuthEndpointsTests, InvitationsTests, TokenServiceTests

Πίνακας 5 — Περίπτωση χρήσης UC-02: Κράτηση θέσης σε μάθημα.

UC-02 — Κράτηση θέσης σε μάθημα	
Actors	Μέλος (κύριος).
Description	Το μέλος κλείνει θέση σε ένα προγραμματισμένο μάθημα.
Goal / Value	Το μέλος εξασφαλίζει θέση στο μάθημα που θέλει, χωρίς επικοινωνία με το γυμναστήριο.
Trigger	Το μέλος πατά «Κράτηση» σε ένα μάθημα του εβδομαδιαίου προγράμματος.

UC-02 — Κράτηση θέσης σε μάθημα	
Preconditions	Ο χρήστης είναι συνδεδεμένος και έχει ενεργή συνδρομή, το μάθημα είναι ενεργό και έχει διαθέσιμη θέση.
Postconditions	Δημιουργείται επιβεβαιωμένη κράτηση, μειώνεται η διαθέσιμη χωρητικότητα και καταναλώνεται μία συνεδρία από τη συνδρομή.
Primary path	1. Το μέλος επιλέγει ένα μάθημα από το πρόγραμμα και πατά «Κράτηση». 2. Το σύστημα ξεκινά ένα transaction και κλειδώνει προσωρινά την εγγραφή του μαθήματος και στη συνέχεια τον χρήστη. 3. Ελέγχει: ύπαρξη/ενεργό μάθημα, όχι ακυρωμένο, όχι διπλή κράτηση, όχι χρονική επικάλυψη, διαθέσιμη θέση. 4. Καταναλώνει μία συνεδρία από την ενεργή συνδρομή. 5. Καταχωρεί την κράτηση, αυξάνει τον μετρητή θέσεων και ολοκληρώνει το transaction.
Alternate path	Αν το μάθημα είναι γεμάτο, προτείνεται η είσοδος στη λίστα αναμονής (UC-04).
Exception path	Χωρίς ενεργή συνδρομή, χρονική επικάλυψη, ήδη κρατημένο ή γεμάτο μάθημα → το transaction ακυρώνεται με αντίστοιχο μήνυμα.
Priority / Frequency of Use	Πολύ υψηλή / Πολύ υψηλή.
Acceptance criteria	**Δεδομένου ότι** το μάθημα έχει ελεύθερη θέση και το μέλος ενεργή συνδρομή, **όταν** πατήσει «Κράτηση», **τότε** η κράτηση καταχωρείται, οι κρατημένες θέσεις αυξάνονται κατά 1 και το κουμπί γίνεται «Ακύρωση». **Δεδομένου ότι** απομένει μία θέση, **όταν** Ν μέλη κάνουν ταυτόχρονα κράτηση, **τότε** πετυχαίνει ακριβώς μία κράτηση.
Traceability	FR-04, FR-05, FR-06, NFR-01, NFR-02 · οθόνες: Πρόγραμμα (Εικόνες 6 και 11) · tests: ScheduleTests, InstructorsTests, BookingTests, BookingEndpointsTests, BookingSubscriptionTests

Πίνακας 6 *— Περίπτωση χρήσης UC-03: Ακύρωση κράτησης.*

UC-03 — Ακύρωση κράτησης	
Actors	Μέλος (κύριος), Προσωπικό (δευτερεύων).
Description	Το μέλος (ή το προσωπικό) ακυρώνει μια υπάρχουσα κράτηση.
Goal / Value	Το μέλος απελευθερώνει μια θέση που δεν θα χρησιμοποιήσει και παίρνει πίσω τη συνεδρία του, ενώ η θέση περνά σε όποιον περιμένει.
Trigger	Το μέλος πατά «Ακύρωση» σε μια κρατημένη θέση του προγράμματος (ή το προσωπικό από το παρουσιολόγιο).
Preconditions	Υπάρχει ενεργή κράτηση του χρήστη.
Postconditions	Η κράτηση ακυρώνεται, ελευθερώνεται η θέση, επιστρέφεται η συνεδρία στη συνδρομή και, αν υπάρχει λίστα αναμονής, προάγεται ο επόμενος.

UC-03 — Ακύρωση κράτησης	
Primary path	1. Το μέλος επιλέγει μια κράτηση και πατά «Ακύρωση». 2. Το σύστημα κλειδώνει το μάθημα και την κράτηση, ελέγχει ξανά ότι η κράτηση είναι ακόμα ενεργή και μετά ελέγχει την πολιτική ακυρώσεων (ελάχιστο περιθώριο πριν από το μάθημα). 3. Ακυρώνει την κράτηση, μειώνει τον μετρητή θέσεων και επιστρέφει τη συνεδρία στη συνδρομή. 4. Αν υπάρχει κενή θέση και λίστα αναμονής, προάγει αυτόματα τον επόμενο έγκυρο χρήστη.
Alternate path	Ακύρωση από το προσωπικό: γίνεται χωρίς τον έλεγχο της πολιτικής ακυρώσεων.
Exception path	Ακύρωση εκτός του επιτρεπτού παραθύρου → απόρριψη. Ήδη ακυρωμένη κράτηση, π.χ. από δεύτερο ταυτόχρονο αίτημα → καμία ενέργεια.
Priority / Frequency of Use	Υψηλή / Μέτρια.
Acceptance criteria	**Δεδομένου ότι** το μάθημα απέχει περισσότερο από το περιθώριο ακύρωσης, **όταν** το μέλος ακυρώσει, **τότε** η κράτηση ακυρώνεται, η θέση ελευθερώνεται και η συνεδρία επιστρέφει στη συνδρομή. **Δεδομένου ότι** απομένει λιγότερος χρόνος από το περιθώριο, **όταν** το μέλος προσπαθήσει να ακυρώσει, **τότε** η ακύρωση απορρίπτεται. **Δεδομένου ότι** υπάρχει λίστα αναμονής, **όταν** ακυρωθεί μια κράτηση, **τότε** το πρώτο έγκυρο μέλος της λίστας αποκτά αυτόματα τη θέση.
Traceability	FR-06, FR-07, FR-09, FR-12 · οθόνες: Πρόγραμμα (Εικόνα 11), Παρουσιολόγιο (Εικόνα 14) · tests: CancellationPolicyTests, BookingTests, BookingSubscriptionTests, WaitlistPromoteTests, StaffBookingTests

Πίνακας 7 — Περίπτωση χρήσης UC-04: Είσοδος σε λίστα αναμονής.

UC-04 — Είσοδος σε λίστα αναμονής	
Actors	Μέλος (κύριος).
Description	Όταν ένα μάθημα είναι γεμάτο, το μέλος μπαίνει στη λίστα αναμονής.
Goal / Value	Το μέλος διατηρεί την ευκαιρία να συμμετάσχει σε γεμάτο μάθημα χωρίς να ελέγχει συνεχώς το πρόγραμμα.
Trigger	Το μέλος πατά «Αναμονή» σε ένα γεμάτο μάθημα.
Preconditions	Το μάθημα είναι γεμάτο, ο χρήστης έχει ενεργή συνδρομή και δεν έχει ήδη κράτηση ή θέση στη λίστα.
Postconditions	Ο χρήστης προστίθεται στη λίστα αναμονής με σειρά προτεραιότητας (FIFO).
Primary path	1. Το μέλος επιλέγει γεμάτο μάθημα και πατά «Λίστα αναμονής». 2. Το σύστημα ελέγχει ότι το μάθημα είναι γεμάτο, ότι δεν υπάρχει ήδη κράτηση/εγγραφή και ότι υπάρχει ενεργή συνδρομή. 3. Καταχωρεί την εγγραφή με χρονική σήμανση.

UC-04 — Είσοδος σε λίστα αναμονής	
Alternate path	Αν το μάθημα δεν είναι γεμάτο, το σύστημα προτρέπει σε κανονική κράτηση.
Exception path	Ήδη στη λίστα ή χωρίς ενεργή συνδρομή → εμφάνιση μηνύματος και διακοπή.
Priority / Frequency of Use	Μέτρια / Μέτρια.
Acceptance criteria	**Δεδομένου ότι** το μάθημα είναι γεμάτο και το μέλος έχει ενεργή συνδρομή, **όταν** μπει στη λίστα αναμονής, **τότε** καταχωρείται με τη σειρά άφιξης και βλέπει τη θέση του στη λίστα. **Δεδομένου ότι** το μάθημα έχει ελεύθερες θέσεις, **όταν** επιχειρηθεί είσοδος στη λίστα, **τότε** η είσοδος απορρίπτεται.
Traceability	FR-08, FR-09 · οθόνη: Πρόγραμμα (Εικόνα 11) · tests: WaitlistServiceTests, WaitlistEndpointsTests, WaitlistPromoteTests

Πίνακας 8 *— Περίπτωση χρήσης UC-05: Δημιουργία και διαχείριση μαθήματος.*

UC-05 — Δημιουργία και διαχείριση μαθήματος	
Actors	Εκπαιδευτής ή Διαχειριστής (κύριος).
Description	Το προσωπικό δημιουργεί, ενημερώνει ή ακυρώνει προγραμματισμένα μαθήματα.
Goal / Value	Το προσωπικό κρατά το πρόγραμμα ενημερωμένο, ώστε τα μέλη να βλέπουν πάντα τα πραγματικά διαθέσιμα μαθήματα.
Trigger	Ο εκπαιδευτής ή ο διαχειριστής επιλέγει «Προσθήκη» στις ώρες μαθημάτων ή ακύρωση ενός υπάρχοντος μαθήματος.
Preconditions	Συνδεδεμένος με ρόλο εκπαιδευτή ή διαχειριστή, υπάρχει τουλάχιστον ένας τύπος μαθήματος.
Postconditions	Δημιουργείται/ενημερώνεται/ακυρώνεται ένα προγραμματισμένο μάθημα και ενημερώνεται το πρόγραμμα.
Primary path	1. Ο εκπαιδευτής επιλέγει «Νέο μάθημα». 2. Ορίζει τύπο μαθήματος, ημερομηνία και ώρα, εκπαιδευτή και χωρητικότητα. 3. Το σύστημα δημιουργεί το μάθημα και το εμφανίζει στο εβδομαδιαίο πρόγραμμα.
Alternate path	Ακύρωση ενός υπάρχοντος μαθήματος.
Exception path	Ελλιπή στοιχεία → μήνυμα σφάλματος. Χρήστης χωρίς δικαίωμα → απόρριψη πρόσβασης.
Priority / Frequency of Use	Υψηλή / Μέτρια.
Acceptance criteria	**Δεδομένου ότι** υπάρχει τύπος μαθήματος, **όταν** ο εκπαιδευτής δημιουργήσει νέο μάθημα με μελλοντική ημερομηνία, **τότε** το μάθημα εμφανίζεται στο εβδομαδιαίο πρόγραμμα των μελών. **Δεδομένου ότι** η ημερομηνία είναι στο παρελθόν ή ο τύπος μαθήματος δεν υπάρχει, **όταν** υποβληθεί η φόρμα, **τότε** η δημιουργία απορρίπτεται. **Δεδομένου ότι** ο χρήστης είναι απλό

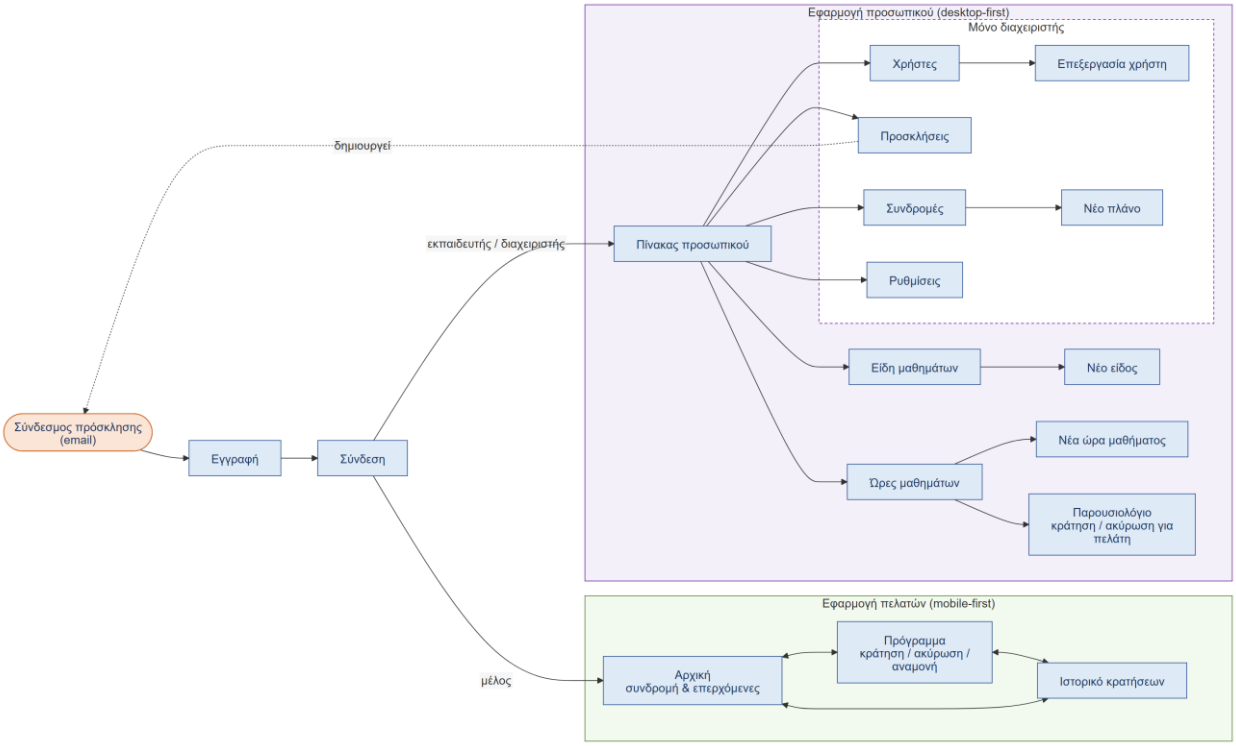
UC-05 — Δημιουργία και διαχείριση μαθήματος	
	μέλος, **όταν** επιχειρήσει να δημιουργήσει τύπο μαθήματος, **τότε** λαμβάνει άρνηση πρόσβασης (403).
Traceability	FR-10, FR-11, NFR-05 · οθόνες: Ώρες μαθημάτων (Εικόνες 8 και 14), Είδη μαθημάτων · tests: ClassSessionsTests, ClassTypesTests

Πίνακας 9 — Περίπτωση χρήσης UC-06: Ανάθεση συνδρομής σε χρήστη.

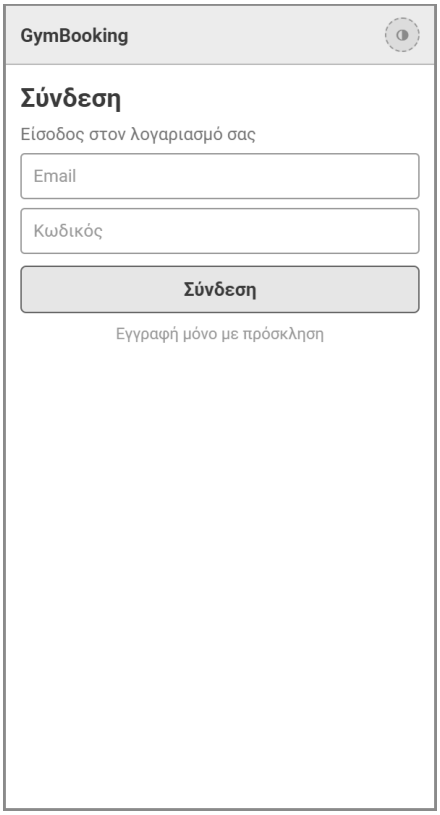
UC-06 — Ανάθεση συνδρομής σε χρήστη	
Actors	Διαχειριστής (κύριος).
Description	Ο διαχειριστής αναθέτει ένα συνδρομητικό πλάνο σε έναν χρήστη.
Goal / Value	Το μέλος αποκτά δικαίωμα κράτησης σύμφωνα με το πακέτο που επέλεξε.
Trigger	Ο διαχειριστής επιλέγει «Συνδρομές» για να αναθέσει πλάνο σε ένα μέλος.
Preconditions	Συνδεδεμένος ως διαχειριστής, υπάρχει το συνδρομητικό πλάνο και ο χρήστης.
Postconditions	Ο χρήστης αποκτά ενεργή συνδρομή με διάρκεια και υπόλοιπο συνεδριών.
Primary path	1. Ο διαχειριστής επιλέγει «Συνδρομές» και τον χρήστη (μέσω email). 2. Επιλέγει το συνδρομητικό πλάνο. 3. Το σύστημα δημιουργεί την ενεργή συνδρομή.
Alternate path	—
Exception path	Ανύπαρκτος χρήστης ή πλάνο → μήνυμα σφάλματος. Χρήστης χωρίς δικαίωμα → απόρριψη.
Priority / Frequency of Use	Μέτρια / Χαμηλή.
Acceptance criteria	**Δεδομένου ότι** υπάρχουν το πλάνο και το μέλος, **όταν** ο διαχειριστής αναθέσει το πλάνο, **τότε** το μέλος βλέπει την ενεργή συνδρομή του στην αρχική οθόνη και μπορεί να κάνει κράτηση. **Δεδομένου ότι** το πλάνο είναι πακέτο συνεδριών, **όταν** το μέλος κάνει κράτηση, **τότε** το υπόλοιπο των συνεδριών μειώνεται κατά 1.
Traceability	FR-06, FR-14 · οθόνες: Συνδρομές, Αρχική πελάτη (Εικόνα 10) · tests: SubscriptionsTests, MembershipPlansTests, BookingSubscriptionTests

3.6 Σχεδίαση της διεπαφής χρήστη (wireframes)

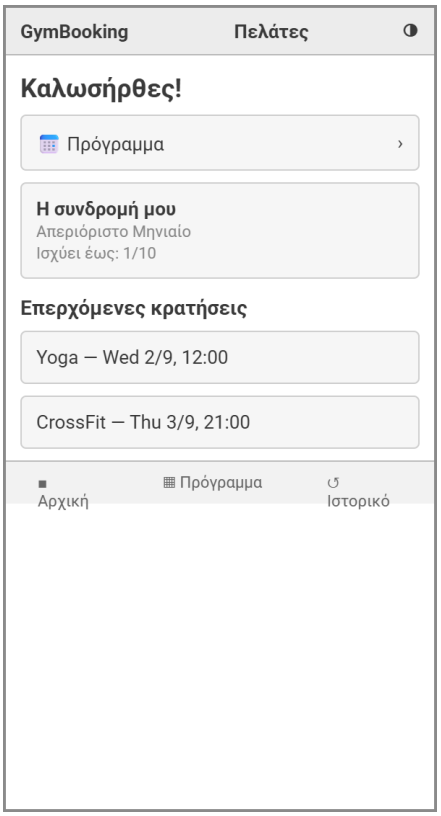
Πριν από την υλοποίηση, σχεδιάστηκαν ασπρόμαυρα wireframes χαμηλής πιστότητας (lo-fi) για τις βασικές οθόνες. Τα wireframes δημιουργήθηκαν ως απλές σελίδες HTML και ελέγχθηκαν έναντι των περιπτώσεων χρήσης και των απαιτήσεων: κάθε οθόνη αντιστοιχεί σε συγκεκριμένα UC και FR. Η ροή μεταξύ των οθονών των δύο εφαρμογών φαίνεται στο Διάγραμμα 2. Ακολουθούν η οθόνη σύνδεσης (Εικόνα 4), η αρχική οθόνη και το πρόγραμμα της εφαρμογής πελατών (Εικόνες 5–6), καθώς και ο πίνακας προσωπικού, η διαχείριση ωρών μαθημάτων και η διαχείριση χρηστών (Εικόνες 7–9).



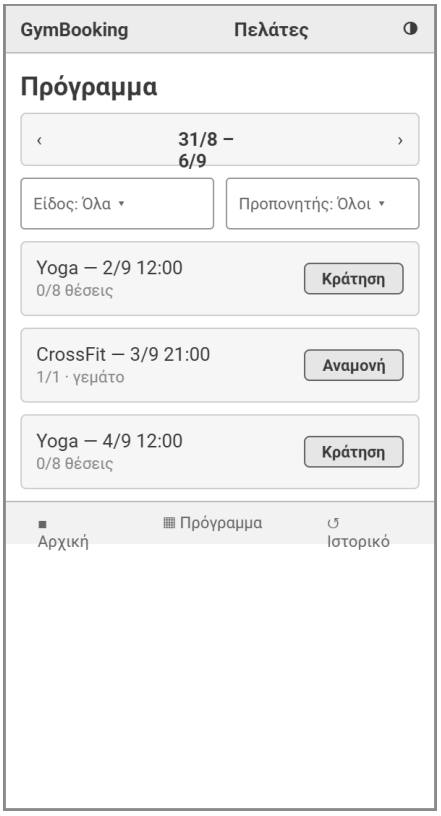
Διάγραμμα 2 *— Χάρτης ροής οθονών των δύο εφαρμογών.*



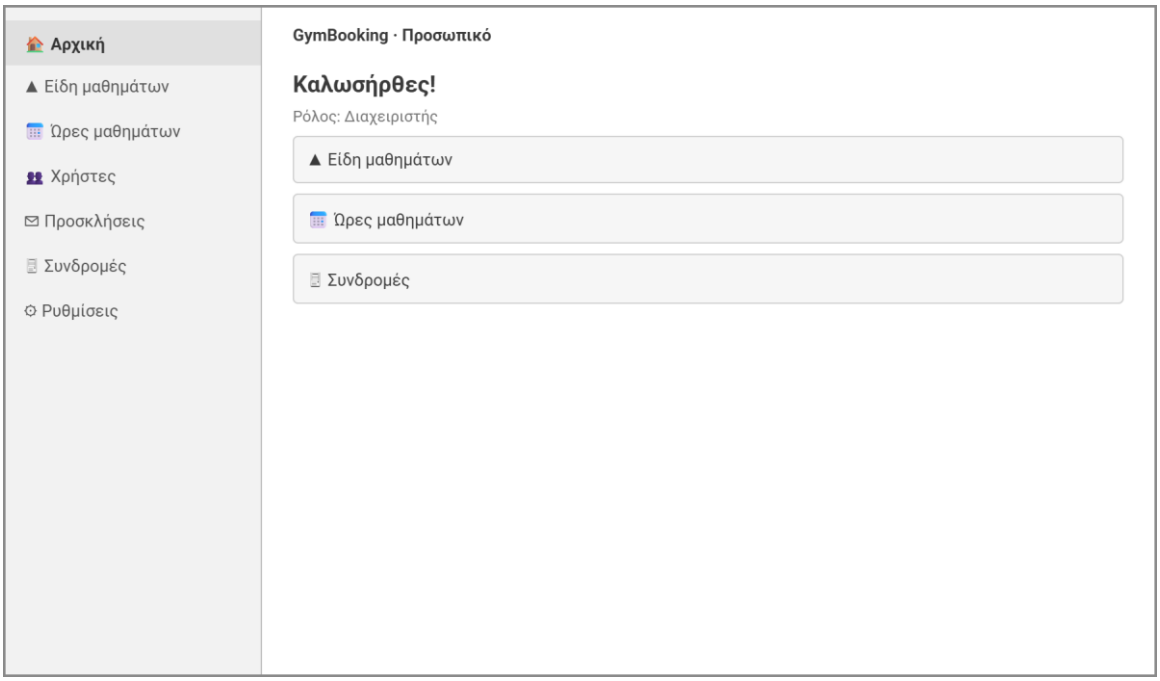
Εικόνα 4 *— Wireframe: οθόνη σύνδεσης (login).*



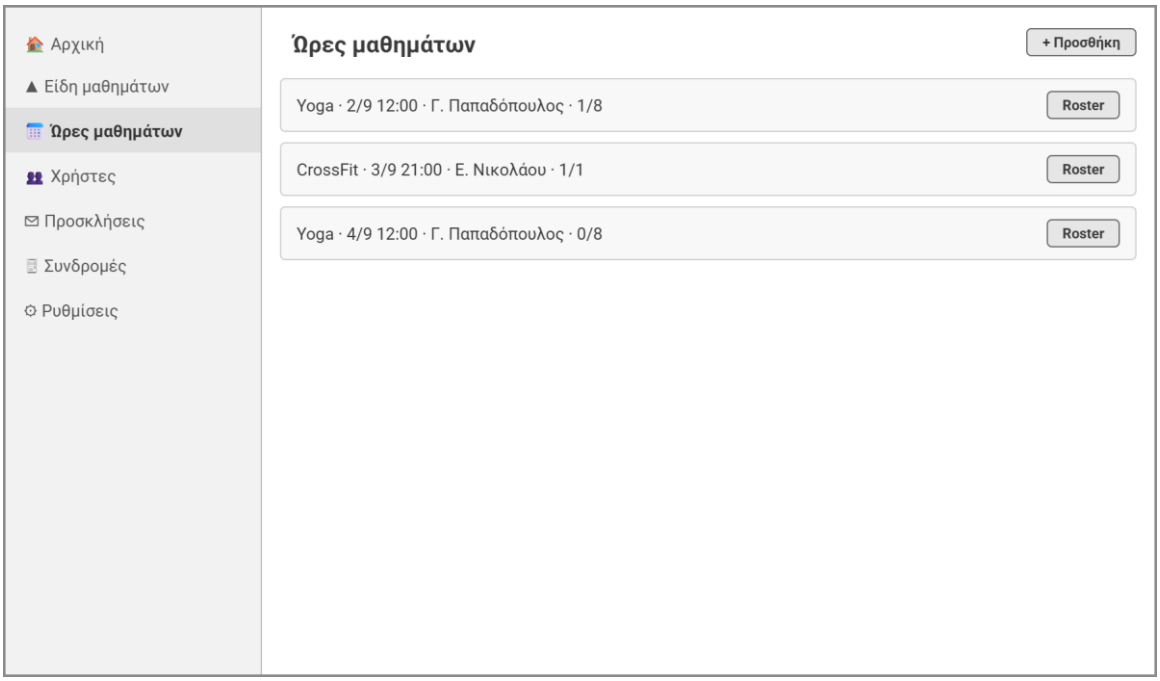
Εικόνα 5 — Wireframe: αρχική οθόνη πελάτη.



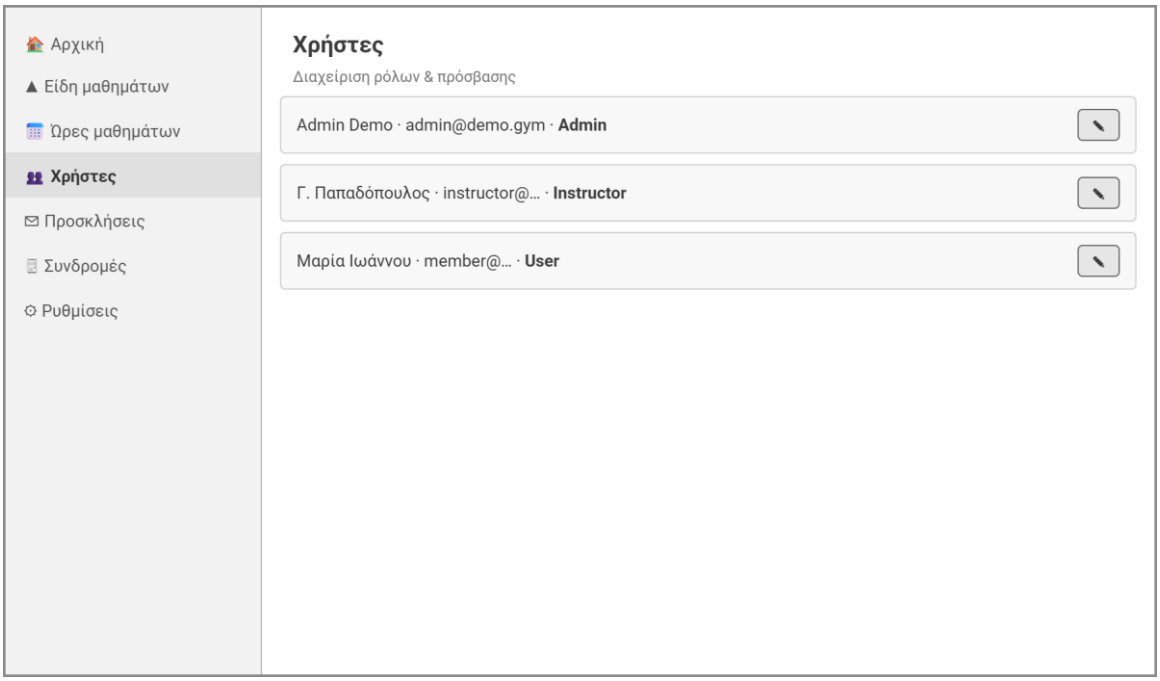
Εικόνα 6 — Wireframe: εβδομαδιαίο πρόγραμμα και κράτηση.



Εικόνα 7 — Wireframe: πίνακας προσωπικού (dashboard).



Εικόνα 8 — Wireframe: διαχείριση ωρών μαθημάτων.



Εικόνα 9 — Wireframe: διαχείριση χρηστών.

3.6.1 Καταστάσεις της διεπαφής

Πέρα από την κύρια διαδρομή, η διεπαφή χειρίζεται και τις υπόλοιπες καταστάσεις που μπορεί να συναντήσει ο χρήστης:

- **Φόρτωση:** εμφανίζεται ένδειξη όσο φορτώνουν το πρόγραμμα, οι κρατήσεις και το ιστορικό.
- **Κενή κατάσταση:** όταν δεν υπάρχουν δεδομένα, εμφανίζεται μήνυμα με εικονίδιο, π.χ. «Δεν υπάρχουν μαθήματα αυτή την εβδομάδα», «Κενό ιστορικό» ή «Δεν υπάρχουν είδη μαθημάτων ακόμα».
- **Σφάλμα:** εμφανίζεται πλαίσιο με το μήνυμα του διακομιστή (π.χ. γεμάτο μάθημα ή χρονική επικάλυψη), ενώ στη σύνδεση υπάρχουν ξεχωριστά μηνύματα για λάθος στοιχεία, κλειδωμένο λογαριασμό και υπερβολικό αριθμό προσπαθειών.
- **Χωρίς δικαίωμα:** οι επιλογές που δεν αντιστοιχούν στον ρόλο του χρήστη δεν εμφανίζονται στο μενού, ενώ οι φύλακες διαδρομών (guards) επιστρέφουν τον χρήστη στη σύνδεση. Ειδική περίπτωση είναι το μέλος χωρίς ενεργή συνδρομή, που βλέπει ενημερωτικό μήνυμα ότι δεν μπορεί να κάνει κράτηση ή να μπει σε λίστα αναμονής.
- **Επιτυχία:** μετά από κράτηση ή ακύρωση, το πρόγραμμα ανανεώνεται αμέσως, το κουμπί αλλάζει από «Κράτηση» σε «Ακύρωση» και ενημερώνεται η πληρότητα του μαθήματος.

3.6.2 Από τα wireframes στην τελική εφαρμογή

Η τελική εφαρμογή διατήρησε τη διάταξη και τη ροή των wireframes, με ορισμένες αλλαγές που φαίνονται συγκρίνοντας τις Εικόνες 4–9 με τις Εικόνες 10–15 του κεφαλαίου 6:

- **Εμφάνιση:** Σκοτεινό θέμα με διαφορετικό βασικό χρώμα ανά εφαρμογή (πράσινο για τα μέλη, μωβ για το προσωπικό), ώστε ο χρήστης να αναγνωρίζει αμέσως σε ποια εφαρμογή βρίσκεται.
- **Πρόγραμμα:** κάθε μάθημα δείχνει πλέον και τον εκπαιδευτή, ενώ η πληρότητα εμφανίζεται με εικονίδιο και κόκκινο χρώμα όταν το μάθημα γεμίσει. Για μάθημα που το μέλος έχει ήδη κλείσει, το κουμπί γίνεται «Ακύρωση», ώστε η ακύρωση να γίνεται από το ίδιο σημείο. Τα

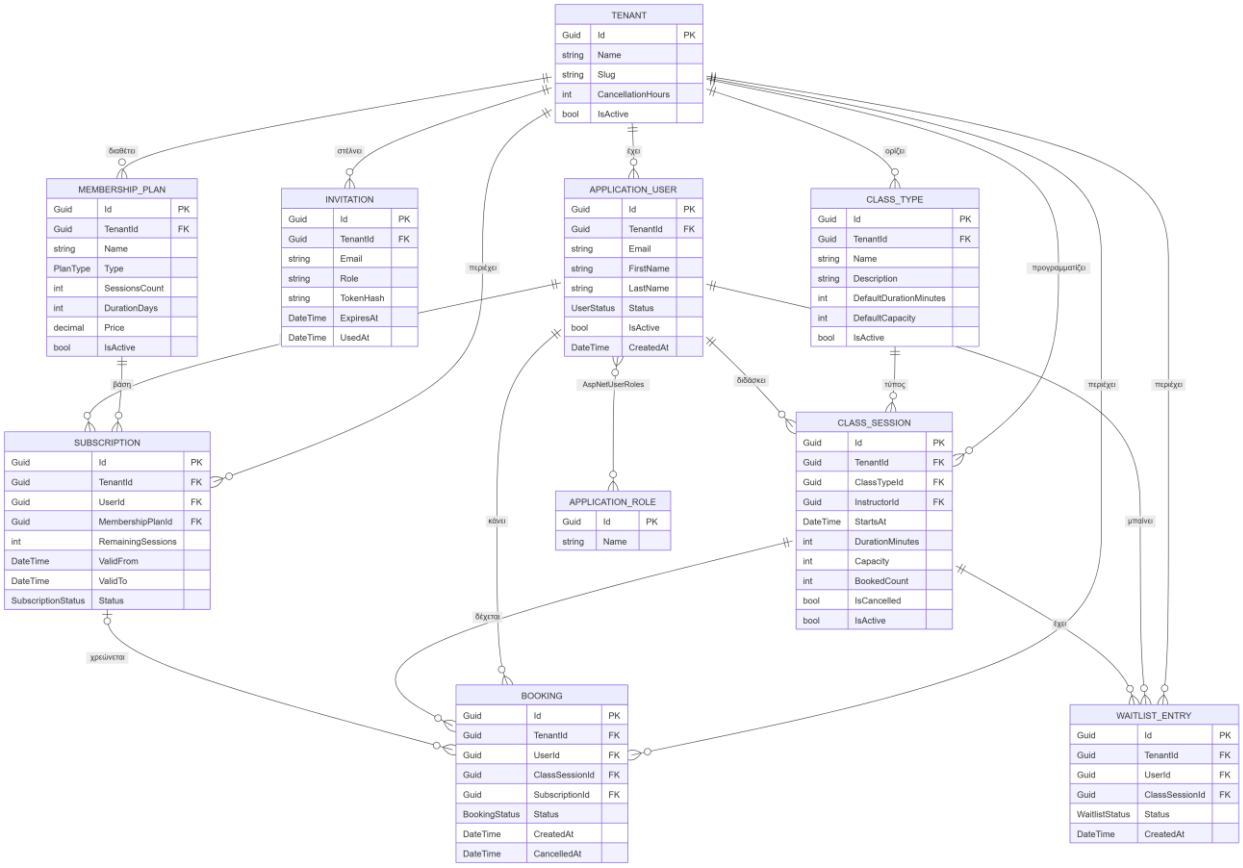
φίλτρα έγιναν τυπικά πεδία επιλογής του Angular Material με ετικέτα, για καλύτερη προσβασιμότητα.

- **Ημερομηνίες:** προστέθηκε η ημέρα της εβδομάδας, ενώ η συνδρομή δείχνει πλήρη ημερομηνία λήξης και τον τύπο του πλάνου.
- **Εφαρμογή προσωπικού:** το πλευρικό μενού μαζεύεται (κουμπί ☰), ώστε η εφαρμογή να λειτουργεί και σε μικρότερες οθόνες. Οι λίστες έγιναν δύο γραμμών (τίτλος και λεπτομέρειες) για ευκολότερη ανάγνωση, και στη λίστα χρηστών προστέθηκε η κατάσταση του λογαριασμού (ενεργός ή ανενεργός).

3.7 Σχεδίαση βάσης δεδομένων (ERD)

Η βάση δεδομένων οργανώνεται γύρω από τον Tenant, στον οποίο ανήκουν όλες οι υπόλοιπες οντότητες [4]. Οι κύριες οντότητες είναι οι χρήστες (με τους ρόλους τους), οι προσκλήσεις, οι τύποι μαθημάτων, τα προγραμματισμένα μαθήματα, οι κρατήσεις, τα συνδρομητικά πλάνα, οι συνδρομές και οι εγγραφές στη λίστα αναμονής.

Κάθε προγραμματισμένο μάθημα συνδέεται με έναν τύπο μαθήματος και έναν εκπαιδευτή και έχει συγκεκριμένη χωρητικότητα. Μια κράτηση συνδέει έναν χρήστη με ένα μάθημα και προαιρετικά με τη συνδρομή από την οποία χρεώθηκε. Για την αποφυγή διπλών κρατήσεων χρησιμοποιούνται unique indexes στη βάση, ενώ η διαγραφή είναι τύπου soft-delete, ώστε να μη χάνονται δεδομένα που είναι χρήσιμα για την διατήρηση ιστορικού. Το πλήρες διάγραμμα οντοτήτων–συσχετίσεων παρουσιάζεται στο Διάγραμμα 3.



Διάγραμμα 3 — Οντοτήτων–συσχετίσεων (ERD).

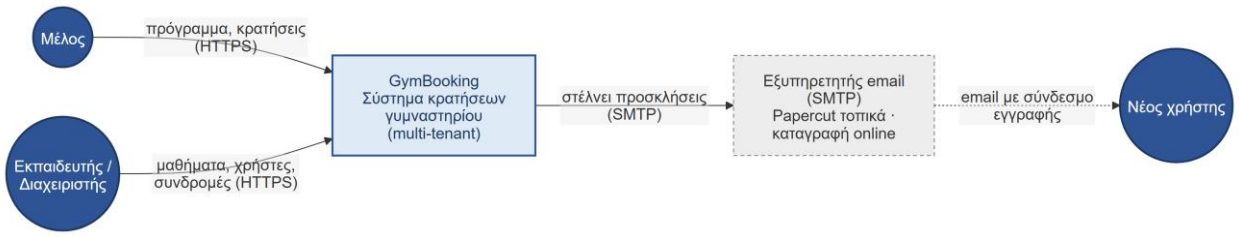
4. Αρχιτεκτονική και υλοποίηση

Στο κεφάλαιο αυτό περιγράφεται ο τρόπος με τον οποίο λειτουργεί η λύση: η γενική αρχιτεκτονική σε επίπεδα, ο τρόπος με τον οποίο το σύστημα εξυπηρετεί πολλά γυμναστήρια, η ταυτοποίηση και η εξουσιοδότηση, οι τεχνολογίες που χρησιμοποιήθηκαν, οι ροές εκτέλεσης των βασικών λειτουργιών, τα πιο απαιτητικά χαρακτηριστικά, η ασφάλεια και η εκτέλεση της εφαρμογής με Docker, καθώς και οι βασικές αρχιτεκτονικές αποφάσεις.

4.1 Γενική αρχιτεκτονική (frontend / backend / βάση)

4.1.1 Πλαίσιο συστήματος (system context)

Σε υψηλό επίπεδο, το σύστημα αλληλεπιδρά με τρεις κατηγορίες ανθρώπων και με λίγες εξωτερικές υπηρεσίες (Διάγραμμα 4). Τα μέλη χρησιμοποιούν την εφαρμογή πελατών από το κινητό ή τον υπολογιστή τους, ενώ οι εκπαιδευτές και οι διαχειριστές χρησιμοποιούν την εφαρμογή προσωπικού. Εξωτερικά, το σύστημα χρησιμοποιεί μόνο έναν εξυπηρετητή email (SMTP) για την αποστολή των προσκλήσεων, που στην τοπική εκτέλεση είναι το Papercut. Η περιγραφή ακολουθεί το μοντέλο C4, με διαγράμματα πλαισίου (context) και containers, χωρίς το επίπεδο του κώδικα [24].



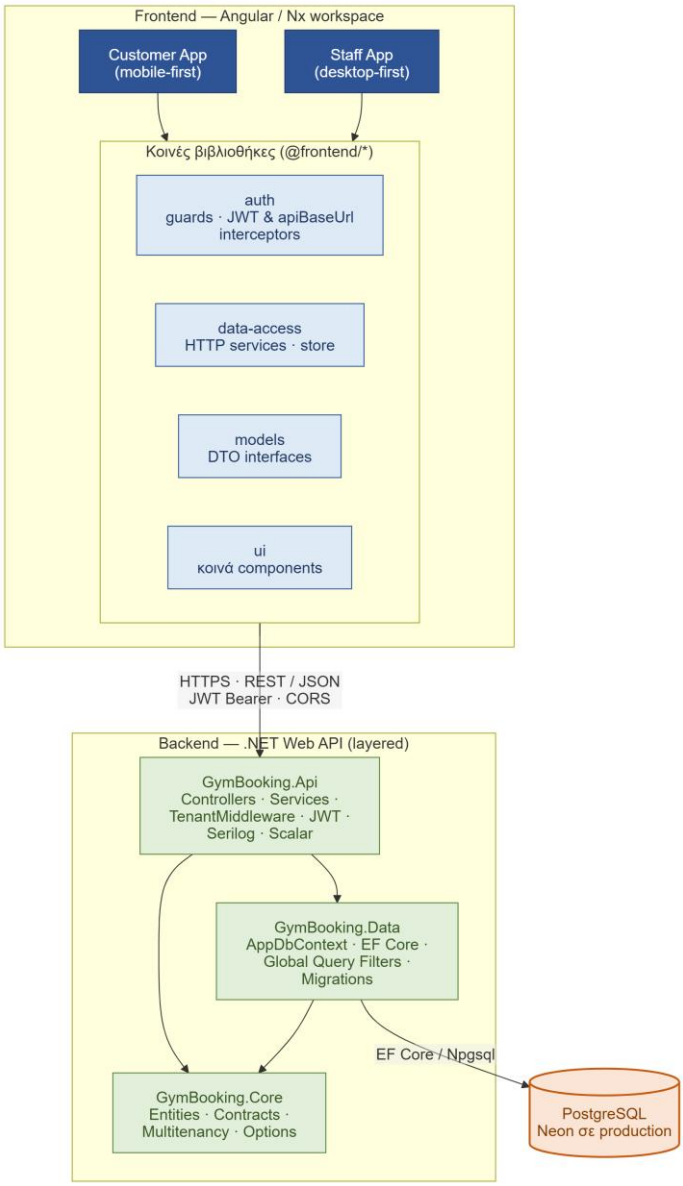
Διάγραμμα 4 *— Πλαίσιο συστήματος (C4 – system context).*

4.1.2 Containers και επικοινωνία

Η εφαρμογή ακολουθεί μια κλασική αρχιτεκτονική τριών επιπέδων: το frontend (πλευρά χρήστη), το backend (server) και τη βάση δεδομένων [25]. Τα τρία μέρη επικοινωνούν μέσω ενός REST API πάνω από HTTP (HTTPS σε εγκατάσταση παραγωγής), με ανταλλαγή δεδομένων σε μορφή JSON.

Το frontend αναπτύχθηκε ως ενιαίο (monorepo) με το Nx και περιλαμβάνει δύο ξεχωριστές εφαρμογές: την εφαρμογή των πελατών, σχεδιασμένη κυρίως για κινητά, και την εφαρμογή του προσωπικού, σχεδιασμένη για υπολογιστή(100% responsive σε κινητά). Οι δύο εφαρμογές μοιράζονται κοινές βιβλιοθήκες για τα μοντέλα δεδομένων, την πρόσβαση στο API, την ταυτοποίηση και τα κοινά στοιχεία επικοινωνίας με τον Server.

Το backend είναι μια εφαρμογή Web API οργανωμένη σε επίπεδα: το επίπεδο παρουσίασης (controllers και services), το επίπεδο πυρήνα με τις οντότητες και τη λογική, και το επίπεδο δεδομένων που επικοινωνεί με τη βάση [18], [26]. Η επικοινωνία με τη βάση γίνεται μέσω του Entity Framework Core. Η γενική εικόνα της αρχιτεκτονικής φαίνεται στο Διάγραμμα 5.



Διάγραμμα 5 — Αρχιτεκτονική του συστήματος.

Κάθε container έχει σαφή ευθύνη. Οι δύο εφαρμογές Angular εκτελούνται στον browser και επικοινωνούν με το API μέσω HTTP και JSON, με το JWT στην επικεφαλίδα Authorization. Το API (ASP.NET Core) εφαρμόζει τους κανόνες της λογικής και την εξουσιοδότηση, ενώ η PostgreSQL είναι η μοναδική πηγή αλήθειας για τα δεδομένα, μόνο το API έχει πρόσβαση σε αυτήν. Η επικύρωση των δεδομένων γίνεται σε δύο σημεία: στις φόρμες του ui, για άμεσα errors χωρίς την ανάγκη επικοινωνίας με το API, και υποχρεωτικά στο API, το οποίο απορρίπτει τα μη έγκυρα αιτήματα με HTTP 400.

4.2 Multi-tenancy

Το σύστημα σχεδιάστηκε ώστε να μπορεί να εξυπηρετεί πολλά γυμναστήρια από την ίδια εγκατάσταση. Κάθε γυμναστήριο αποτελεί έναν ξεχωριστό Tenant και τα δεδομένα όλων αποθηκεύονται στην ίδια βάση, με ένα πεδίο αναγνωριστικού (TenantId) σε κάθε εγγραφή [9], [10].

Ο διαχωρισμός των δεδομένων επιβάλλεται αυτόματα από τη βάση δεδομένων, μέσω καθολικών φίλτρων (global query filters) του Entity Framework Core: κάθε request περιορίζεται αυτόματα στα δεδομένα του τρέχοντος Tenant. Το TenantId δεν το δηλώνει ποτέ ο χρήστης, αλλά προκύπτει από το token της σύνδεσης (JWT), ώστε ένας χρήστης να μην μπορεί να δει δεδομένα άλλου γυμναστηρίου.

Ο διαχωρισμός των δεδομένων υλοποιείται σε δύο σημεία (Κώδικας 1). Ένα middleware διαβάζει σε κάθε αίτημα το αναγνωριστικό του γυμναστηρίου από το JWT, και το AppDbContext ορίζει ένα καθολικό φίλτρο για κάθε οντότητα, ώστε κάθε ερώτημα να περιορίζεται αυτόματα στο τρέχον γυμναστήριο.

Κώδικας 1 — Επιβολή της απομόνωσης ανά γυμναστήριο (C#, backend/src/GymBooking.Api/Services/TenantMiddleware.cs και backend/src/GymBooking.Data/AppDbContext.cs, γρ. 34–41).

```
var tenantIdClaim = context.User.FindFirst("tenantId")?.Value;
if (Guid.TryParse(tenantIdClaim, out var tenantId))
{
    currentTenant.SetTenant(tenantId);
}

modelBuilder.Entity<ClassSession>().HasQueryFilter(s => s.TenantId == _currentTenant.TenantId);
modelBuilder.Entity<Booking>().HasQueryFilter(b => b.TenantId == _currentTenant.TenantId);
```

4.3 Ταυτοποίηση και εξουσιοδότηση (JWT, ρόλοι)

Η διαχείριση των χρηστών και των κωδικών στηρίζεται στο ASP.NET Core Identity [27]. Κατά τη σύνδεση, ο server επαληθεύει τα στοιχεία και δίνει ένα token (JWT), το οποίο ο client στέλνει σε κάθε επόμενο αίτημα. Το token περιέχει το αναγνωριστικό του χρήστη, τον Tenant στον οποίο ανήκει και τους ρόλους του [17].

Υποστηρίζονται τρεις ρόλοι: το μέλος, ο εκπαιδευτής και ο διαχειριστής. Η πρόσβαση στα διάφορα σημεία του API ελέγχεται με βάση τον ρόλο: ορισμένες ενέργειες επιτρέπονται σε κάθε συνδεδεμένο χρήστη, άλλες μόνο σε εκπαιδευτές και διαχειριστές, και άλλες αποκλειστικά σε διαχειριστές. Ο ίδιος έλεγχος γίνεται και στην πλευρά του χρήστη, όπου φύλακες(guards) δεν επιτρέπουν την πρόσβαση σε οθόνες που δεν έχει το δικαίωμα να δει ο χρήστης.

4.4 Τεχνολογίες που χρησιμοποιήθηκαν

Οι βασικές τεχνολογίες και τα εργαλεία που χρησιμοποιήθηκαν για την ανάπτυξη της εφαρμογής παρουσιάζονται παρακάτω, χωρισμένα ανά μέρος του συστήματος.

Οι ακριβείς εκδόσεις των βασικών τεχνολογιών, ο ρόλος τους και ο λόγος επιλογής τους συνοψίζονται στον Πίνακα 10. Πέρα από τα τεχνικά κριτήρια, η επιλογή στηρίχθηκε και στην προηγούμενη εξοικείωση μου με το Angular και το .NET.

Πίνακας 10 — Τεχνολογίες, εκδόσεις και αιτιολόγηση της επιλογής τους.

Τεχνολογία	Έκδοση	Ρόλος	Γιατί επιλέχθηκε
.NET / ASP.NET Core	10 (SDK 10.0.401)	Backend, REST API	Ώριμο και γρήγορο framework με ενσωματωμένα Identity, dependency injection, rate limiting και OpenAPI
Entity Framework Core + Npgsql	10.0	ORM, migrations	Καθολικά φίλτρα για multi-tenancy, code-first migrations
ASP.NET Core Identity	10.0	Χρήστες, κωδικοί, lockout	Έτοιμο και ελεγμένο hashing και κλείδωμα λογαριασμού, χωρίς δική μας κρυπτογραφία
PostgreSQL	16	Βάση δεδομένων	Ανοιχτού κώδικα, αξιόπιστα transactions και SELECT … FOR UPDATE
Angular + Angular Material	21.2	Frontend (2 SPA)	Ολοκληρωμένο framework με signals, guards και interceptors, προσβάσιμα components

Τεχνολογία	Έκδοση	Ρόλος	Γιατί επιλέχθηκε
Nx	23.0	Monorepo	Κοινές βιβλιοθήκες ανάμεσα στις δύο εφαρμογές και όρια ανάμεσα στα modules
Tailwind CSS	4.3	Μορφοποίηση	Γρήγορη και συνεπής μορφοποίηση δίπλα στο Material(πολύ καλό responsiveness)
TypeScript / Node.js	5.9 / 20	Γλώσσα frontend / build	Στατικοί τύποι στο frontend, περιβάλλον build και εργαλείων
Serilog	10.0	Logging	Δομημένα logs σε κονσόλα και αρχείο
Scalar / OpenAPI	2.16	Τεκμηρίωση API	Διαδραστική τεκμηρίωση και δοκιμή των endpoints
xUnit / Testcontainers	2.9 / 4.13	Δοκιμές	Αυτοματοποιημένες δοκιμές, και σε πραγματική PostgreSQL
Docker / Docker Compose	—	Εκτέλεση	Ολόκληρο το σύστημα με μία εντολή, ίδιο σε κάθε περιβάλλον

4.4.1 Frontend (Angular, Nx, Angular Material, Tailwind CSS)

Η πλευρά του χρήστη υλοποιήθηκε με Angular, ένα σύγχρονο πλαίσιο για διαδικτυακές εφαρμογές μίας σελίδας (SPA). Ο κώδικας οργανώθηκε σε ενιαίο χώρο εργασίας (monorepo) με το Nx, το οποίο διευκολύνει την κοινή χρήση βιβλιοθηκών ανάμεσα στις δύο εφαρμογές και επιβάλλει σαφή όρια μεταξύ των τμημάτων του κώδικα.

- Angular: το κύριο πλαίσιο ανάπτυξης της διεπαφής [28].
- Nx: εργαλείο διαχείρισης του monorepo και των κοινών βιβλιοθηκών [29].
- Angular Material: έτοιμα, προσβάσιμα UI components (πεδία, κουμπιά, μενού κ.λπ.) [30].
- Tailwind CSS: βιβλιοθήκη για γρήγορη και συνεπή μορφοποίηση [31].

4.4.2 Backend (.NET, ASP.NET Core, EF Core, ASP.NET Core Identity)

Η πλευρά του διακομιστή υλοποιήθηκε σε .NET με το ASP.NET Core, ως εφαρμογή Web API [32], [33].

- ASP.NET Core: το πλαίσιο για την κατασκευή του REST API.
- Entity Framework Core: ORM που διαχειρίζεται την πρόσβαση στα δεδομένα και τις μεταβολές στο σχήμα της βάσης(migrations) [5].
- ASP.NET Core Identity: η διαχείριση χρηστών, κωδικών και ρόλων.
- Serilog: η καταγραφή συμβάντων (logging) του διακομιστή [34].
- Scalar / OpenAPI: η αυτόματη, διαδραστική τεκμηρίωση του API [35].

4.4.3 Βάση δεδομένων (PostgreSQL)

Ως σύστημα διαχείρισης βάσης δεδομένων επιλέχθηκε η PostgreSQL, μια σχεσιακή βάση ανοιχτού κώδικα [36]. Η επικοινωνία με το backend γίνεται μέσω του Entity Framework Core και Npgsql [37]. Η PostgreSQL υποστηρίζει επίσης τα κλειδώματα εγγραφών που χρησιμοποιούνται για την ασφαλή διαχείριση των ταυτόχρονων κρατήσεων.

4.4.4 Υποδομή και εργαλεία (Docker, Git/GitHub)

Για την ανάπτυξη και την εκτέλεση της εφαρμογής χρησιμοποιήθηκαν επιπλέον τα ακόλουθα εργαλεία:

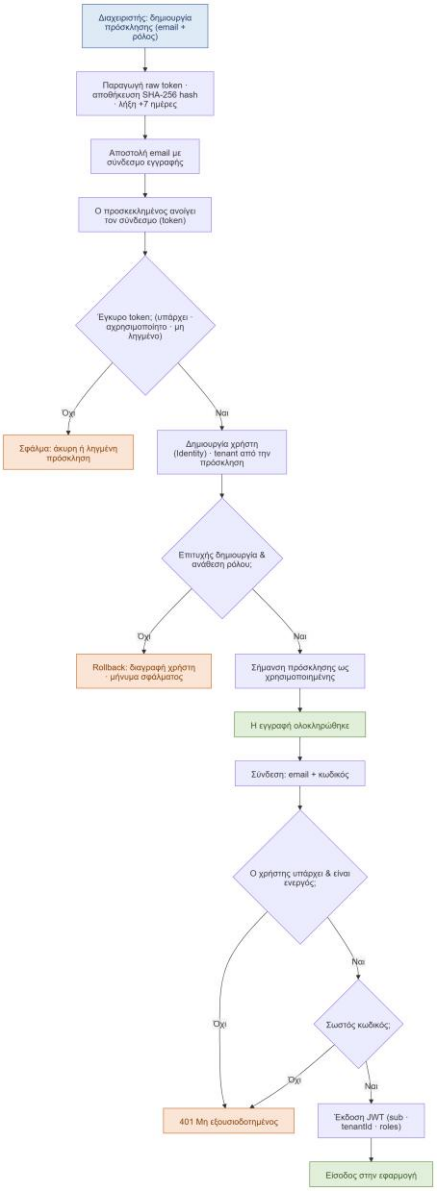
- Docker και Docker Compose: για την εκτέλεση ολόκληρου του συστήματος (βάση, API και εφαρμογές) τοπικά, με μία εντολή και χωρίς χειροκίνητη εγκατάσταση [8].
- Git και GitHub: για τον έλεγχο εκδόσεων και τη φύλαξη του κώδικα.

4.5 Ροές εκτέλεσης βασικών λειτουργιών

Οι πιο σημαντικές λειτουργίες της εφαρμογής παρουσιάζονται παρακάτω με τη μορφή διαγραμμάτων ροής, τα οποία δείχνουν τη σειρά των ελέγχων και των αποφάσεων που εκτελούνται σε κάθε περίπτωση, ώστε να γίνεται κατανοητή η εσωτερική λογική του συστήματος.

4.5.1 Εγγραφή και σύνδεση χρήστη (invitation / registration)

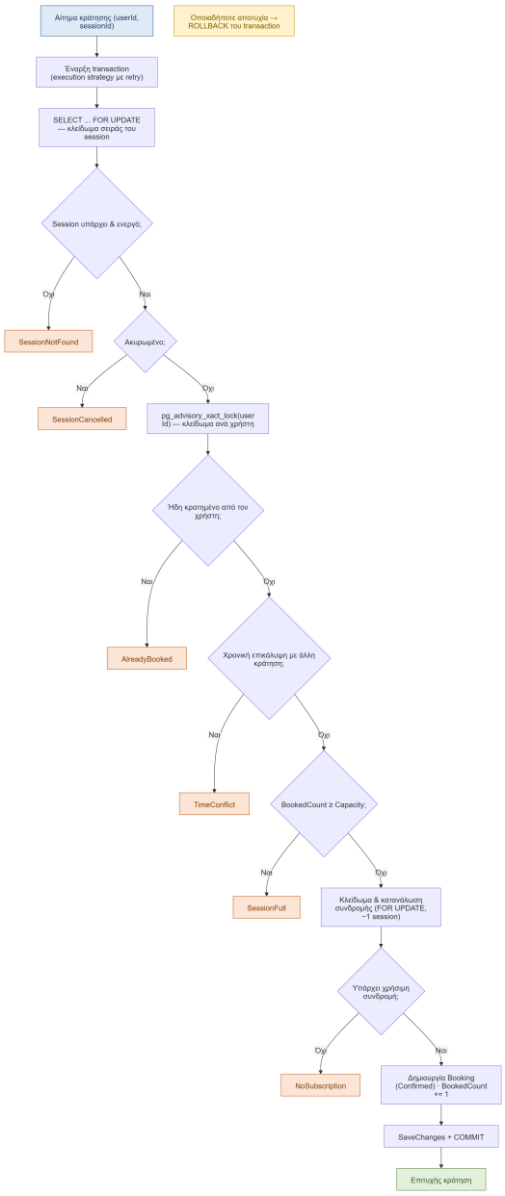
Η εγγραφή νέου χρήστη γίνεται μόνο κατόπιν πρόσκλησης από διαχειριστή. Ο διαχειριστής δημιουργεί μια πρόσκληση για ένα email και έναν ρόλο, το σύστημα παράγει έναν προσωρινό σύνδεσμο και τον αποθηκεύει κρυπτογραφημένο. Ο προσκεκλημένος ανοίγει τον σύνδεσμο, το σύστημα ελέγχει ότι είναι έγκυρος και δεν έχει χρησιμοποιηθεί ή λήξει, και στη συνέχεια δημιουργείται ο λογαριασμός. Κατά τη σύνδεση, εφόσον τα στοιχεία είναι σωστά και ο λογαριασμός ενεργός, εκδίδεται το token πρόσβασης. Η πλήρης ροή φαίνεται στο Διάγραμμα 6.



Διάγραμμα 6 *— Ροή εγγραφής μέσω πρόσκλησης και σύνδεσης.*

4.5.2 Κράτηση θέσης σε μάθημα (atomic booking)

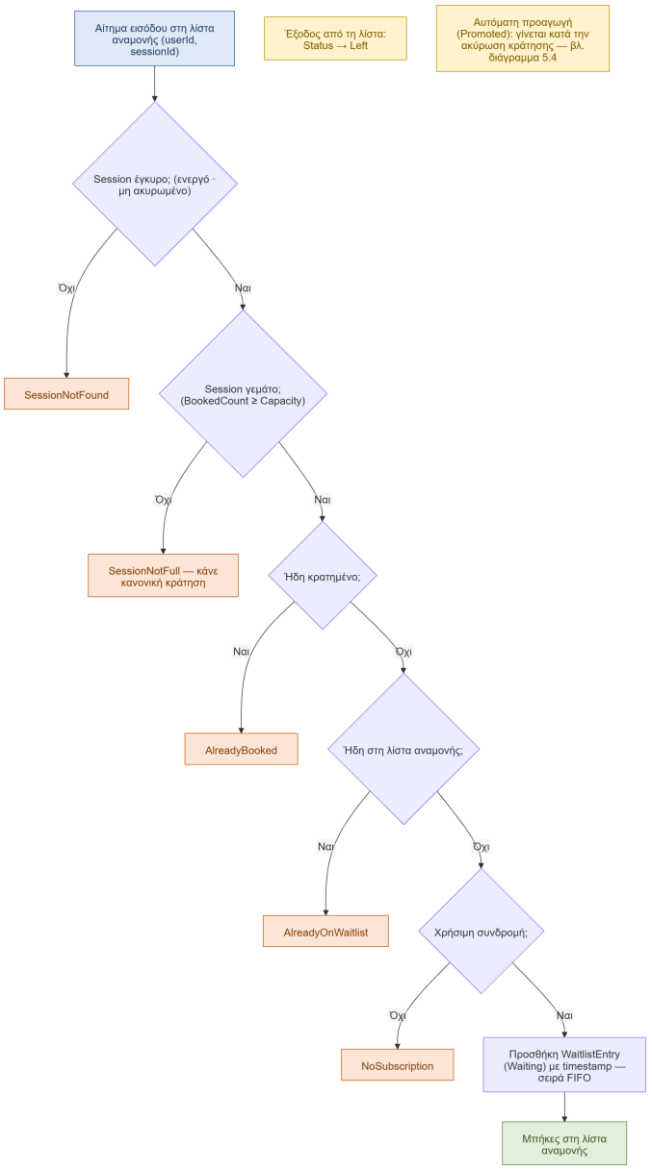
Η κράτηση μιας θέσης είναι η πιο κρίσιμη λειτουργία, γιατί πολλοί χρήστες μπορεί να προσπαθούν ταυτόχρονα να κλείσουν θέση στο ίδιο μάθημα. Για τον λόγο αυτό, η κράτηση εκτελείται μέσα σε ένα transaction, όπου κλειδώνονται προσωρινά πρώτα η εγγραφή του μαθήματος και μετά ο χρήστης που κάνει την κράτηση, ώστε να γίνονται σειριακά οι απαραίτητοι έλεγχοι: ότι το μάθημα υπάρχει και δεν έχει ακυρωθεί, ότι ο χρήστης δεν έχει ήδη κράτηση, ότι δεν υπάρχει χρονική επικάλυψη, ότι υπάρχει διαθέσιμη θέση και ότι υπάρχει ενεργή συνδρομή. Μόνο αν όλα ισχύουν, καταχωρείται η κράτηση και ενημερώνεται ο αριθμός των θέσεων. Σε κάθε αποτυχία, το transaction ακυρώνεται. Η ροή αυτή φαίνεται στο Διάγραμμα 7.



Διάγραμμα 7 — Ροή ατομικής κράτησης θέσης (atomic booking).

4.5.3 Λίστα αναμονής (waitlist)

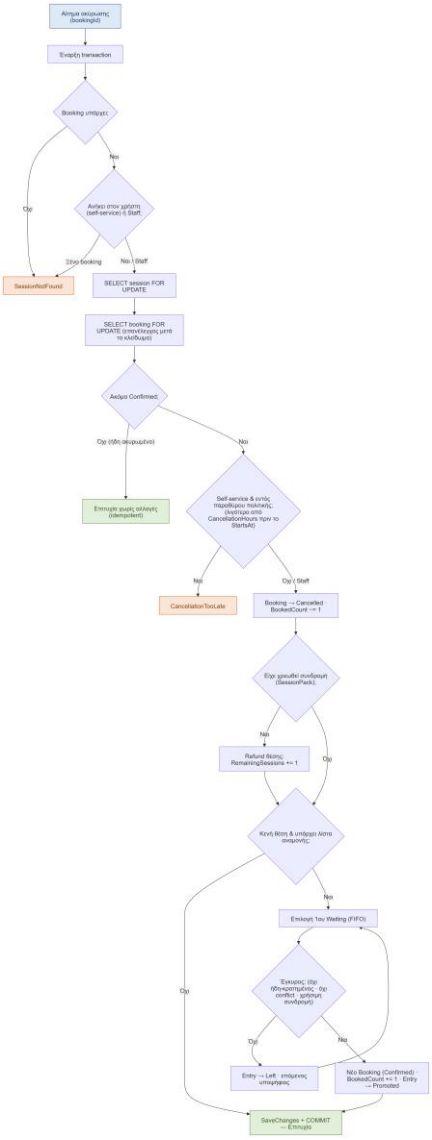
Όταν ένα μάθημα είναι γεμάτο, ο χρήστης μπορεί να μπει στη λίστα αναμονής. Το σύστημα ελέγχει ότι το μάθημα είναι πράγματι γεμάτο, ότι ο χρήστης δεν έχει ήδη κράτηση ή θέση στη λίστα και ότι διαθέτει ενεργή συνδρομή, εφόσον ισχύουν, καταχωρείται στη λίστα με σειρά προτεραιότητας. Η ροή εισόδου στη λίστα αναμονής φαίνεται στο Διάγραμμα 8.



Διάγραμμα 8 — *Ροή εισόδου στη λίστα αναμονής.*

4.5.4 Ακύρωση κράτησης και πολιτική ακύρωσης

Κατά την ακύρωση μιας κράτησης, κλειδώνεται πρώτα η εγγραφή του μαθήματος και στη συνέχεια η ίδια η κράτηση, η οποία ελέγχεται ξανά μετά το κλείδωμα. Αν η κράτηση δεν είναι πια ενεργή, για παράδειγμα επειδή την ακύρωσε ένα δεύτερο αίτημα που έφτασε την ίδια στιγμή, η διαδικασία σταματά χωρίς καμία αλλαγή. Έτσι η μετάβαση από Confirmed σε Cancelled γίνεται μόνο μία φορά, και η θέση, η χρέωση της συνδρομής και η προαγωγή από τη λίστα αναμονής δεν επαναλαμβάνονται. Στη συνέχεια ελέγχεται η πολιτική ακυρώσεων του γυμναστηρίου, δηλαδή αν έχει περάσει το ελάχιστο χρονικό περιθώριο πριν από το μάθημα (ο έλεγχος αυτός δεν ισχύει όταν την ακύρωση κάνει το προσωπικό). Εφόσον η ακύρωση επιτρέπεται, η θέση ελευθερώνεται, επιστρέφεται τυχόν χρέωση στη συνδρομή και, αν υπάρχει λίστα αναμονής, μπαίνει αυτόματα ο επόμενος έγκυρος χρήστης. Όλα αυτά γίνονται μέσα στο ίδιο transaction. Η πλήρης ροή φαίνεται στο Διάγραμμα 9.



Διάγραμμα 9 *— Ροή ακύρωσης κράτησης με πολιτική και αυτόματη προαγωγή.*

4.6 Ιδιαίτερα απαιτητικά χαρακτηριστικά

Ακολουθούν τα σημεία της εφαρμογής που απαίτησαν τη μεγαλύτερη προσοχή κατά την υλοποίηση, είτε λόγω της πολυπλοκότητάς τους είτε επειδή αφορούν την ορθότητα και την αξιοπιστία του συστήματος.

4.6.1 Ατομικές κρατήσεις χωρίς overbooking

Η μεγαλύτερη πρόκληση ήταν να διασφαλιστεί ότι δύο ή περισσότεροι χρήστες δεν μπορούν να κλείσουν την ίδια τελευταία θέση ταυτόχρονα (βλ. ροή στην ενότητα 4.5.2). Η λύση χρησιμοποιεί κλείδωμα τύπου pessimistic locking στην εγγραφή του μαθήματος, ώστε οι ταυτόχρονες κρατήσεις να εξυπηρετούνται σειριακά, ως πρόσθετη δικλείδα ασφαλείας υπάρχει και μοναδικός περιορισμός (unique constraint) στη βάση δεδομένων.

Η υλοποίηση φαίνεται στον Κώδικα 2. Όλη η κράτηση εκτελείται μέσα σε ένα transaction, και η πρώτη ενέργεια είναι το κλείδωμα της γραμμής του μαθήματος με SELECT … FOR UPDATE [14], κάθε άλλη ταυτόχρονη κράτηση για το ίδιο μάθημα περιμένει μέχρι να ολοκληρωθεί η τρέχουσα, οπότε ο έλεγχος της χωρητικότητας γίνεται πάντα με ενημερωμένα δεδομένα.

*Κώδικας 2 — Ατομική κράτηση με κλείδωμα γραμμής (C#, backend/src/GymBooking.Api/Services/BookingService.cs, γρ. 23–88, συντομευμένο).*2

```
var strategy = _dbContext.Database.CreateExecutionStrategy();
return await strategy.ExecuteAsync(async () =>
{
    await using var tx = await _dbContext.Database.BeginTransactionAsync();
    var session = await _dbContext.ClassSessions
        .FromSqlInterpolated($@"SELECT * FROM ""ClassSessions""
              WHERE ""Id"" = {classSessionId} AND ""TenantId"" = {tenantId} FOR UPDATE")
        .IgnoreQueryFilters().AsTracking().FirstOrDefaultAsync();
    await LockUserAsync(userId);   // κλείδωμα ανά χρήστη (pg_advisory_xact_lock)
    if (session.BookedCount >= session.Capacity)
        return (BookingOutcome.SessionFull, null);
    session.BookedCount += 1;   // το μοναδικό σημείο αλλαγής του BookedCount
    await _dbContext.SaveChangesAsync();
    await tx.CommitAsync();
    return (BookingOutcome.Success, booking);
});
```

Τρεις λεπτομέρειες έχουν σημασία. Πρώτον, το ερώτημα χρησιμοποιεί IgnoreQueryFilters(), επειδή το φίλτρο του γυμναστηρίου μπαίνει ρητά μέσα στο SQL. Δεύτερον, το transaction εκτελείται μέσα σε execution strategy: ο πάροχος Npgsql έχει ενεργοποιημένη την αυτόματη επανάληψη σε σφάλματα σύνδεσης, και σε αυτή την περίπτωση το Entity Framework Core δεν επιτρέπει χειροκίνητα transactions έξω από το execution strategy. Το πρόβλημα εμφανίστηκε μόνο σε πραγματική PostgreSQL και όχι στις δοκιμές με βάση στη μνήμη, και λύθηκε με την παραπάνω δομή. Τρίτον, ως δεύτερη δικλίδα υπάρχει μερικός μοναδικός δείκτης (partial unique index) στα πεδία (UserId, ClassSessionId) για τις ενεργές κρατήσεις, ώστε η διπλή κράτηση να αποκλείεται και σε επίπεδο βάσης.

Το ίδιο μοτίβο ακολουθεί και η ακύρωση (ενότητα 4.5.4). Αφού κλειδωθεί η εγγραφή του μαθήματος, η κράτηση διαβάζεται ξανά με SELECT … FOR UPDATE και ελέγχεται ότι είναι ακόμα Confirmed. Αν δύο αιτήματα ακύρωσης για την ίδια κράτηση φτάσουν ταυτόχρονα, το δεύτερο περιμένει το πρώτο και στη συνέχεια βρίσκει την κράτηση ήδη ακυρωμένη, οπότε δεν αλλάζει τίποτα. Χωρίς αυτόν τον επανέλεγχο, ο μετρητής θέσεων θα μειωνόταν δύο φορές και θα μπορούσαν να μπουν στο μάθημα περισσότερα άτομα από τη λίστα αναμονής από όσες θέσεις ελευθερώθηκαν. Η συμπεριφορά αυτή ελέγχεται με δύο δοκιμές που στέλνουν 10 ταυτόχρονες ακυρώσεις της ίδιας κράτησης.

Το κλείδωμα του μαθήματος δεν αρκεί για τον κανόνα ότι ένας χρήστης δεν μπορεί να έχει δύο κρατήσεις που επικαλύπτονται χρονικά. Αν ο ίδιος χρήστης ζητήσει την ίδια στιγμή δύο διαφορετικά μαθήματα με επικαλυπτόμενες ώρες, τα δύο αιτήματα κλειδώνουν διαφορετικές εγγραφές, και κανένα δεν βλέπει την κράτηση του άλλου όταν ελέγχει για επικάλυψη. Γι’ αυτό, μετά το κλείδωμα του μαθήματος, παίρνεται και ένα κλείδωμα ανά χρήστη με τη συνάρτηση pg_advisory_xact_lock της PostgreSQL, με κλειδί που προκύπτει από το αναγνωριστικό του χρήστη. Το κλείδωμα αυτό δεν αφορά κάποια γραμμή πίνακα και ελευθερώνεται αυτόματα στο τέλος του transaction. Έτσι, το δεύτερο αίτημα περιμένει το πρώτο, και όταν συνεχίσει βλέπει την κράτηση που μόλις έγινε και απαντά με χρονική επικάλυψη. Το ίδιο κλείδωμα παίρνεται και για κάθε χρήστη που προάγεται από τη λίστα αναμονής. Η σειρά είναι παντού η ίδια, πρώτα το μάθημα και μετά ο χρήστης, ώστε να αποφεύγονται τα deadlocks. Η συμπεριφορά ελέγχεται με δοκιμή όπου ο ίδιος χρήστης ζητά ταυτόχρονα δύο επικαλυπτόμενα μαθήματα σε 20 διαδοχικούς γύρους, και σε κάθε γύρο πετυχαίνει μόνο η μία κράτηση.

4.6.2 Λίστα αναμονής και αυτόματη προαγωγή

Η λίστα αναμονής συνδέεται στενά με την ακύρωση, όπως περιγράφηκε στην ενότητα 4.5.4. Η ιδιαιτερότητα είναι ότι η προαγωγή δεν μπορεί να γίνει τυφλά: ελέγχονται ξανά όλες οι

προϋποθέσεις (ότι ο χρήστης δεν έχει ήδη κράτηση, δεν υπάρχει χρονική επικάλυψη και υπάρχει ενεργή συνδρομή), ώστε η προαγωγή να είναι πάντα έγκυρη.

4.6.3 Πολιτική ακυρώσεων

Το ελάχιστο χρονικό περιθώριο πριν από το μάθημα (ενότητα 4.5.4) δεν είναι σταθερό για όλα τα γυμναστήρια: κάθε tenant ρυθμίζει τη δική του τιμή, ώστε η πολιτική να προσαρμόζεται στις ανάγκες του. Το προσωπικό εξαιρείται από τον περιορισμό, για την κάλυψη έκτακτων περιπτώσεων.

4.7 Ασφάλεια

Η ασφάλεια αντιμετωπίστηκε ως βασική απαίτηση σε όλα τα επίπεδα της εφαρμογής [20], ακολουθούν οι μηχανισμοί που προστατεύουν τη σύνδεση, τα δεδομένα των χρηστών και την επικοινωνία, καθώς και η αντιμετώπιση των προσωπικών δεδομένων.

4.7.1 Προστασία της διαδικασίας σύνδεσης (account lockout, rate limiting)

Για την προστασία από επαναλαμβανόμενες τυχαίες προσπάθειες σύνδεσης, εφαρμόστηκαν δύο μηχανισμοί [21]. Ο πρώτος είναι το προσωρινό κλείδωμα του λογαριασμού μετά από επανειλημμένες αποτυχημένες προσπάθειες σύνδεσης. Ο δεύτερος είναι ο περιορισμός του ρυθμού των αιτημάτων σύνδεσης ανά διεύθυνση, ώστε να αποτρέπονται μαζικές αυτοματοποιημένες προσπάθειες. Και στις δύο περιπτώσεις ο χρήστης λαμβάνει σαφές μήνυμα για τον λόγο της αποτυχίας.

Οι δύο μηχανισμοί ρυθμίζονται κατά την εκκίνηση του API (Κώδικας 3), και ο περιορισμός ρυθμού εφαρμόζεται στο endpoint της σύνδεσης με το attribute [EnableRateLimiting("login")].

Κώδικας 3 — Κλείδωμα λογαριασμού και περιορισμός ρυθμού στη σύνδεση (C#, backend/src/GymBooking.Api/Program.cs, γρ. 56–74 και 108–114, συντομευμένο).

```
builder.Services.AddIdentity<ApplicationUser, ApplicationRole>(options =>
{
    options.Lockout.MaxFailedAccessAttempts = 5;
    options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(15);
});

options.AddPolicy(LoginRateLimitPolicy, httpContext =>
    RateLimitPartition.GetFixedWindowLimiter(
        httpContext.Connection.RemoteIpAddress?.ToString() ?? "unknown",
        _ => new FixedWindowRateLimiterOptions
        {
            PermitLimit = 10, Window = TimeSpan.FromMinutes(1), QueueLimit = 0,
        }));
```

4.7.2 Αποθήκευση κωδικών και tokens προσκλήσεων (hashing)

Οι κωδικοί των χρηστών δεν αποθηκεύονται ποτέ σε καθαρή μορφή, αποθηκεύονται μόνο σε hash μέσω του ASP.NET Core Identity. Με τον ίδιο τρόπο, οι σύνδεσμοι των προσκλήσεων δεν αποθηκεύονται αυτούσιοι, αλλά μόνο η κρυπτογραφική τους σύνοψη, έτσι, ακόμη και αν κάποιος αποκτούσε πρόσβαση στη βάση, δεν θα μπορούσε να ανακτήσει τα αρχικά στοιχεία.

4.7.3 Απομόνωση δεδομένων ανά tenant

Ο διαχωρισμός των δεδομένων κάθε γυμναστηρίου, όπως περιγράφηκε στην ενότητα 4.2, αποτελεί ταυτόχρονα και μέτρο ασφάλειας: επειδή το κριτήριο του μισθωτή προέρχεται πάντα από το token της σύνδεσης και όχι από τον ίδιο τον χρήστη, δεν υπάρχει τεχνικός τρόπος να δει ή να τροποποιήσει κανείς δεδομένα άλλου γυμναστηρίου.

4.7.4 CORS και ασφάλεια επικοινωνίας client–API

Στην τοπική εκτέλεση η επικοινωνία γίνεται μέσω HTTP, ενώ σε εγκατάσταση παραγωγής πρέπει να γίνεται αποκλειστικά μέσω HTTPS. Στον διακομιστή ρυθμίστηκαν οι κανόνες CORS, ώστε να

δέχεται αιτήματα μόνο από τις γνωστές διευθύνσεις των δύο εφαρμογών. Σε συνδυασμό με τον έλεγχο του token σε κάθε αίτημα, διασφαλίζεται ότι μόνο εξουσιοδοτημένοι χρήστες και από εγκεκριμένες προελεύσεις έχουν πρόσβαση στα δεδομένα.

4.7.5 Ιδιωτικότητα και προσωπικά δεδομένα (GDPR)

Η εφαρμογή αποθηκεύει περιορισμένα προσωπικά δεδομένα, όσα είναι απαραίτητα για τη λειτουργία της (όνομα, email και στοιχεία κρατήσεων/συνδρομών). Ακολουθείται η αρχή της ελαχιστοποίησης των δεδομένων, οι κωδικοί φυλάσσονται μόνο σε κρυπτογραφημένη μορφή και η διαγραφή γίνεται λογικά (soft-delete), διατηρώντας τη συνέπεια των ιστορικών εγγραφών. Σε ένα πραγματικό, παραγωγικό περιβάλλον θα προστίθεντο επιπλέον οι διαδικασίες συγκατάθεσης και διαγραφής δεδομένων που προβλέπει ο Κανονισμός (GDPR).

4.7.6 Όρια εμπιστοσύνης (trust boundaries)

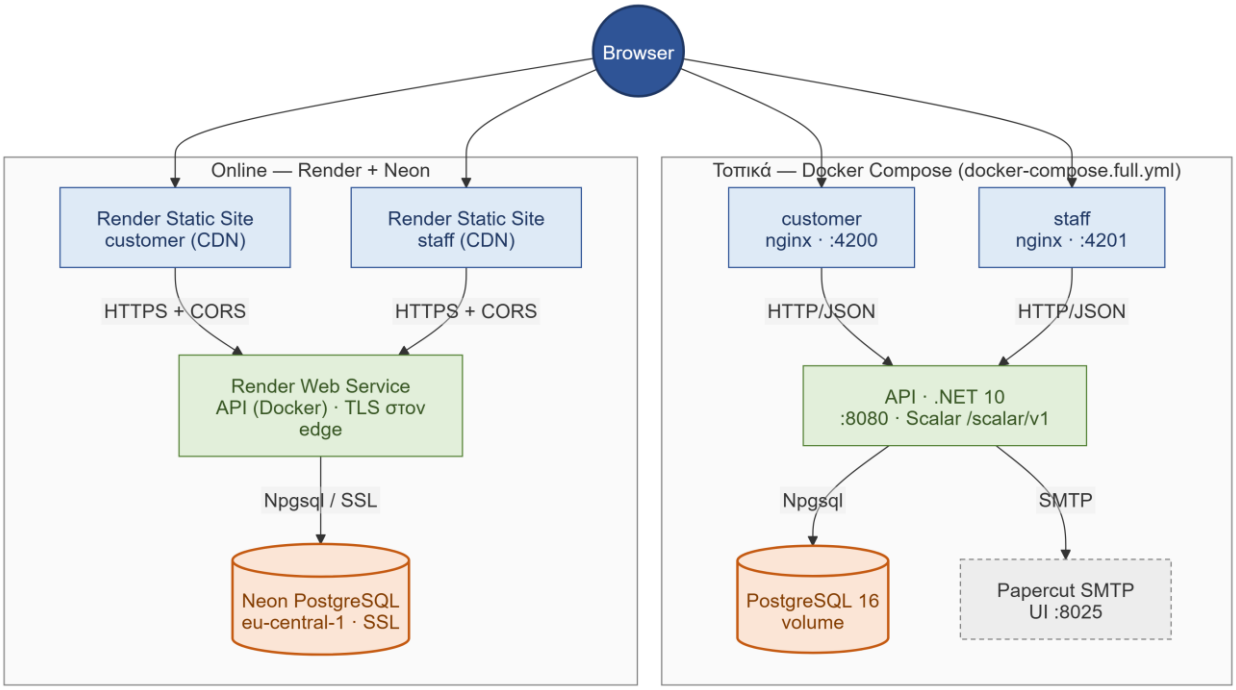
Το σύστημα έχει δύο βασικά όρια εμπιστοσύνης:

- **Browser ↔ API:** ό,τι έρχεται από τον browser θεωρείται μη αξιόπιστο. Κάθε αίτημα ελέγχεται για έγκυρο JWT (υπογραφή, εκδότης, λήξη), για τον ρόλο του χρήστη και για την ιδιοκτησία του πόρου, για παράδειγμα, ένα μέλος μπορεί να ακυρώσει μόνο τις δικές του κρατήσεις. Το αναγνωριστικό του γυμναστηρίου δεν γίνεται ποτέ δεκτό από το σώμα του αιτήματος.
- **API ↔ βάση:** η πρόσβαση γίνεται μόνο από το API, με παραμετροποιημένα ερωτήματα (και στο FromSqlInterpolated), ώστε να αποκλείεται το SQL injection. Τα στοιχεία σύνδεσης της βάσης και το κλειδί υπογραφής του JWT δίνονται ως μεταβλητές περιβάλλοντος και δεν υπάρχουν στον κώδικα.

4.8 Εκτέλεση με Docker (deployment)

Για την τοπική εκτέλεση, ολόκληρο το σύστημα πακετάρεται με το Docker και ξεκινά με μία εντολή (αναλυτικές οδηγίες δίνονται στο κεφάλαιο 6). Κατά την εκκίνηση του διακομιστή εφαρμόζονται αυτόματα οι αλλαγές του σχήματος (migrations) και δημιουργούνται τα αρχικά δεδομένα, ώστε η εφαρμογή να είναι άμεσα λειτουργική.

Η τοπολογία της εκτέλεσης με Docker φαίνεται στο Διάγραμμα 10, ενώ ο Πίνακας 11 τη συγκρίνει με την εκτέλεση για ανάπτυξη (ενότητα 6.6).



Διάγραμμα 10 — *Deployment: τοπική εκτέλεση ολόκληρου του συστήματος με Docker Compose.*

Πίνακας 11 — *Διαφορές ανάμεσα στην εκτέλεση για ανάπτυξη και την εκτέλεση με Docker.*

	Ανάπτυξη (ενότητα 6.6)	**Docker (demo)**
Βάση δεδομένων	PostgreSQL σε Docker (docker/docker-compose.infra.yml)	Container postgres:16
Email	Papercut	Papercut
Τεκμηρίωση API (Scalar)	Ναι	Ναι
CORS	Ανοιχτό για localhost	localhost:4200 και :4201
HTTPS	Ανακατεύθυνση σε HTTPS	HTTP (τοπικά)

Οι ρυθμίσεις δίνονται σε αρχεία appsettings ανά περιβάλλον και υπερισχύουν από μεταβλητές περιβάλλοντος, όπως ConnectionStrings__Default, Jwt__SigningKey, Cors__AllowedOrigins__0 και Seed__AdminPassword, τα μυστικά δεν αποθηκεύονται στο αποθετήριο. Το API καταγράφει δομημένα logs με το Serilog, σε κονσόλα και σε ημερήσιο αρχείο με διατήρηση 14 ημερών, με ένα log για κάθε HTTP αίτημα. Για την παρακολούθηση της λειτουργίας υπάρχει το endpoint /health, το οποίο ελέγχει και τη σύνδεση με τη βάση δεδομένων.

4.9 Αρχιτεκτονικές αποφάσεις

Οι σημαντικότερες αρχιτεκτονικές αποφάσεις καταγράφονται στους Πίνακες 12–15. Για καθεμία δίνονται το πρόβλημα, οι εναλλακτικές που εξετάστηκαν, η τελική επιλογή, η αιτιολόγηση, οι συνέπειες και η σύνδεση με τις απαιτήσεις και τις δοκιμές.

Πίνακας 12 — *Αρχιτεκτονική απόφαση ΑΑ-1: Μοντέλο multi-tenancy.*

ΑΑ-1	**Μοντέλο multi-tenancy**
Πρόβλημα	Πολλά γυμναστήρια πρέπει να εξυπηρετούνται από την ίδια εγκατάσταση με απομονωμένα δεδομένα.
Εναλλακτικές	(α) ξεχωριστή βάση ανά γυμναστήριο, (β) ξεχωριστό schema ανά γυμναστήριο, (γ) κοινή βάση με στήλη TenantId.
Απόφαση	(γ) Κοινή βάση με TenantId και καθολικά φίλτρα του EF Core.
Αιτιολόγηση	Απλούστερη λειτουργία και migrations, χαμηλό κόστος φιλοξενίας, επαρκής για μικρά και μεσαία γυμναστήρια.

AA-1	Μοντέλο multi-tenancy
Συνέπειες	Η απομόνωση εξαρτάται από τον κώδικα, γι' αυτό χρειάζονται καθολικά φίλτρα και ειδικές δοκιμές, ένα πολύ φορτωμένο γυμναστήριο επηρεάζει και τα υπόλοιπα.
Σύνδεση	NFR-03, EE3 · TenantIsolationTests, TenantIsolationHttpTests

Πίνακας 13 *— Αρχιτεκτονική απόφαση AA-2: Έλεγχος ταυτοχρονισμού στις κρατήσεις.*

AA-2	Έλεγχος ταυτοχρονισμού στις κρατήσεις
Πρόβλημα	Ταυτόχρονες κρατήσεις στο ίδιο μάθημα μπορούν να ξεπεράσουν τη χωρητικότητα, και ταυτόχρονες κρατήσεις του ίδιου χρήστη σε διαφορετικά μαθήματα μπορούν να παρακάμψουν τον έλεγχο χρονικής επικάλυψης.
Εναλλακτικές	(α) καμία προστασία, (β) optimistic concurrency (row version και επανάληψη), (γ) επίπεδο απομόνωσης serializable, (δ) pessimistic locking με SELECT … FOR UPDATE.
Απόφαση	(δ) Κλείδωμα της γραμμής του μαθήματος μέσα σε transaction και στη συνέχεια κλείδωμα ανά χρήστη (pg_advisory_xact_lock), με σταθερή σειρά: πρώτα το μάθημα, μετά ο χρήστης.
Αιτιολόγηση	Η ζήτηση συγκεντρώνεται στο ίδιο μάθημα, όπου η optimistic προσέγγιση θα οδηγούσε σε πολλές αποτυχίες και επαναλήψεις, το κλείδωμα είναι απλό.
Συνέπειες	Οι κρατήσεις για το ίδιο μάθημα εκτελούνται σειριακά, με πιθανή αύξηση του χρόνου απόκρισης υπό φορτίο (μετράται στο EE2).
Σύνδεση	FR-05, NFR-01, NFR-02, EE1, EE2 · BookingTests

Πίνακας 14 *— Αρχιτεκτονική απόφαση AA-3: Μηχανισμός ταυτοποίησης.*

AA-3	Μηχανισμός ταυτοποίησης
Πρόβλημα	Ασφαλής σύνδεση για δύο SPA και ένα ξεχωριστό API.
Εναλλακτικές	(α) cookies και συνεδρία στον διακομιστή, (β) μόνο access JWT με διάρκεια 2 ωρών, (γ) JWT με refresh token rotation.
Απόφαση	(β), σε φάσεις: στη φάση 1 μόνο access token, με πρόβλεψη για refresh tokens στη φάση 2.
Αιτιολόγηση	Απλή υλοποίηση που ταιριάζει σε SPA με ξεχωριστό API, η φάση 2 μπορεί να προστεθεί χωρίς να ξαναγραφτεί η υπάρχουσα λογική.
Συνέπειες	Το token αποθηκεύεται στο localStorage και είναι εκτεθειμένο σε επίθεση XSS (το Angular κάνει αυτόματο escaping), το token δεν ανακαλείται πριν από τη λήξη του.
Σύνδεση	FR-01, NFR-05 · TokenServiceTests · ενότητα 7.4

Πίνακας 15 *— Αρχιτεκτονική απόφαση AA-4: Οργάνωση του frontend.*

AA-4	Οργάνωση του frontend
Πρόβλημα	Δύο κοινά (μέλη και προσωπικό) με διαφορετικές ανάγκες και συσκευές.
Εναλλακτικές	(α) μία εφαρμογή με ρόλους, (β) δύο ανεξάρτητα αποθετήρια, (γ) Nx monorepo με δύο εφαρμογές και κοινές βιβλιοθήκες.
Απόφαση	(γ) Nx monorepo με τις εφαρμογές customer και staff και τις βιβλιοθήκες models, data-access, auth και ui.
Αιτιολόγηση	Επιτρέπει σχεδίαση mobile-first για τα μέλη και desktop-first για το προσωπικό, χωρίς διπλό κώδικα.
Συνέπειες	Περισσότερη αρχική ρύθμιση (Nx), τα όρια ανάμεσα στα modules επιβάλλονται με tags και ελέγχονται από το lint.
Σύνδεση	NFR-08, NFR-11

4.10 Γνωστά ζητήματα

Κατά την παράδοση, είναι γνωστά τα εξής ζητήματα, κανένα από τα οποία δεν επηρεάζει την ορθότητα των κρατήσεων:

- Δεν υπάρχει ενεργή διαδικασία συνεχούς ενσωμάτωσης (CI), το workflow υπάρχει στο αποθετήριο αλλά δεν έχει ενεργοποιηθεί.
- Το token αποθηκεύεται στο localStorage (βλ. AA-3).
- Δεν υπάρχει ενημέρωση σε πραγματικό χρόνο, η πληρότητα των μαθημάτων ανανεώνεται με την επαναφόρτωση του προγράμματος.

5. Αξιολόγηση

Στο κεφάλαιο αυτό αξιολογείται αν η εφαρμογή ικανοποιεί τις απαιτήσεις του κεφαλαίου 3 και αν απαντώνται τα ερευνητικά ερωτήματα της ενότητας 1.4. Η αξιολόγηση γίνεται σε δύο επίπεδα: λειτουργικός έλεγχος με αυτοματοποιημένες δοκιμές (5.1) και ελεγχόμενα πειράματα για την ορθότητα υπό ταυτοχρονισμό, τον χρόνο απόκρισης, την απομόνωση των δεδομένων και την προσβασιμότητα (5.2–5.3). Το κεφάλαιο κλείνει με τις απειλές εγκυρότητας (5.4). Ο κώδικας των πειραμάτων και τα αρχεία των αποτελεσμάτων βρίσκονται στο αποθετήριο (φάκελος evaluation), ώστε οι μετρήσεις να μπορούν να επαναληφθούν.

5.1 Λειτουργικός έλεγχος (unit tests, QA)

Για τον έλεγχο της ορθότητας αναπτύχθηκε ένα σύνολο αυτοματοποιημένων δοκιμών (unit tests) στην πλευρά του διακομιστή, με έμφαση στην κρίσιμη λογική των κρατήσεων: την τήρηση της χωρητικότητας, τη σωστή συμπεριφορά υπό ταυτόχρονη πρόσβαση, την ακύρωση με την πολιτική του γυμναστηρίου και την αυτόματη προαγωγή από τη λίστα αναμονής. Πέρα από τις αυτοματοποιημένες δοκιμές, έγινε και χειροκίνητος έλεγχος (QA) της διεπαφής, ιδίως ως προς τη σωστή εμφάνιση σε κινητό και σε υπολογιστή.

Συνολικά εκτελούνται 81 αυτοματοποιημένες δοκιμές στο backend (xUnit) και 26 στο frontend (Vitest), και όλες περνούν επιτυχώς. Οι δοκιμές του backend που αφορούν τις κρατήσεις, τη λίστα αναμονής και την απομόνωση σε επίπεδο HTTP εκτελούνται σε πραγματική PostgreSQL (Testcontainers) [38], ενώ οι υπόλοιπες σε βάση στη μνήμη. Ο Πίνακας 16 αντιστοιχίζει τις δοκιμές στις απαιτήσεις του κεφαλαίου 3.

Πίνακας 16 — *Αυτοματοποιημένες δοκιμές ανά περιοχή λειτουργικότητας.*

Περιοχή	Δοκιμές	Απαιτήσεις	Αποτέλεσμα
Ταυτοποίηση και token	8	FR-01, NFR-04, NFR-05	8/8 ✓
Προσκλήσεις και εγγραφή	7	FR-02, FR-03	7/7 ✓
Πρόγραμμα και εκπαιδευτές	3	FR-04	3/3 ✓
Κρατήσεις (και από το προσωπικό)	14	FR-05, FR-12, NFR-01	14/14 ✓
Συνδρομές και πλάνα	6	FR-06, FR-14	6/6 ✓
Πολιτική ακυρώσεων	3	FR-07	3/3 ✓
Λίστα αναμονής και προαγωγή	11	FR-08, FR-09	11/11 ✓
Τύποι μαθημάτων και μαθήματα	8	FR-10, FR-11, NFR-05	8/8 ✓
Χρήστες και ρυθμίσεις	6	FR-13, FR-15	6/6 ✓
Απομόνωση tenant	9	NFR-03	9/9 ✓
Αρχικοποίηση, email, υποδομή	6	NFR-10	6/6 ✓
Σύνολο backend (xUnit)	**81**		**81/81 ✓**
Frontend (Vitest): guards, interceptor, auth, κοινά components	26	FR-01, NFR-05	26/26 ✓

5.2 Πρωτόκολλο αξιολόγησης

Περιβάλλον. Όλες οι μετρήσεις έγιναν στο ίδιο μηχάνημα: Intel Core Ultra 9 285H (16 λογικοί πυρήνες), 32 GB RAM, Windows 11, .NET 10.0.12 και PostgreSQL 16 σε Docker Desktop (Testcontainers) με max_connections=300. Το API εκτελείται in-process (TestServer), με τις ίδιες ρυθμίσεις βάσης όπως στην παραγωγή (αυτόματη επανάληψη σε παροδικά σφάλματα).

ΕΕ1 – υπερκρατήσεις. Υπόθεση: με κλείδωμα, οι επιβεβαιωμένες κρατήσεις δεν ξεπερνούν ποτέ τη χωρητικότητα. Ανεξάρτητες μεταβλητές είναι η υλοποίηση (με κλείδωμα ή baseline χωρίς κλείδωμα) και ο αριθμός N ταυτόχρονων αιτημάτων (10, 50, 100) για μάθημα χωρητικότητας C = 5. Το baseline είναι μια υλοποίηση «διάβασε – έλεγξε – γράψε» χωρίς κλείδωμα και χωρίς ρητό

transaction, που υπάρχει μόνο στον κώδικα του πειράματος. Όλα τα αιτήματα κάθε γύρου ξεκινούν ταυτόχρονα από κοινή «πύλη» εκκίνησης, με 10 επαναλήψεις ανά συνδυασμό (60 γύροι συνολικά). Μετρικές: επιβεβαιωμένες κρατήσεις στη βάση και υπερκρατήσεις (πλεόνασμα πάνω από το C).

ΕΕ2 – χρόνος απόκρισης. Μετρήθηκε ο χρόνος κάθε HTTP αιτήματος με 1, 10, 25 και 50 ταυτόχρονους χρήστες, σε τρία σενάρια: κράτηση στο ίδιο μάθημα (η χειρότερη περίπτωση για το κλείδωμα), κράτηση σε διαφορετικά μαθήματα (χωρίς ανταγωνισμό για το κλείδωμα) και προβολή του εβδομαδιαίου προγράμματος με 20 μαθήματα. Έγιναν 5 επαναλήψεις ανά συνδυασμό, μετά από έναν γύρο «ζεστάματος» που δεν καταγράφηκε. Μετρικές: p50, p95 και η τυπική απόκλιση του p95 ανάμεσα στις επαναλήψεις, τα όρια είναι αυτά του NFR-02, που ορίστηκαν πριν από τη μέτρηση.

ΕΕ3 – απομόνωση. Εκτός από τις 6 δοκιμές σε επίπεδο δεδομένων, 3 δοκιμές σε επίπεδο HTTP πάνω σε πραγματική PostgreSQL επιχειρούν, ως χρήστης του γυμναστηρίου Α, να διαβάσουν και να κλείσουν μάθημα του γυμναστηρίου Β με το αναγνωριστικό του, καθώς και να δουν το πρόγραμμά του.

NFR-09 – προσβασιμότητα. Χρησιμοποιήθηκε το Lighthouse 12 (κατηγορία Accessibility) [39] σε 8 οθόνες του Docker stack: οι οθόνες της εφαρμογής μελών σε προφίλ κινητού και της εφαρμογής προσωπικού σε προφίλ desktop, με σύνδεση μέσω demo λογαριασμών για τις οθόνες που το απαιτούν.

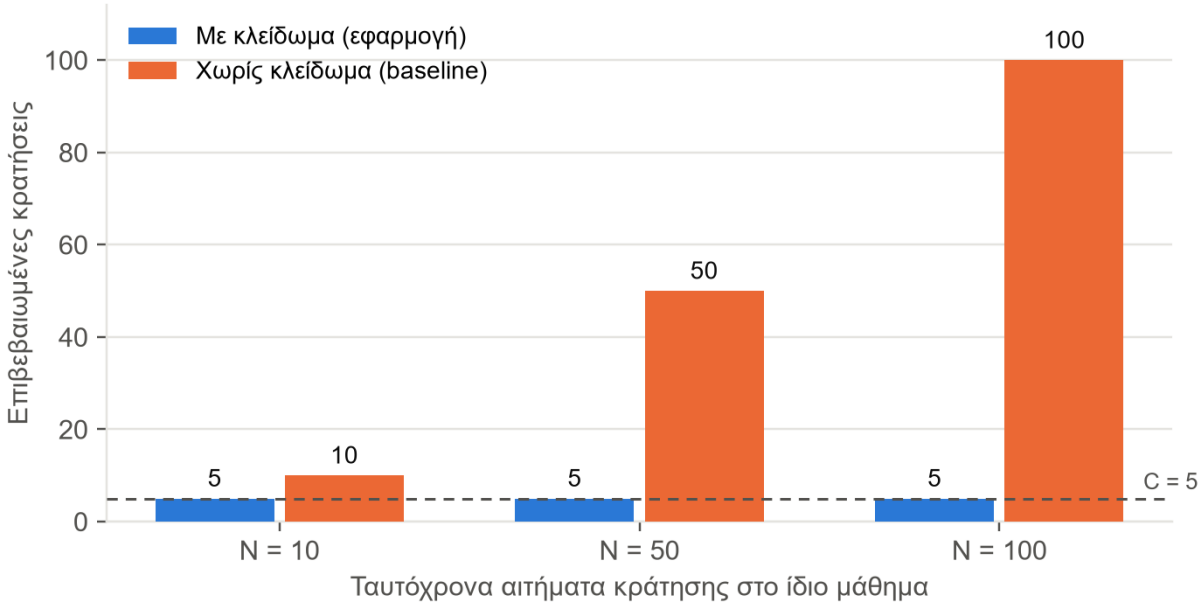
5.3 Αποτελέσματα

5.3.1 ΕΕ1: υπερκρατήσεις με και χωρίς κλείδωμα

Τα αποτελέσματα του πρώτου πειράματος συνοψίζονται στον Πίνακα 17 και στο Διάγραμμα 11.

Πίνακας 17 — ΕΕ1: αποτελέσματα ταυτόχρονων κρατήσεων σε μάθημα χωρητικότητας C = 5 (10 επαναλήψεις ανά γραμμή).

Υλοποίηση	N	Επιβεβ. κρατήσεις	Υπερ-κρατήσεις	Γύροι με overbooking	Μετρητής θέσεων	Διάρκεια γύρου (ms)
Με κλείδωμα	10	5	0	0/10	5	77 (±8)
Με κλείδωμα	50	5	0	0/10	5	263 (±23)
Με κλείδωμα	100	5	0	0/10	5	438 (±63)
Χωρίς κλείδωμα	10	10	5	10/10	1	14 (±5)
Χωρίς κλείδωμα	50	50	45	10/10	1	63 (±9)
Χωρίς κλείδωμα	100	100	95	10/10	1	120 (±12)



Διάγραμμα 11 *— ΕΕ1: επιβεβαιωμένες κρατήσεις με και χωρίς κλείδωμα (C = 5, μέσος όρος 10 επαναλήψεων).*

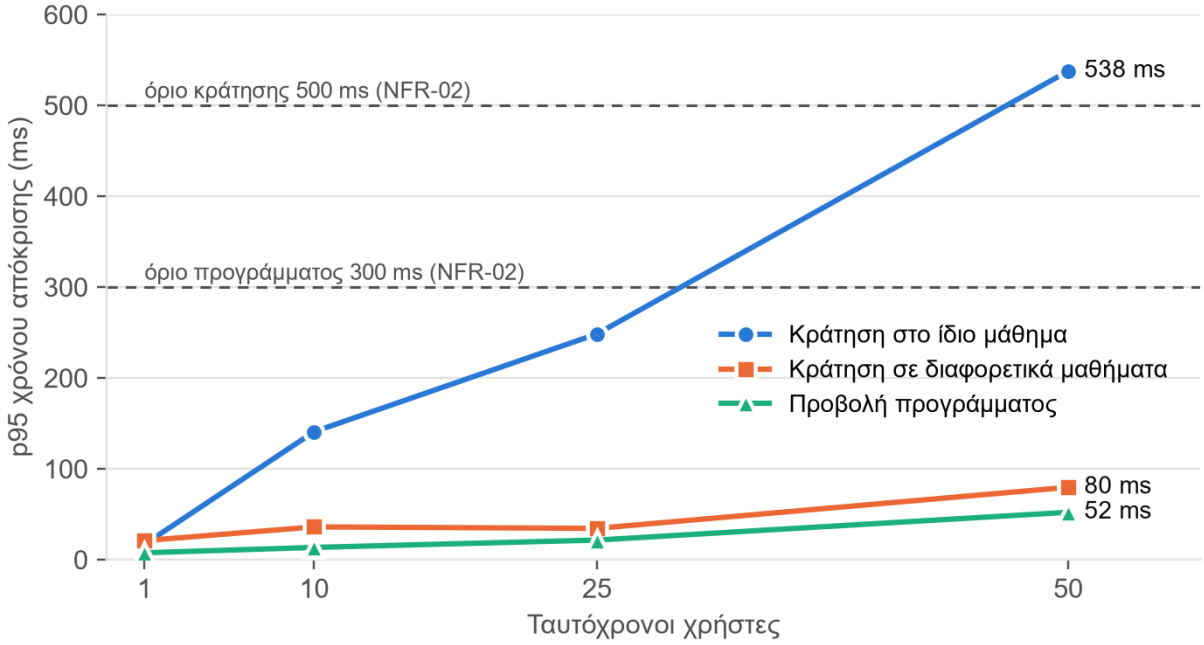
Με κλείδωμα, σε κανέναν από τους 30 γύρους δεν έγινε overbooking, ακόμη και με 100 ταυτόχρονα αιτήματα για 5 θέσεις. Χωρίς κλείδωμα, όλα τα αιτήματα έγιναν δεκτά σε κάθε γύρο, ενώ ο μετρητής θέσεων του μαθήματος έμεινε στο 1 (lost update): το σύστημα «πίστευε» ότι υπήρχαν ακόμη ελεύθερες θέσεις, ενώ είχαν ήδη γίνει δεκάδες κρατήσεις. Το κόστος του κλειδώματος είναι ότι κάθε γύρος διαρκεί περισσότερο (438 έναντι 120 ms για N = 100), επειδή οι κρατήσεις εκτελούνται σειριακά. Απάντηση στο ΕΕ1: η προσέγγιση αποκλείει πλήρως το overbooking στις συνθήκες του πειράματος, ενώ η απλή υλοποίηση αποτυγχάνει συστηματικά.

5.3.2 ΕΕ2: χρόνος απόκρισης υπό ταυτοχρονισμό

Ο Πίνακας 18 και το Διάγραμμα 12 δείχνουν το p95 του χρόνου απόκρισης για κάθε σενάριο.

Πίνακας 18 *— ΕΕ2: p95 του χρόνου απόκρισης σε ms (σε παρένθεση η τυπική απόκλιση ανάμεσα στις 5 επαναλήψεις).*

Σενάριο	1 χρήστης	10 χρήστες	25 χρήστες	50 χρήστες	Όριο NFR-02
Κράτηση, ίδιο μάθημα	17 (±2)	141 (±27)	249 (±19)	538 (±18)	500 ms ◐
Κράτηση, διαφορετικά μαθήματα	21 (±3)	36 (±8)	35 (±4)	80 (±9)	500 ms ✓
Προβολή προγράμματος	8 (±1)	14 (±2)	22 (±3)	52 (±6)	300 ms ✓



Διάγραμμα 12 — *EE2: p95 του χρόνου απόκρισης ανά σενάριο και αριθμό ταυτόχρονων χρηστών.*

Η κράτηση σε διαφορετικά μαθήματα και η προβολή του προγράμματος ικανοποιούν άνετα το NFR-02. Αντίθετα, η κράτηση στο ίδιο μάθημα με 50 ταυτόχρονους χρήστες βρίσκεται στο όριο: στην εκτέλεση του Πίνακα 18 το p95 ήταν 538 ms (±18 ms), δηλαδή λίγο πάνω από τα 500 ms. Επειδή οι χρόνοι αυτοί επηρεάζονται από τον φόρτο του μηχανήματος, το πείραμα εκτελέστηκε 5 φορές και στον πίνακα δίνεται η διάμεση εκτέλεση. Στις 5 εκτελέσεις το p95 κυμάνθηκε από 525 έως 723 ms. Επομένως το NFR-02 ικανοποιείται μόνο οριακά (◐) όταν 50 χρήστες κλείνουν ταυτόχρονα το ίδιο μάθημα, ενώ με 25 χρήστες το p95 (249 ms) είναι αρκετά κάτω από το όριο. Σημαντικό εύρημα είναι η μορφή των καμπυλών: όταν οι κρατήσεις αφορούν διαφορετικά μαθήματα, ο χρόνος μένει σχεδόν σταθερός (έως 80 ms), ενώ όταν όλοι στοχεύουν το ίδιο μάθημα αυξάνεται περίπου γραμμικά με τον αριθμό των χρηστών, κατά περίπου 10 ms για κάθε επιπλέον ταυτόχρονο αίτημα, επειδή κάθε κράτηση περιμένει την προηγούμενη να απελευθερώσει το κλείδωμα. Οι μετρήσεις έγιναν μετά την προσθήκη του κλειδώματος ανά χρήστη (ενότητα 4.6.1), που προσθέτει μία ακόμη εντολή στη βάση σε κάθε κράτηση. Σε συγκριτικές εκτελέσεις με την προηγούμενη έκδοση, η διαφορά ήταν μέσα στη διακύμανση των μετρήσεων. Απάντηση στο EE2: το κλείδωμα επιβαρύνει τον χρόνο απόκρισης μόνο όταν υπάρχει ανταγωνισμός για το ίδιο μάθημα. Η επιβάρυνση είναι αποδεκτή για τα μεγέθη ενός γυμναστηρίου, όπου ακόμη και 25 ταυτόχρονες κρατήσεις στο ίδιο μάθημα είναι σπάνιες, αλλά με 50 φτάνει στο όριο του NFR-02. Για μεγαλύτερο ταυτοχρονισμό θα χρειαζόταν διαφορετική προσέγγιση (ενότητα 7.4).

5.3.3 EE3: απομόνωση των δεδομένων

Και οι 9 δοκιμές απομόνωσης πέτυχαν. Χρήστης άλλου γυμναστηρίου λαμβάνει απάντηση 404 όταν ζητά ή επιχειρεί να κλείσει μάθημα με γνωστό αναγνωριστικό, ο μετρητής θέσεων του μαθήματος παραμένει 0, και το εβδομαδιαίο πρόγραμμα περιέχει μόνο τα μαθήματα του δικού του γυμναστηρίου. Απάντηση στο EE3: στα σενάρια που ελέγχθηκαν, ο διαχωρισμός δεν επιτρέπει πρόσβαση σε δεδομένα άλλου γυμναστηρίου, ούτε με απευθείας χρήση των αναγνωριστικών τους.

5.3.4 Προσβασιμότητα

Πίνακας 19 — *Βαθμολογίες Lighthouse Accessibility ανά οθόνη.*

Εφαρμογή	Σύνδεση	Αρχική	Πρόγραμμα / Ώρες	Ιστορικό / Χρήστες
Πελατών (κινητό)	100	100	100	100
Προσωπικού (desktop)	94	95	95	95

Όλες οι οθόνες ξεπερνούν το όριο του 90 (Πίνακας 19). Στην εφαρμογή προσωπικού εντοπίστηκε ένα πρόβλημα αντίθεσης χρωμάτων: το κείμενο του μωβ κουμπιού έχει αντίθεση 4,35:1 και το σήμα «Προσωπικό» 4,11:1, ενώ το WCAG AA απαιτεί 4,5:1 [40]. Το εύρημα καταγράφεται ως περιορισμός, με εύκολη διόρθωση (σκουρότερος τόνος του χρώματος έμφασης).

5.3.5 Σύνοψη

Ο Πίνακας 20 συνοψίζει την επαλήθευση των μη λειτουργικών απαιτήσεων που ελέγχθηκαν με δοκιμές ή μετρήσεις. Για το όριο ρυθμού του NFR-04, που είναι απενεργοποιημένο στο περιβάλλον δοκιμών, έγινε χειροκίνητος έλεγχος στην τοπική εκτέλεση: 12 διαδοχικά αιτήματα σύνδεσης με λανθασμένα στοιχεία έδωσαν 10 απαντήσεις 401 και στη συνέχεια 429 (Too Many Requests) με επικεφαλίδα Retry-After: 60.

Πίνακας 20 — Σύνοψη επαλήθευσης των μη λειτουργικών απαιτήσεων.

Κωδ.	Κριτήριο	Αποτέλεσμα	
NFR-01	0 υπερκρατήσεις όταν N > C	0 από 30 γύρους με overbooking	✓
NFR-02	p95 κράτησης ≤ 500 ms και προγράμματος ≤ 300 ms με 50 χρήστες	Ίδιο μάθημα 538 ms (διάμεση από 5 εκτελέσεις, εύρος 525–723 ms), πρόγραμμα 52 ms	◐
NFR-03	0 εγγραφές άλλου γυμναστηρίου	9/9 δοκιμές απομόνωσης	✓
NFR-04	Κλείδωμα μετά από 5 αποτυχίες, 10 αιτήματα/λεπτό ανά IP	Δοκιμή κλειδώματος, χειροκίνητα: 10 × 401 και στη συνέχεια 429 με Retry-After: 60	✓
NFR-05	403 χωρίς ρόλο, λήξη token σε 2 ώρες	Δοκιμές 403 και λήξης token	✓
NFR-09	Lighthouse Accessibility ≥ 90	Βαθμολογίες 94–100 (εύρημα αντίθεσης στην εφαρμογή προσωπικού)	✓
NFR-11	Όλες οι αυτοματοποιημένες δοκιμές περνούν	81/81 backend, 26/26 frontend	✓

5.4 Απειλές εγκυρότητας

•Εσωτερική εγκυρότητα: το API εκτελείται in-process, οπότε οι χρόνοι δεν περιλαμβάνουν δίκτυο, TLS ή reverse proxy, στο περιβάλλον δοκιμών είναι ενεργή η καταγραφή των ερωτημάτων της βάσης, που προσθέτει μικρή επιβάρυνση, κάθε συνδυασμός του EE2 μετρήθηκε με 5 επαναλήψεις, οπότε τα p95 έχουν διακύμανση (έως ±27 ms μέσα σε μία εκτέλεση). Ανάμεσα σε διαφορετικές εκτελέσεις η διακύμανση είναι μεγαλύτερη: το p95 της κράτησης στο ίδιο μάθημα με 50 χρήστες κυμάνθηκε από 525 έως 723 ms, ανάλογα με τον φόρτο του μηχανήματος. Γι' αυτό το πείραμα εκτελέστηκε 5 φορές και αναφέρεται η διάμεση εκτέλεση.

- **Εξωτερική εγκυρότητα:** οι μετρήσεις έγιναν σε ισχυρό φορητό υπολογιστή και όχι σε εξυπηρετητή παραγωγής, όπου οι χρόνοι θα εξαρτώνταν από το υλικό και το δίκτυο, τα δεδομένα είναι συνθετικά και το φορτίο είναι «ριπή» ταυτόχρονων αιτημάτων και όχι πραγματική κίνηση.
- **Εγκυρότητα κατασκευής:** το baseline είναι σκόπιμα απλό, μια optimistic υλοποίηση θα ήταν δικαιότερη σύγκριση για τον χρόνο απόκρισης, αλλά όχι για την ορθότητα. Το Lighthouse ελέγχει αυτόματα μόνο ένα μέρος των κριτηρίων προσβασιμότητας και δεν αντικαθιστά τον έλεγχο με πραγματικούς χρήστες, ενώ δεν έγινε δοκιμή χρηστικότητας με χρήστες.

6. Οδηγός εγκατάστασης και χρήσης

Στο κεφάλαιο αυτό δίνονται οι οδηγίες για την εγκατάσταση και την εκτέλεση της εφαρμογής, καθώς και μια περιγραφή της χρήσης της μέσα από στιγμιότυπα οθόνης.

Ολόκληρη η εφαρμογή (βάση δεδομένων, API και οι δύο web εφαρμογές, καθώς και ο τοπικός εξυπηρετητής email) τρέχει με μία εντολή, χωρίς να απαιτείται εγκατάσταση .NET ή Node.js — αρκεί το Docker Desktop. Την πρώτη φορά, η βάση γεμίζει αυτόματα με demo δεδομένα και δημιουργείται ο αρχικός λογαριασμός διαχειριστή (auto-migrate και seeding εκτελούνται από το API κατά την εκκίνηση). Δεν χρειάζεται καμία χειροκίνητη δημιουργία χρήστη για να ξεκινήσει η χρήση.

6.1 Προαπαιτούμενα

Η εφαρμογή εκτελείται σε Windows, macOS ή Linux, σε κάθε υπολογιστή όπου λειτουργεί το Docker Desktop, και χρησιμοποιείται από οποιονδήποτε σύγχρονο browser (Chrome, Edge, Firefox, Safari). Απαιτούνται:

- **Για την εκτέλεση με Docker (συνιστάται):** Docker Desktop σε λειτουργία (https://www.docker.com/products/docker-desktop/). Δεν χρειάζεται εγκατάσταση .NET, Node.js ή PostgreSQL.
- **Για την εκτέλεση χωρίς Docker (ανάπτυξη):** .NET SDK 10, Node.js 20 με npm, και Docker Desktop μόνο για τη βάση δεδομένων και τον τοπικό εξυπηρετητή email (ενότητα 6.6).
- **Λήψη του κώδικα:** από το αποθετήριο στο GitHub (git clone) ή από το αρχείο της παράδοσης.

6.2 Εκτέλεση με Docker

Από τον ριζικό φάκελο του project, εκτελείται η ακόλουθη εντολή:

```
docker compose -f docker/docker-compose.full-stack.yml up --build
```

Την πρώτη φορά, το χτίσιμο (build) διαρκεί μερικά λεπτά, καθώς κατεβαίνουν τα images και χτίζονται οι εφαρμογές. Μόλις στα logs εμφανιστεί ότι το API ξεκίνησε, η εφαρμογή είναι έτοιμη.

6.3 Διευθύνσεις και έτοιμοι λογαριασμοί

Οι λογαριασμοί του Πίνακα 21 δημιουργούνται αυτόματα στην τοπική εκτέλεση και είναι έτοιμοι για άμεση σύνδεση. Πρόκειται για δοκιμαστικούς λογαριασμούς που υπάρχουν μόνο στην τοπική βάση δεδομένων.

Πίνακας 21 — Έτοιμοι λογαριασμοί της τοπικής εκτέλεσης (Docker).

Ρόλος	Email	Κωδικός
Διαχειριστής (Admin)	admin@demo.gym	Admin123!
Εκπαιδευτής (Instructor)	instructor@demo.gym	Demo1234!
Μέλος (με ενεργή συνδρομή)	member3@demo.gym	Demo1234!
Μέλος	member@demo.gym	Demo1234!

Οι διευθύνσεις των υπηρεσιών δίνονται στον Πίνακα 22.

Πίνακας 22 — Διευθύνσεις των υπηρεσιών στην τοπική εκτέλεση.

Υπηρεσία	Διεύθυνση (URL)
Εφαρμογή πελατών (mobile-first)	http://localhost:4200
Εφαρμογή προσωπικού / διαχείρισης	http://localhost:4201
API και διαδραστική τεκμηρίωση (Scalar)	http://localhost:8080/scalar/v1
Papercut (προβολή απεσταλμένων email)	http://localhost:8025

Για γρήγορη δοκιμή: σύνδεση στην εφαρμογή προσωπικού (http://localhost:4201) ως admin@demo.gym για τη διαχείριση, ή στην εφαρμογή πελατών (http://localhost:4200) ως member3@demo.gym, ο οποίος έχει ήδη ενεργή συνδρομή και μπορεί άμεσα να κάνει κράτηση.

6.4 Δημιουργία και διαχείριση νέων χρηστών

Η εγγραφή νέων χρηστών γίνεται μόνο κατόπιν πρόσκλησης από διαχειριστή (δεν υπάρχει ανοιχτή δημόσια εγγραφή). Τα βήματα:

1. Σύνδεση στην εφαρμογή προσωπικού ως admin@demo.gym και επιλογή «Προσκλήσεις».

2. Δημιουργία πρόσκλησης με το email και τον ρόλο του νέου χρήστη (μέλος ή εκπαιδευτής). Παράγεται ένας σύνδεσμος εγγραφής.

3. Στο Docker, το email της πρόσκλησης δεν φεύγει προς το διαδίκτυο, ο σύνδεσμος εμφανίζεται στο Papercut (http://localhost:8025) και στην ίδια την οθόνη προσκλήσεων.

4. Ο νέος χρήστης ανοίγει τον σύνδεσμο, ορίζει κωδικό και ολοκληρώνει την εγγραφή.

Σημείωση: ένας νέος χρήστης δεν έχει συνδρομή εξ ορισμού, επομένως δεν μπορεί να κάνει κράτηση μέχρι ο διαχειριστής να του αναθέσει μία μέσω της ενότητας «Συνδρομές». Για άμεση δοκιμή κράτησης χωρίς αυτό το βήμα, χρησιμοποιείται ο έτοιμος λογαριασμός member3@demo.gym.

6.5 Σταμάτημα και καθαρή εκκίνηση

Τερματισμός των υπηρεσιών:

```
docker compose -f docker/docker-compose.full-stack.yml down
```

Τερματισμός με ταυτόχρονη διαγραφή της βάσης, ώστε την επόμενη φορά να ξεκινήσει από καθαρή κατάσταση με νέα demo δεδομένα:

```
docker compose -f docker/docker-compose.full-stack.yml down -v
```

6.6 Εκτέλεση για ανάπτυξη (χωρίς Docker)

Για την ανάπτυξη, το API και οι δύο εφαρμογές εκτελούνται απευθείας στον υπολογιστή, ενώ το Docker χρησιμοποιείται μόνο για τη βάση δεδομένων και τον τοπικό εξυπηρετητή email. Από τον ριζικό φάκελο του project εκτελούνται, σε ξεχωριστά τερματικά:

```
# 1. Βάση δεδομένων (PostgreSQL 16) και Papercut
docker compose -f docker/docker-compose.infra.yml up -d

# 2. API (εφαρμόζει αυτόματα τα migrations και τα demo δεδομένα)
cd backend
dotnet run --project src/GymBooking.Api

# 3. Εφαρμογές (από τον φάκελο frontend)
cd frontend
npm ci
npx nx serve customer
npx nx serve staff --port 4201
```

Το API ακούει στη διεύθυνση http://localhost:5076 (τεκμηρίωση στο /scalar/v1), η εφαρμογή πελατών στη http://localhost:4200, η εφαρμογή προσωπικού στη http://localhost:4201 και το Papercut στη http://localhost:8080. Οι αυτοματοποιημένες δοκιμές εκτελούνται με τις παρακάτω εντολές (το Docker Desktop πρέπει να λειτουργεί, επειδή ορισμένες δοκιμές χρησιμοποιούν πραγματική PostgreSQL), ενώ τα πειράματα του κεφαλαίου 5 επαναλαμβάνονται με το script evaluation/run-evaluation.ps1:

```
cd backend
dotnet test

cd frontend
npx nx run-many -t lint test build
```

6.7 Μεταβλητές περιβάλλοντος

Οι ρυθμίσεις του API δίνονται στα αρχεία appsettings ανά περιβάλλον και μπορούν να αντικατασταθούν από μεταβλητές περιβάλλοντος (Πίνακας 23). Στην τοπική εκτέλεση με Docker ορίζονται ήδη στο docker/docker-compose.full-stack.yml με δοκιμαστικές τιμές, σε εγκατάσταση παραγωγής ορίζονται ως μεταβλητές περιβάλλοντος του διακομιστή, και οι πραγματικές τιμές τους (κωδικός της βάσης, κλειδί υπογραφής) δεν αποθηκεύονται στο αποθετήριο. Στις εφαρμογές Angular, η διεύθυνση του API ορίζεται κατά το build στα αρχεία environment κάθε εφαρμογής.

Πίνακας 23 *— Μεταβλητές περιβάλλοντος του API.*

Μεταβλητή	Σκοπός	Παράδειγμα / τιμή
ASPNETCORE_ENVIRONMENT	Περιβάλλον εκτέλεσης	Development (τοπικά, Docker) · Production (παραγωγή)
ConnectionStrings__Default	Σύνδεση με την PostgreSQL	Host=postgres;Port=5432;Database=gymbooking;Username=gym;Password=…
Jwt__SigningKey	Κλειδί υπογραφής των JWT (≥ 32 χαρακτήρες)	Τυχαία τιμή, δεν δημοσιεύεται
Jwt__Issuer, Jwt__Audience, Jwt__AccessTokenExpiryMinutes	Στοιχεία και διάρκεια του token	GymBooking · GymBooking · 120
Cors__AllowedOrigins__0, __1	Επιτρεπόμενες διευθύνσεις των δύο εφαρμογών σε εγκατάσταση παραγωγής	https://… (τοπικά επιτρέπονται οι localhost:4200 και :4201)
Email__SmtpHost, Email__SmtpPort, Email__FromAddress	Εξυπηρετητής email για τις προσκλήσεις	papercut · 2525 · noreply@demo.gym
Invitations__RegisterUrlBase	Βάση του συνδέσμου εγγραφής στις προσκλήσεις	http://localhost:4200/register
Seed__AdminEmail, Seed__AdminPassword	Αρχικός διαχειριστής (δημιουργείται μόνο σε άδεια βάση)	admin@demo.gym · ορίζεται από όποιον εγκαθιστά
Seed__CancellationHours	Αρχικό περιθώριο ακύρωσης σε ώρες	24

6.8 Συχνά προβλήματα

- **«port is already allocated»:** κάποια από τις θύρες 4200, 4201, 8080, 8025, 5432 ή 2525 χρησιμοποιείται ήδη, για παράδειγμα από το stack ανάπτυξης της ενότητας 6.6. Λύση: τερματισμός της άλλης υπηρεσίας (π.χ. docker compose -f docker/docker-compose.infra.yml stop) και νέα εκτέλεση.
- **«Cannot connect to the Docker daemon»:** το Docker Desktop δεν λειτουργεί, πρέπει να ξεκινήσει πριν από την εκτέλεση.
- **Η σύνδεση αποτυγχάνει αμέσως μετά την εκκίνηση:** το API μπορεί να μην έχει ολοκληρώσει ακόμη τα migrations, αρκεί αναμονή λίγων δευτερολέπτων. Η κατάστασή του ελέγχεται στη διεύθυνση http://localhost:8080/health και τα logs του με την εντολή docker compose -f docker/docker-compose.full-stack.yml logs api.
- **«Ο λογαριασμός κλειδώθηκε προσωρινά» ή «Πάρα πολλές προσπάθειες»:** μετά από 5 λανθασμένες προσπάθειες ο λογαριασμός κλειδώνει για 15 λεπτά, ενώ μετά από 10 αιτήματα σύνδεσης μέσα σε ένα λεπτό απαιτείται αναμονή ενός λεπτού (ενότητα 4.7.1).
- **Νέος χρήστης δεν μπορεί να κάνει κράτηση:** δεν έχει ακόμη ενεργή συνδρομή, ο διαχειριστής την αναθέτει από την ενότητα «Συνδρομές» (ενότητα 6.4).

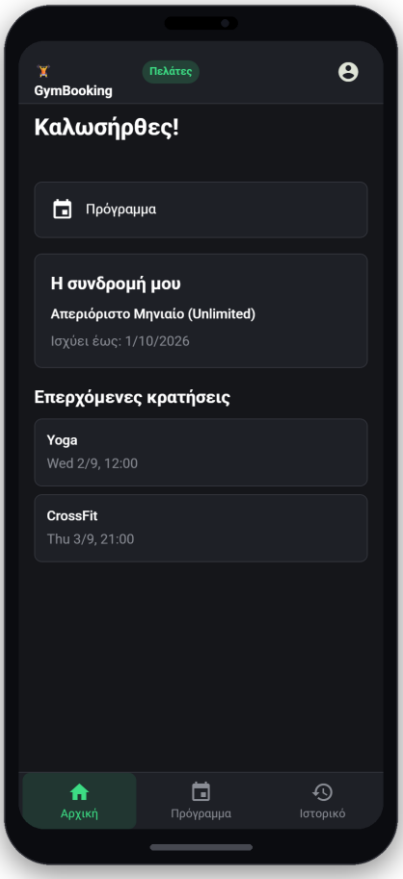
- **Καθαρή εκκίνηση και απεγκατάσταση:** η εντολή docker compose -f docker/docker-compose.full-stack.yml down -v διαγράφει και τη βάση, ώστε η επόμενη εκκίνηση να ξεκινήσει με νέα demo δεδομένα, με την παράμετρο --rmi local αφαιρούνται επιπλέον και τα images που δημιουργήθηκαν.

6.9 Χρήση της εφαρμογής

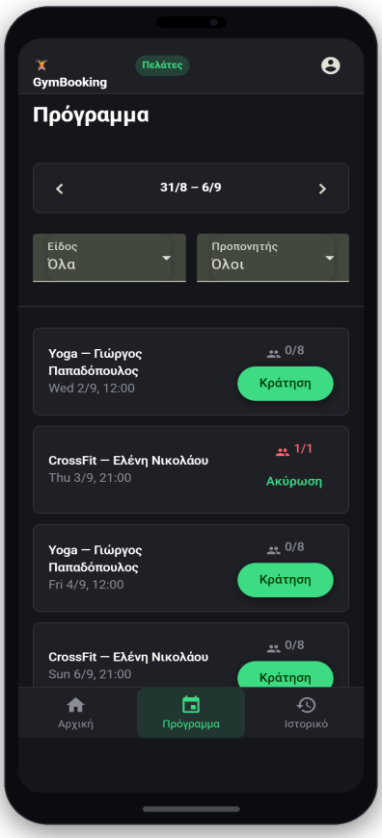
Οι λειτουργίες της εφαρμογής παρουσιάζονται εδώ όπως τις βλέπει ο χρήστης, με στιγμιότυπα οθόνης, χωρισμένες σε αυτές της εφαρμογής των πελατών και σε αυτές της εφαρμογής του προσωπικού. Η πλήρης ροή χρήσης, από άκρο σε άκρο, παρουσιάζεται επιπλέον στο βίντεο επίδειξης που συνοδεύει την εργασία (ενότητα 1.7).

6.9.1 Λειτουργίες πελάτη (customer app)

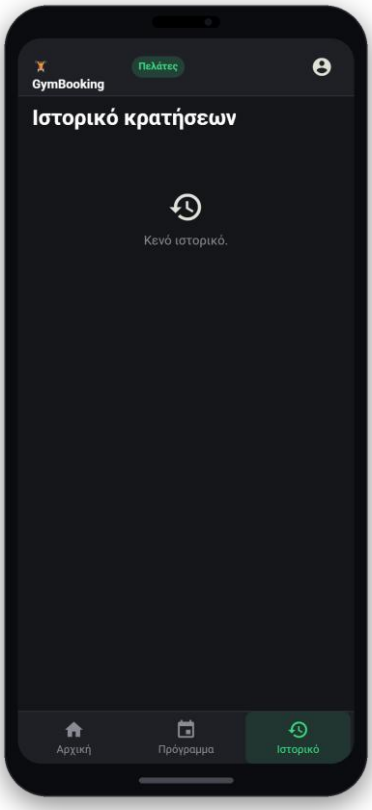
Η εφαρμογή των πελατών είναι σχεδιασμένη κυρίως για κινητό. Μετά τη σύνδεση, το μέλος βλέπει την αρχική οθόνη με τη συνδρομή του και τις επερχόμενες κρατήσεις (Εικόνα 10). Από το εβδομαδιαίο πρόγραμμα μπορεί να δει τα διαθέσιμα μαθήματα, να τα φιλτράρει και να κάνει ή να ακυρώσει κράτηση (Εικόνα 11), ενώ στο ιστορικό βλέπει τις προηγούμενες κρατήσεις του (Εικόνα 12).



Εικόνα 10 — *Αρχική οθόνη πελάτη: συνδρομή και επερχόμενες κρατήσεις.*



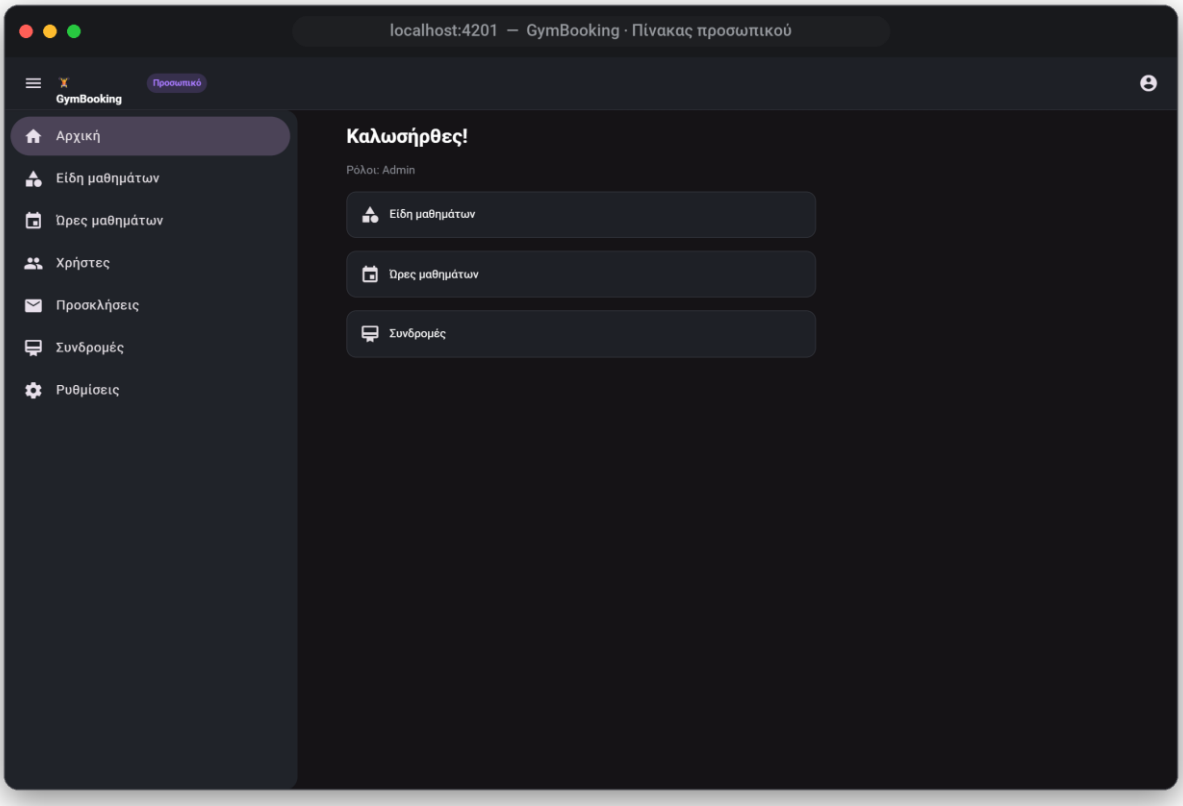
Εικόνα 11 — *Εβδομαδιαίο πρόγραμμα και κράτηση μαθήματος.*



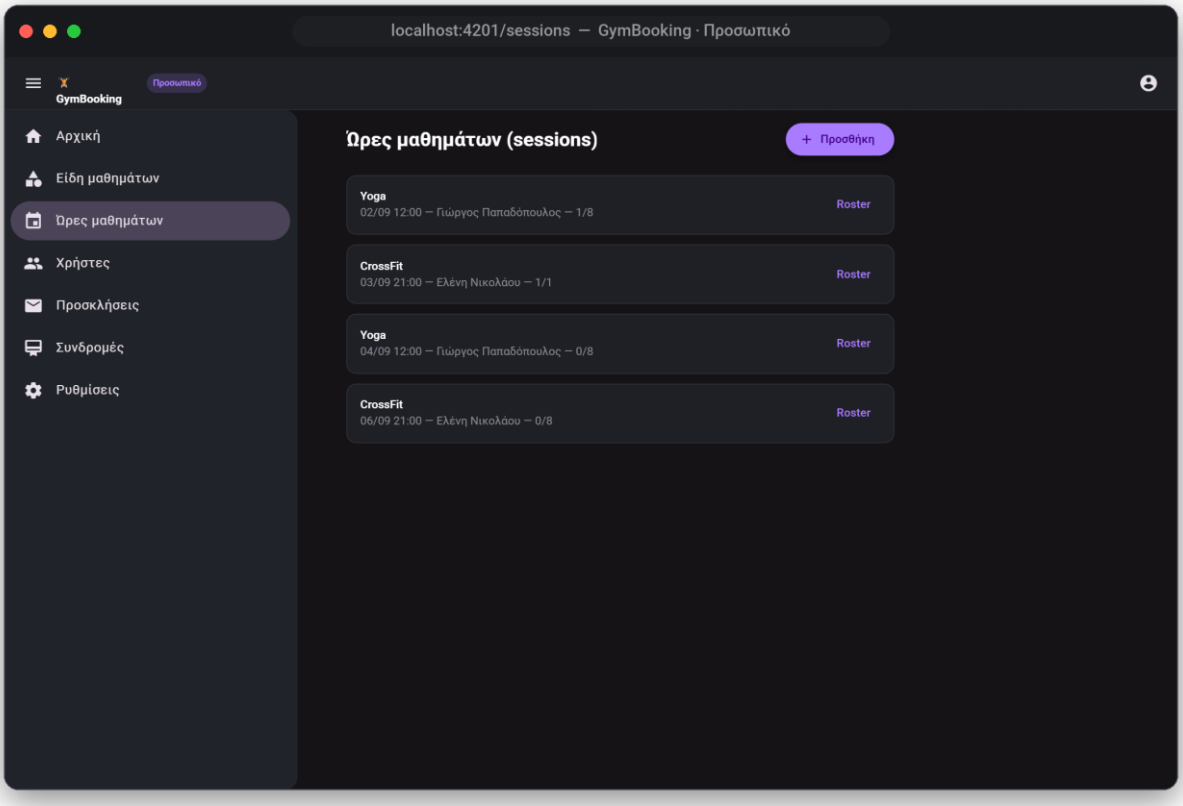
Εικόνα 12 — *Ιστορικό κρατήσεων πελάτη.*

6.9.2 Λειτουργίες προσωπικού και διαχειριστή (staff app)

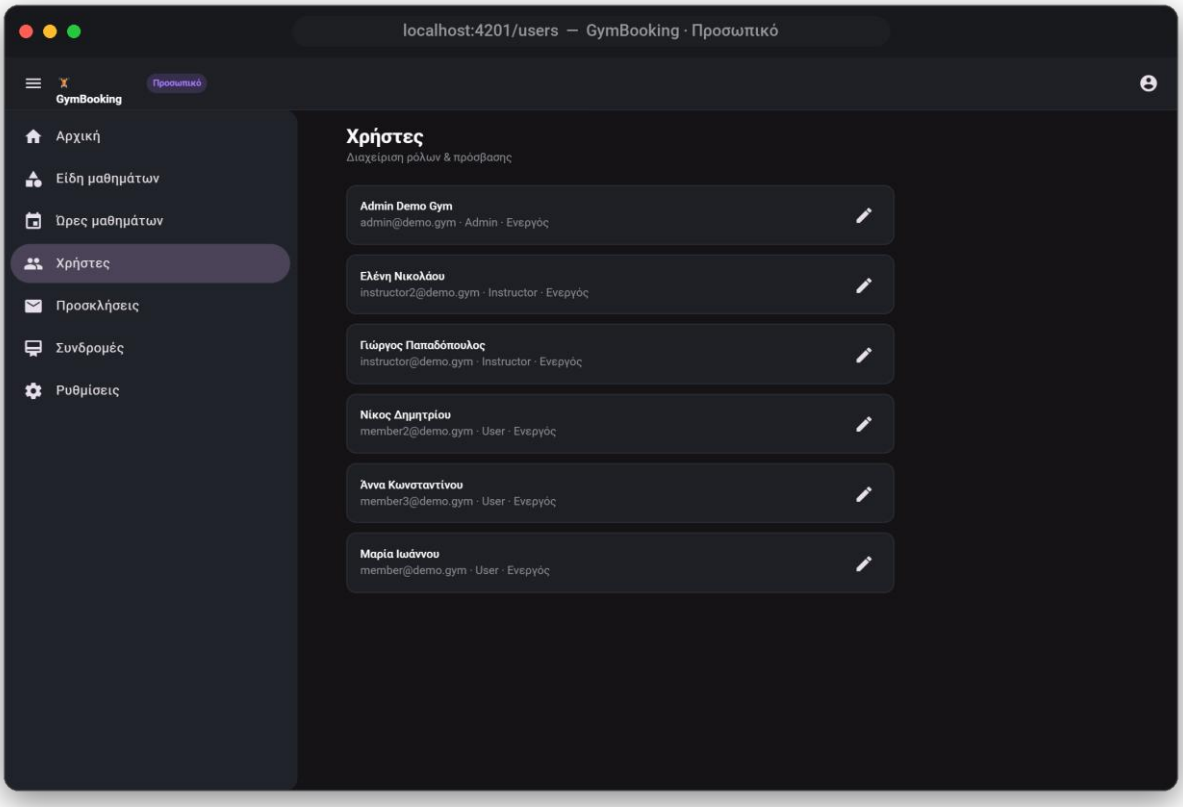
Η εφαρμογή του προσωπικού είναι σχεδιασμένη για υπολογιστή, με πλευρικό μενού πλοήγησης. Ο διαχειριστής έχει στη διάθεσή του έναν πίνακα με τις βασικές ενέργειες (Εικόνα 13), τη διαχείριση των ωρών των μαθημάτων με πρόσβαση στο παρουσιολόγιο (Εικόνα 14), καθώς και τη διαχείριση των χρηστών και των ρόλων τους (Εικόνα 15). Ανάλογα με τον ρόλο, εμφανίζονται και οι αντίστοιχες δυνατότητες, όπως η διαχείριση συνδρομητικών πλάνων και οι προσκλήσεις.



Εικόνα 13 — *Πίνακας προσωπικού (dashboard διαχειριστή).*



Εικόνα 14 *— Διαχείριση ωρών μαθημάτων και πρόσβαση στο παρουσιολόγιο.*



Εικόνα 15 *— Διαχείριση χρηστών και ρόλων.*

7. Συμπεράσματα και μελλοντική εργασία

Στο τελευταίο κεφάλαιο συνοψίζονται τα αποτελέσματα της εργασίας, οι περιορισμοί της, οι δυσκολίες που αντιμετωπίστηκαν και οι πιθανές μελλοντικές επεκτάσεις.

7.1 Συμπεράσματα

Η εργασία είχε ως στόχο τον σχεδιασμό, την υλοποίηση και την αξιολόγηση μιας multi-tenant εφαρμογής κρατήσεων γυμναστηρίου. Η εφαρμογή που αναπτύχθηκε καλύπτει όλες τις λειτουργικές απαιτήσεις του κεφαλαίου 3 (FR-01 έως FR-15), οι οποίες επαληθεύονται με 107 αυτοματοποιημένες δοκιμές. Ως προς τα ερευνητικά ερωτήματα:

- **ΕΕ1:** η εκτέλεση της κράτησης μέσα σε transaction με κλείδωμα της εγγραφής του μαθήματος αποκλείει το overbooking. Σε 30 γύρους, με έως 100 ταυτόχρονα αιτήματα για 5 θέσεις, δεν έγινε ούτε μία κράτηση πάνω από τις θέσεις. Αντίθετα, η απλή υλοποίηση δέχτηκε όλα τα αιτήματα και εμφάνισε το lost update που περιγράφει η βιβλιογραφία [13].
- ΕΕ2: το κλείδωμα επιβαρύνει τον χρόνο απόκρισης μόνο όταν οι χρήστες στοχεύουν το ίδιο μάθημα, και η επιβάρυνση αυξάνεται γραμμικά (περίπου 10 ms για κάθε επιπλέον ταυτόχρονο αίτημα). Με 25 ταυτόχρονους χρήστες το p95 ήταν 249 ms, ενώ με 50 έφτασε τα 538 ms, λίγο πάνω από το όριο των 500 ms, οπότε το NFR-02 ικανοποιείται οριακά σε αυτή την ακραία περίπτωση. Χωρίς ανταγωνισμό ο χρόνος έμεινε κάτω από 80 ms.
- **ΕΕ3:** ο διαχωρισμός των δεδομένων μέσω του αναγνωριστικού από το token και των καθολικών φίλτρων δεν επέτρεψε, σε κανένα από τα σενάρια που ελέγχθηκαν, πρόσβαση σε δεδομένα άλλου γυμναστηρίου.

Επομένως, για το μέγεθος ενός τυπικού γυμναστηρίου η απάντηση στο κεντρικό ερώτημα είναι θετική: η εφαρμογή αποκλείει το overbooking, διατηρεί αποδεκτό χρόνο απόκρισης (με την επιφύλαξη ότι με 50 ταυτόχρονες κρατήσεις στο ίδιο μάθημα το p95 φτάνει στο όριο) και απομονώνει τα δεδομένα, με τους περιορισμούς της ενότητας 7.2. Η σημαντικότερη αρχιτεκτονική επιλογή, το ρητό κλείδωμα, αποδείχθηκε απλή και αξιόπιστη, με γνωστό και μετρήσιμο κόστος, ενώ το αποτέλεσμα αποτελεί στέρεη βάση για εξέλιξη σε πραγματικό προϊόν.

7.2 Περιορισμοί

Τα αποτελέσματα πρέπει να ερμηνεύονται λαμβάνοντας υπόψη τους παρακάτω περιορισμούς:

- Η αξιολόγηση έγινε σε ένα μηχάνημα, με συνθετικά δεδομένα και με το API in-process, σε πραγματική εγκατάσταση οι χρόνοι απόκρισης θα διαφέρουν.
- Με 50 χρήστες στο ίδιο μάθημα, το p95 (538 ms στη διάμεση από 5 εκτελέσεις) ξεπερνά λίγο το όριο των 500 ms, για πολύ μεγαλύτερο ταυτοχρονισμό, η σειριακή εκτέλεση των κρατήσεων θα γινόταν εμπόδιο.
- Σε μια πολύ σπάνια περίπτωση, δύο ακυρώσεις σε διαφορετικά μαθήματα που προάγουν από τη λίστα αναμονής τους ίδιους χρήστες με αντίθετη σειρά μπορεί να οδηγήσουν σε deadlock. Η PostgreSQL το εντοπίζει και ακυρώνει το ένα transaction, το οποίο εκτελείται ξανά από το execution strategy.
- Δεν έγινε δοκιμή χρηστικότητας με πραγματικούς χρήστες ούτε πιλοτική χρήση σε γυμναστήριο.
- Στην εφαρμογή προσωπικού εντοπίστηκε ανεπαρκής αντίθεση χρωμάτων σε δύο στοιχεία (4,35:1 και 4,11:1, έναντι 4,5:1).
- Η ταυτοποίηση χρησιμοποιεί μόνο access token αποθηκευμένο στο localStorage, χωρίς refresh tokens και χωρίς δυνατότητα ανάκλησης (ΑΑ-3).

- Δεν υπάρχει ενεργή διαδικασία συνεχούς ενσωμάτωσης (CI), και η εφαρμογή δεν έχει εγκατασταθεί σε περιβάλλον παραγωγής.
- Εκτός εύρους έμειναν οι πληρωμές, οι ειδοποιήσεις και η ενημέρωση σε πραγματικό χρόνο (ενότητα 1.6).

7.3 Δυσκολίες που αντιμετωπίστηκαν

Οι κυριότερες δυσκολίες αφορούσαν τη σωστή διαχείριση των ταυτόχρονων κρατήσεων, ώστε να αποφεύγεται η υπέρβαση της χωρητικότητας, και την υλοποίηση της αυτόματης προαγωγής από τη λίστα αναμονής μέσα στο ίδιο transaction με την ακύρωση. Επίσης, το transaction της κράτησης χρειάστηκε να εκτελείται μέσα σε execution strategy, λόγω της αυτόματης επανάληψης του Npgsql, το πρόβλημα εμφανίστηκε μόνο σε πραγματική PostgreSQL και όχι στις δοκιμές με βάση στη μνήμη (ενότητα 4.6.1). Η αντιμετώπιση αυτών των ζητημάτων ήταν και η πιο διδακτική πλευρά της εργασίας.

7.4 Μελλοντικές επεκτάσεις

Η εφαρμογή μπορεί να επεκταθεί με αρκετούς τρόπους σε μελλοντικό στάδιο:

- Για μεγάλο ταυτοχρονισμό στο ίδιο μάθημα, ατομική ενημέρωση του μετρητή με μία εντολή UPDATE … WHERE BookedCount < Capacity ή ουρά κρατήσεων, με μέτρηση σε σύγκριση με την τρέχουσα λύση.
- Διόρθωση της αντίθεσης χρωμάτων και δοκιμή χρηστικότητας με μέλη και προσωπικό γυμναστηρίου.
- Εγκατάσταση σε περιβάλλον παραγωγής με HTTPS και παρακολούθηση της λειτουργίας.
- Ενίσχυση της ταυτοποίησης με refresh tokens και ασφαλέστερη διαχείριση της συνεδρίας.
- Ενημερώσεις σε πραγματικό χρόνο για τη διαθεσιμότητα των θέσεων, ώστε να ενημερώνονται άμεσα οι χρήστες.
- Ενσωμάτωση ηλεκτρονικών πληρωμών για την αγορά συνδρομών.
- Αυτοματοποιημένη διαδικασία ελέγχου και δημοσίευσης (CI/CD).
- Ειδοποιήσεις (email ή push) για υπενθυμίσεις μαθημάτων και προαγωγές από τη λίστα αναμονής.

Βιβλιογραφία

Οι πηγές παρατίθενται με τη σειρά πρώτης αναφοράς τους στο κείμενο, σύμφωνα με το σύστημα IEEE. Όλα τα διαγράμματα, τα γραφήματα και τα στιγμιότυπα οθόνης της εργασίας δημιουργήθηκαν από τον συγγραφέα.

[1] R. Krebs, C. Momm, and S. Kounev, “Architectural concerns in multi-tenant SaaS applications,” in *Proc. 2nd Int. Conf. Cloud Computing and Services Science (CLOSER)*, Porto, Portugal, 2012, pp. 426–431, doi: 10.5220/0003957604260431.

[2] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.

[3] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” IETF, RFC 7519, May 2015, doi: 10.17487/RFC7519.

[4] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[5] Microsoft, “Entity Framework Core documentation,” Microsoft Learn. Accessed: Sep. 29, 2026. [Online]. Available: https://learn.microsoft.com/ef/core/

[6] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. San Mateo, CA, USA: Morgan Kaufmann, 1993.

[7] P. A. Bernstein, V. Hadzilacos, and N. Goodman, *Concurrency Control and Recovery in Database Systems*. Reading, MA, USA: Addison-Wesley, 1987.

[8] Docker Inc., “Docker documentation.” Accessed: Sep. 29, 2026. [Online]. Available: https://docs.docker.com/

[9] F. Chong and G. Carraro, “Architecture strategies for catching the long tail,” Microsoft Corp., Redmond, WA, USA, Tech. Rep., 2006.

[10] Microsoft, “Architect multitenant solutions on Azure,” Microsoft Learn. Accessed: Sep. 29, 2026. [Online]. Available: https://learn.microsoft.com/azure/architecture/guide/multitenant/overview

[11] S. Aulbach, T. Grust, D. Jacobs, A. Kemper, and J. Rittinger, “Multi-tenant databases for software as a service: Schema-mapping techniques,” in *Proc. ACM SIGMOD Int. Conf. Management of Data*, Vancouver, BC, Canada, 2008, pp. 1195–1206, doi: 10.1145/1376616.1376736.

[12] C.-P. Bezemer and A. Zaidman, “Multi-tenant SaaS applications: Maintenance dream or nightmare?,” in *Proc. Joint ERCIM Workshop on Software Evolution and Int. Workshop on Principles of Software Evolution (IWPSE-EVOL)*, Antwerp, Belgium, 2010, doi: 10.1145/1862372.1862393.

[13] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O’Neil, and P. O’Neil, “A critique of ANSI SQL isolation levels,” in *Proc. ACM SIGMOD Int. Conf. Management of Data*, San Jose, CA, USA, 1995, pp. 1–10, doi: 10.1145/223784.223785.

[14] The PostgreSQL Global Development Group, “Explicit locking,” PostgreSQL Documentation. Accessed: Sep. 29, 2026. [Online]. Available: https://www.postgresql.org/docs/current/explicit-locking.html

[15] H. T. Kung and J. T. Robinson, “On optimistic methods for concurrency control,” *ACM Trans. Database Syst.*, vol. 6, no. 2, pp. 213–226, Jun. 1981, doi: 10.1145/319566.319567.

[16] D. R. K. Ports and K. Grittner, "Serializable snapshot isolation in PostgreSQL," *Proc. VLDB Endow.*, vol. 5, no. 12, pp. 1850–1861, Aug. 2012, doi: 10.14778/2367502.2367523.

[17] Y. Sheffer, D. Hardt, and M. Jones, "JSON Web Token best current practices," IETF, RFC 8725, Feb. 2020, doi: 10.17487/RFC8725.

[18] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.

[19] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA, USA: Prentice Hall, 2017.

[20] OWASP Foundation, "OWASP Top 10 – Web application security risks." Accessed: Sep. 29, 2026. [Online]. Available: https://owasp.org/www-project-top-ten/

[21] OWASP Foundation, "Authentication cheat sheet," OWASP Cheat Sheet Series. Accessed: Sep. 29, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

[22] Mermaid, "Mermaid – Diagramming and charting tool." Accessed: Sep. 29, 2026. [Online]. Available: https://mermaid.js.org/

[23] K. Kupersmith, P. Mulvey, and K. McGoey, "How to create use case description for your business analysis report," Dummies, Mar. 2016. Accessed: Oct. 1, 2026. [Online]. Available: https://www.dummies.com/article/business-careers-money/business/general-business/how-to-create-use-case-description-for-your-business-analysis-report-162468/

[24] S. Brown, "The C4 model for visualising software architecture." Accessed: Sep. 29, 2026. [Online]. Available: https://c4model.com/

[25] Microsoft, "Common web application architectures," Microsoft Learn. Accessed: Sep. 29, 2026. [Online]. Available: https://learn.microsoft.com/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures

[26] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA, USA: Addison-Wesley, 2003.

[27] Microsoft, "Introduction to Identity on ASP.NET Core," Microsoft Learn. Accessed: Sep. 29, 2026. [Online]. Available: https://learn.microsoft.com/aspnet/core/security/authentication/identity

[28] Google, "Angular documentation." Accessed: Sep. 29, 2026. [Online]. Available: https://angular.dev/

[29] Nx, "Nx documentation." Accessed: Sep. 29, 2026. [Online]. Available: https://nx.dev/

[30] Google, "Angular Material – Component library." Accessed: Sep. 29, 2026. [Online]. Available: https://material.angular.io/

[31] Tailwind Labs, "Tailwind CSS documentation." Accessed: Sep. 29, 2026. [Online]. Available: https://tailwindcss.com/docs

[32] Microsoft, ".NET documentation," Microsoft Learn. Accessed: Sep. 29, 2026. [Online]. Available: https://learn.microsoft.com/dotnet/

[33] Microsoft, "ASP.NET Core documentation," Microsoft Learn. Accessed: Sep. 29, 2026. [Online]. Available: https://learn.microsoft.com/aspnet/core/

[34] Serilog Contributors, "Serilog – Structured logging for .NET." Accessed: Sep. 29, 2026. [Online]. Available: https://serilog.net/



[35] Scalar, “Scalar – API reference for OpenAPI documents.” Accessed: Sep. 29, 2026. [Online]. Available: https://github.com/scalar/scalar

[36] The PostgreSQL Global Development Group, “PostgreSQL documentation.” Accessed: Sep. 29, 2026. [Online]. Available: https://www.postgresql.org/docs/

[37] Npgsql, “Npgsql – .NET access to PostgreSQL.” Accessed: Sep. 29, 2026. [Online]. Available: https://www.npgsql.org/

[38] Testcontainers, “Testcontainers for .NET.” Accessed: Sep. 29, 2026. [Online]. Available: https://dotnet.testcontainers.org/

[39] Google, “Introduction to Lighthouse,” Chrome for Developers. Accessed: Sep. 29, 2026. [Online]. Available: https://developer.chrome.com/docs/lighthouse/overview

[40] W3C, “Web Content Accessibility Guidelines (WCAG) 2.2,” W3C Recommendation, Oct. 2023. Accessed: Sep. 29, 2026. [Online]. Available: https://www.w3.org/TR/WCAG22/

Παράρτημα Α — Επισκόπηση των API endpoints

Το backend εκθέτει ένα REST API πάνω από HTTP (HTTPS σε εγκατάσταση παραγωγής), με ανταλλαγή δεδομένων σε μορφή JSON. Η ταυτοποίηση γίνεται με JWT (επικεφαλίδα Authorization: Bearer). Στη στήλη «Πρόσβαση»: «Ανώνυμος» = χωρίς σύνδεση, «Συνδεδεμένος» = οποιοσδήποτε χρήστης έχει συνδεθεί, ανεξάρτητα από τον ρόλο, «Instructor / Admin» και «Admin» = μόνο οι αντίστοιχοι ρόλοι. Διαδραστική τεκμηρίωση είναι διαθέσιμη μέσω Scalar/OpenAPI στη διαδρομή /scalar/v1.

Α.1 Ταυτοποίηση (authentication)

Πίνακας 24 — Endpoints ταυτοποίησης.

Μέθοδος & Διαδρομή	Λειτουργία	Πρόσβαση
`POST /auth/login`	Σύνδεση και έκδοση JWT	Ανώνυμος
`POST /auth/register?token=`	Εγγραφή μέσω πρόσκλησης	Ανώνυμος
`GET /auth/me`	Στοιχεία τρέχοντος χρήστη (id, tenant, ρόλοι)	Συνδεδεμένος

Α.2 Πρόγραμμα και μαθήματα

Πίνακας 25 — Endpoints προγράμματος και μαθημάτων.

Μέθοδος & Διαδρομή	Λειτουργία	Πρόσβαση
`GET /schedule`	Εβδομαδιαίο πρόγραμμα με φίλτρα (ημ/νία, είδος, εκπαιδευτής)	Συνδεδεμένος
`GET /class-sessions[/{id}]`	Προβολή συνεδριών μαθημάτων	Συνδεδεμένος
`POST /class-sessions`	Δημιουργία συνεδρίας	Instructor / Admin
`POST /class-sessions/{id}/cancel`	Ακύρωση συνεδρίας	Instructor / Admin
`GET /class-types[/{id}]`	Προβολή τύπων μαθήματος	Συνδεδεμένος
`POST /class-types`	Δημιουργία τύπου μαθήματος	Instructor / Admin
`PUT /class-types/{id}`	Ενημέρωση τύπου μαθήματος	Instructor / Admin
`DELETE /class-types/{id}`	Απενεργοποίηση τύπου μαθήματος	Instructor / Admin
`GET /instructors`	Λίστα εκπαιδευτών	Συνδεδεμένος

Α.3 Κρατήσεις και λίστα αναμονής

Πίνακας 26 — Endpoints κρατήσεων και λίστας αναμονής.

Μέθοδος & Διαδρομή	Λειτουργία	Πρόσβαση
`GET /bookings/me`	Οι κρατήσεις μου	Συνδεδεμένος
`POST /bookings`	Κράτηση θέσης (atomic transaction)	Συνδεδεμένος
`POST /bookings/{id}/cancel`	Ακύρωση δικής μου κράτησης	Συνδεδεμένος
`GET /sessions/{id}/roster`	Παρουσιολόγιο συνεδρίας	Instructor / Admin
`POST /sessions/{id}/bookings`	Κράτηση πελάτη από το προσωπικό	Instructor / Admin
`DELETE /bookings/{id}/staff`	Ακύρωση κράτησης από το προσωπικό	Instructor / Admin
`POST /sessions/{id}/waitlist`	Είσοδος σε λίστα αναμονής	Συνδεδεμένος
`DELETE /sessions/{id}/waitlist`	Έξοδος από λίστα αναμονής	Συνδεδεμένος
`GET /waitlist/me`	Οι λίστες αναμονής μου	Συνδεδεμένος

Α.4 Συνδρομές και πλάνα

Πίνακας 27 — Endpoints συνδρομών και συνδρομητικών πλάνων.

Μέθοδος & Διαδρομή	Λειτουργία	Πρόσβαση
`GET /subscriptions/me`	Η ενεργή συνδρομή μου	Συνδεδεμένος
`POST /subscriptions`	Ανάθεση συνδρομής σε χρήστη	Admin
`GET /membership-plans`	Λίστα συνδρομητικών πλάνων	Admin
`POST /membership-plans`	Δημιουργία πλάνου	Admin
`DELETE /membership-plans/{id}`	Απενεργοποίηση πλάνου	Admin

Α.5 Διαχείριση χρηστών, προσκλήσεων και ρυθμίσεων

Πίνακας 28 — Endpoints διαχείρισης χρηστών, προσκλήσεων και ρυθμίσεων.

Μέθοδος & Διαδρομή	Λειτουργία	Πρόσβαση
`GET /users`	Λίστα χρηστών	Admin
`PUT /users/{id}/role`	Αλλαγή ρόλου χρήστη	Admin
`PUT /users/{id}/activate | /deactivate`	Ενεργοποίηση / απενεργοποίηση χρήστη	Admin
`POST /invitations`	Δημιουργία πρόσκλησης (μέλος ή εκπαιδευτής) και συνδέσμου εγγραφής	Admin
`GET /tenant/settings`	Ρυθμίσεις γυμναστηρίου	Admin
`PUT /tenant/settings`	Ενημέρωση ρυθμίσεων	Admin

Παράρτημα Β — Συντομογραφίες

Πίνακας 29 — Συντομογραφίες.

Συντομογραφία	Επεξήγηση
API	Application Programming Interface — διεπαφή προγραμματισμού εφαρμογών
REST	Representational State Transfer — αρχιτεκτονικό στυλ για web υπηρεσίες
JWT	JSON Web Token — υπογεγραμμένο token ταυτοποίησης
ORM	Object-Relational Mapping — αντιστοίχιση αντικειμένων σε σχεσιακή βάση
EF Core	Entity Framework Core — ο ORM του .NET
SPA	Single-Page Application — εφαρμογή μίας σελίδας
CORS	Cross-Origin Resource Sharing — μηχανισμός πρόσβασης μεταξύ διαφορετικών origins
CRUD	Create, Read, Update, Delete — βασικές λειτουργίες δεδομένων
ERD	Entity-Relationship Diagram — διάγραμμα οντοτήτων–συσχετίσεων
DTO	Data Transfer Object — αντικείμενο μεταφοράς δεδομένων
SaaS	Software as a Service — λογισμικό ως υπηρεσία
SQL	Structured Query Language — γλώσσα ερωτημάτων βάσεων δεδομένων
UTC	Coordinated Universal Time — συντονισμένη παγκόσμια ώρα
SMTP	Simple Mail Transfer Protocol — πρωτόκολλο αποστολής email
GUID	Globally Unique Identifier — καθολικά μοναδικό αναγνωριστικό
UI	User Interface — διεπαφή χρήστη

Παράρτημα Γ — Δήλωση χρήσης εργαλείων τεχνητής νοημοσύνης

Στο παράρτημα αυτό περιγράφεται πού και πώς χρησιμοποιήθηκαν εργαλεία παραγωγικής τεχνητής νοημοσύνης στην εργασία. Το AI χρησιμοποιήθηκε ως βοηθητικό εργαλείο και δεν θεωρείται συγγραφέας της εργασίας. Την ευθύνη για όλο τον κώδικα και όλο το κείμενο την έχει ο συγγραφέας.

Εργαλείο

Χρησιμοποιήθηκε το Claude Code της Anthropic, ένα εργαλείο που βοηθάει στον προγραμματισμό και δουλεύει απευθείας πάνω στα αρχεία του project. Το Claude Code χρησιμοποιεί τα μοντέλα Claude Opus και Claude Sonnet. Η χρήση έγινε από τον Ιούνιο έως τον Οκτώβριο του 2026.

Χρήση στον κώδικα

- Γραφή επαναλαμβανόμενου κώδικα, σε ορισμένες περιπτώσεις.
- Code review και εντοπισμός bugs.
- Γραφή των αυτόματων tests, για το backend και το frontend.
- Γραφή του κώδικα για τα evaluations του κεφαλαίου 5, δηλαδή τα πειράματα και τις μετρήσεις.

Χρήση στο κείμενο της εργασίας

- Δημιουργία των διαγραμμάτων, με βάση τον πραγματικό κώδικα της εφαρμογής.
- Αξιολόγηση (κεφάλαιο 5): σχεδιασμός και εκτέλεση των μετρήσεων, και επεξεργασία των αποτελεσμάτων.
- Μορφοποίηση του τελικού κειμένου σύμφωνα με τις οδηγίες συγγραφής (στυλ, επικεφαλίδες, λεζάντες, πίνακες).
- Παραπομπές στη βιβλιογραφία και μορφοποίηση των πηγών κατά IEEE.
- Δημιουργία του πίνακα περιεχομένων και των καταλόγων εικόνων, διαγραμμάτων, πινάκων και κωδίκων.
- Έλεγχος και διόρθωση της σύνταξης και της γραμματικής, όπου χρειαζόταν.

Τρόπος ελέγχου

- Όλος ο κώδικας που προτάθηκε από το AI ελέγχθηκε από τον συγγραφέα πριν μπει στο project. Τα commits στο repository έγιναν μόνο από τον συγγραφέα.
- Τα tests εκτελούνται και περνούν όλα: 81 στο backend και 26 στο frontend.
- Τα διαγράμματα συγκρίθηκαν με τον κώδικα, ώστε να δείχνουν αυτό που κάνει πραγματικά η εφαρμογή.
- Οι μετρήσεις του κεφαλαίου 5 μπορούν να εκτελεστούν ξανά με τα scripts που υπάρχουν στο repository. Έτσι όποιος θέλει μπορεί να ελέγξει τα αποτελέσματα.
- Οι βιβλιογραφικές πηγές και οι παραπομπές ελέγχθηκαν μία-μία.

Δεδομένα

Στο AI δεν δόθηκαν προσωπικά ή εμπιστευτικά δεδομένα. Χρησιμοποιήθηκε μόνο ο κώδικας του project και δοκιμαστικά (demo) δεδομένα που δημιουργήθηκαν για την εργασία.